STAT 3080: From Data to Knowledge

Jeffrey Woo

2026-10-15

Contents

Preface **7**
Course Overview and Goals . . . 7
Background Needed for the Course . . . 8
Required Software . . . 8
Sections . . . 8
Reporting Issues with Course Notes . . . 9

1 R Basics **11**
1.1 Overview of R . . . 11
1.2 Commands . . . 11
1.3 Working Directory . . . 12
1.4 R Scripts . . . 14
1.5 Basic Math Operations . . . 14
1.6 Assignments and Variables . . . 14
1.7 Functions . . . 16
1.8 Packages . . . 18
1.9 Types, Data Structures, Classes . . . 19
1.10 Other Practical Advice with R . . . 26

2 R Markdown **29**
2.1 Why Use R Markdown . . . 29
2.2 Getting Started . . . 30
2.3 Cross-Referencing with R Markdown . . . 33
2.4 Knitting Your Document . . . 33
2.5 Things to Consider . . . 34

3 Working with Data Structures **37**
3.1 Motivation . . . 37
3.2 Subsetting Data Structures . . . 37
3.3 Loading Data Sets in R . . . 46
3.4 Math Operations on Data Structures . . . 48
3.5 Applying Functions on Data Structures . . . 49

4 Functions **51**
4.1 Motivation . . . 51
4.2 Basic Syntax . . . 51
4.3 Returning a Value . . . 52
4.4 Arguments and Default Values . . . 52
4.5 Argument Validation . . . 54
4.6 Return Multiple Objects . . . 54
4.7 Scope . . . 55

5 Control Flow **57**
5.1 Motivation . . . 57
5.2 Branching . . . 57
5.3 Iterating . . . 60
5.4 Vectorization in R . . . 62

6 Simulations **63**
6.1 Motivation . . . 63
6.2 Sampling . . . 64
6.3 Replicate . . . 65
6.4 Simulation . . . 65
6.5 Random Seeds . . . 69

7 Apply Family **71**
7.1 Overview . . . 71
7.2 `apply()` . . . 71
7.3 `tapply()` . . . 72
7.4 `lapply()` and `sapply()` . . . 72
7.5 `sapply()` . . . 75

8 Data Visualizations (ggplot2) **77**
8.1 Introduction . . . 77
8.2 Visualizing Two Quantitative Variables . . . 78
8.3 Visualizing Two Categorical Variables . . . 86
8.4 Visualizing a Quantitative Variable Across a Categorical Variable 89
8.5 Adding an Additional Categorical Variable . . . 92
8.6 Visualizing Single Variables . . . 93
8.7 Cheat Sheet . . . 97
8.8 Visualizations from Simulations . . . 97
8.9 ggplot vs Base R . . . 99

9 Data Wrangling (dplyr) **101**
9.1 Motivation . . . 101
9.2 Operations on Rows . . . 103
9.3 Operations on Columns . . . 107
9.4 Pipes . . . 112
9.5 Grouping . . . 113

10 Exploratory Data Analysis and Tidying Data 117
10.1 Introduction . . . 117
10.2 EDA . . . 117
10.3 Tidy Data . . . 132

11 Sampling Distributions and the Central Limit Theorem 133
11.1 Key Ideas and Terms . . . 133
11.2 Monte Carlo Simulations . . . 137
11.3 Sampling Distribution of the Sample Mean . . . 139
11.4 Central Limit Theorem . . . 141
11.5 Sampling Distribution of the Sample Proportion . . . 145
11.6 Other Common Sampling Distributions . . . 146
11.7 Sampling Distribution of Other Statistics . . . 147

12 Confidence Intervals 149
12.1 Introduction . . . 149
12.2 Confidence Interval for the Mean . . . 150
12.3 Confidence Interval for the Proportion . . . 162
12.4 Confidence Interval for Difference in Two Independent Means . . . 164
12.5 Confidence Interval for Difference in Two Independent Proportions 165

13 The Bootstrap 167
13.1 Introduction . . . 167
13.2 The Bootstrap . . . 167
13.3 Parametric Vs Non-Parametric Methods . . . 172

14 Hypothesis Testing for Means 173
14.1 Introduction . . . 173
14.2 Hypothesis Test for the Mean . . . 174
14.3 Hypothesis Test for Paired Means . . . 186
14.4 Hypothesis Test for Difference in Two Independent Means . . . 188
14.5 Additional Comments on Hypothesis Test of Parameters . . . 190

15 Type I and II Errors, Significance Level, Power 193
15.1 Introduction . . . 193
15.2 Type I and Type II Errors . . . 193
15.3 Significance Level and Type I Error . . . 194
15.4 Power and Type II Error . . . 196
15.5 Statistical Vs Practical Significance . . . 201

Preface

Course Overview and Goals

This course focuses on a simulation-based inference approach to concepts taught in elementary statistics. It focuses on using simulations to investigate the operating characteristics of common inferential procedures in statistics, so you can make practical decisions on when to use these procedures to learn about the data at hand.

For example, you may have heard that a hypothesis test for the mean from one sample assumes the data come from a normal distribution, or that the sample size is "large enough". You may have wondered what constitutes being "large enough", and you may have learned or read that a sample size of 25 or 30 will be considered "large enough". The true answer is actually "it depends", as various other factors play a role.

You will learn to design and run simulations to develop a more nuanced understanding of how reliable these inferential procedures are when the assumptions they are based on are not met. It turns out that there are mathematical proofs that can answer such questions, but they are usually fairly complicated, so we will use simulations in place of these mathematical proofs.

In order to design such simulations, a fair amount of knowledge with coding using R will be needed, which you will learn in the first portion of this course. This course is not a programming course; it is simulation-based inference using R.

Upon successful completion of this course, you will be able to:

1. Use simulation to study operating characteristics of statistical methodologies.
2. Understand and apply appropriate statistical methodologies under various data scenarios.
3. Use simulation to estimate statistical quantities which are difficult to compute.
4. Use simulation to assess the quality of statistical estimators.

5. Communicate results from simulations in writing, based on principles of reproducible research.

Background Needed for the Course

The official prerequisites are:

1. Intro statistics (One of STAT 1100, STAT 1120, STAT 2020, STAT 2120, STAT 3120, APMA 3110, APMA 3120).
2. Intro programming (One of STAT 1601, STAT 1602, STAT 3250, CS 1110, CS 1111, CS 1112, CS 1113).

The course is presented assuming you are familiar with inferential procedures associated with one- or two-sample means and proportions, as well as basic programming skills. The course will not cover these.

Required Software

The software introduced and featured in this course is R. RStudio is a software that runs R with additional user-friendly features. RStudio will be used for all in-class R demonstrations. Both R and RStudio are free for download.

- R will need to be downloaded first.
- Then downloaded RStudio.

You will also need to use R Markdown for your homeworks. You will upload an .Rmd file onto Gradescope and an autograder will grade your work. Some questions are free response and will be manually graded. R Markdown uses a type-setting platform called LaTeX. LaTeX can be installed by running the following two commands in the R Console:

```
install.packages("tinytex")
tinytex::install_tinytex()
```

You will also be using R Markdown to produce PDF documents for your project. Some basics of using R Markdown are covered in Section 2.

Sections

The sections for the book is as follows:

- Sections 1 to 7 focus on core R skills.
- Sections 8 to 10 focus on the `tidyverse`, which are a set of packages that are useful when working with data.
- Sections 11 to 19 focus on simulation-based inference.
- Section 20 uses simulations to explore key concepts in evaluating statistical estimators and models.

Reporting Issues with Course Notes

If you find any issues (typos, formatting, etc), please report them at my GitHub page. Please be as specific as you can, by specifying the section and paragraph where the issue is found.

Serious issues that need my immediate attention should be posted on our Canvas page on Piazza or emailed to me.

Chapter 1

R Basics

1.1 Overview of R

R is an open-source programming language and software environment specifically designed for statistical computing, data analysis, and graphical visualization. It is widely used by data scientists, statisticians, and researchers to clean, manipulate, and model data. R is maintained by volunteers. All users can access and modify source code.

While R has a basic default interface, users typically write code using integrated development environments (IDEs) such as RStudio, Positron, VS Code, which can provide additional user-friendly features. RStudio will be used in this course.

1.1.1 RStudio Interface

Open RStudio. You will see three panes:

- **Console** (left): where commands are executed
- **Environment / History** (top-right:) objects you have created and the commands you have run.
- **Files / Plots / Packages / Help** (bottom-right): files in working directory, plots created, packages installed, help page.

We will focus on the console for the time being.

1.2 Commands

In order to get R to do what you want it to do, you will need to write a **command**. R is ready for a new command when you see a `>` in the **console**. If your command is incomplete, R will show a `+` instead of a `>`. You can either

complete the command after the `+` or use the ESC key to abort. You will also see the output of your command (if any) in the console. For example:

```
1+2
```

```
## [1] 3
```

Notice a number in brackets before the answer? The number indicates index position of the first element displayed on that specific line.

R also ignores white space between components of a command, so typing `1+2` is the same as typing `1 + 2`.

```
1 + 2
```

```
## [1] 3
```

You can also add **comments** to R code by using the `#` sign. Everything to the right of the `#` is ignored by R. Comments are useful to explain what has been coded and why to yourself or to collaborators.

```
1+2 ## what is the sum of 1 and 2?
```

```
## [1] 3
```

I **highly discourage** you to enter your code directly into the R console. Code typed into the console is difficult to be saved or edited. Any meaningful work that you will use R for will likely need to be saved and edited.

1.3 Working Directory

Before talking about a better way to enter R commands, you will need to set up a **working directory**. The working directory is the path to the folder on your system (typically, the computer you are using) to store files generated during an R session and any files that you save. This is also the folder where R will search for files.

1.3.1 Finding the working directory

Typing `getwd()` will display the path of the working directory:

```
getwd()
```

```
## [1] "C:/Users/jeffr/Dropbox/UVA/book_stat3080/stat3080"
```

1.3.2 Changing the working directory

To change the working directory, use `setwd()`, with the path place as an argument:

```
setwd("C:/Users/jeffr/Dropbox/UVA/book_stat3080/stat3080/HWs")
```

I do not recommend using `setwd()`:

- Typing the path is fairly cumbersome.
- Your code will not run if you run the code on another computer (e.g. when sharing with a collaborator).

Therefore, I prefer to use RStudio Projects.

1.3.3 RStudio Projects

An **RStudio Project** is a folder with a `.Rproj` file that in it. There are a couple of ways to create an RStudio Project.

1. If you have never created one before:

- Open RStudio.
- Click on File, then New Project.
- A few options will be presented. The options that you will most likely use are:
  - New Directory: use this to start in a fresh folder.
  - Existing Directory: use this to work in an existing folder.
- If you are a creating a new directory, select New Project. Then you will enter the name of your project and choose a path for the folder.

If you open the folder associated with the created working directory, you will find a file with a `.Rproj` extension created (its name should be the name of the project that you gave in the last step above).

Now, close your RStudio. Navigate to the project folder on your computer and double-click the `.Rproj` file. Your working directory will be where this `.Rproj` file is located. You can confirm using `getwd()`, or by looking at the pane called "Files / Plots / Packages / Help". Click on the files tab and it will display the path of your working directory, and you can also see the other files in this folder.

2. If you have created an RStudio Project before and have a file with a `.RProj` extension, you can just copy and paste this file and place it into another folder to make it your working directory. This is the approach I almost exclusively use.

I highly recommend that you create a folder on your computer that is exclusively for this class. You should also create separate subfolders for each assignment.

Key idea: The working directory is the folder on your computer where all files that R generates will be saved and where R will search for files.

Practice: create an RStudio Project on your own.

1.4 R Scripts

You should type R commands in an R **script** instead of the R console. Commands can be saved and edited in an R script.

To create a new R script, click on File, New File, R Script. You will now have a new pane on the top-left with a tab labeled with a name similar to Untitled. You have created an untitled R script.

An R script looks like a notepad. Type your commands into this script. Each command should be a separate line. To execute commands, highlight the commands, and then click on Run or press Ctrl + Enter.

To save your R script, click on the Save icon and name your script. You have now saved an R Script, and it should appear in your working directory.

To open a saved R script, open the `.Rproj` file in the working directory, and you can click on the script under the tab called Files.

1.5 Basic Math Operations

R can be used as a calculator by using the standard arithmetic operators:

- `+`: addition
- `-`: subtraction
- `*`: multiplication
- `/`: division
- `^`: power

R is like any calculator in that these operators follow the standard order of operations. Some examples:

```
15+10
12/6
0.7*5
3^3
15+10*3
(15+10)*3
```

1.6 Assignments and Variables

In order to write useful and effective R code, You need to be able to store items to be used later. Anything that is stored is called an object. An object can be a single value resulting from a calculation or a collection of information such as a table of linear regression coefficients.

A **variable** is a memory location for the object, which can be thought of as a label for the object. Variables are created when the object is **assigned** to it, so there is no need to declare the variable first.

Note: the term variable as defined above comes from computer science. In statistics, the term variable means something different (any characteristic that can be measured).

Variable names can consist of letters, numbers, periods, and underscores. There are a few limitations to how these types of characters can be combined. The following are characteristics of all variable names:

1. Variable names cannot begin with a number.
2. Variable names cannot contain spaces.
3. Variable names can begin with a period as long as they are not followed by a number.
4. Since R is case-sensitive, `x` and `X` are different variable names. Wherever possible, it is best practice to avoid using the names of existing functions (ie. mean, sd, log) and predefined variables (ie. pi) to avoid confusion for anyone reading your code and possibly R itself.

It is also best practice to use variable names that are short and descriptive of the information contained in the object.

Once you have decided on a variable name for the object, you will need to assign the object to the variable by using either the assignment operator `<-` or `=` (the first is the more commonly used in R). I would discourage using `=` as an assignment operator as it is also used when specifying values for arguments in functions (see section 1.7.1.)

To store the sum of 10 and 5 and the rounded value of 5.7 (to whole number) into variables called `total1` and `value1` respectively:

```
total1 <- 10+15
value1 <- round(5.7,0)
```

Notice that these variables and their values are displayed in the environment tab in the pane at the top-right.

R will recall or show the object stored in a variable when the variable name is submitted as a command.

```
total1
```

```
## [1] 25
```

```
value1
```

```
## [1] 6
```

Stored objects can be used in subsequent operations. For example:

```
total1/5
```

```
## [1] 5
```

```
total1/value1
```

```
## [1] 4.166667
```

Any object that you would like stored needs to be given a variable name.

```
ratio1 <- total1/value1
```

Variables can be overwritten by storing a new object using the same variable name.

```
ratio1 <- total1/2
```

1.7 Functions

R has an extensive list of existing functions that can be used. Each function has a specific name that is followed by a set of parentheses. Using a function calls a specific collection of code to be executed. Generally the code that will be executed needs specific arguments or inputs to run correctly and the user provides these values in the parentheses that follow the function name.

Functions can be used for basic calculations, for complex procedures, and many other things in between. You can also write your own functions.

R is case-sensitive, so functions that you want to use must be typed exactly as it was defined.

Some common functions are listed below:

```
log(2.7) ## log with base e, sometimes written as ln
exp(1) ## e to power of 1
sqrt(9) ## squareroot of 9
factorial(3) ## 3!
abs(-5) ## absolute value
sum(2,4,6) ## summing up all these numbers
floor(5.7) ## round down to nearest integer
ceiling(5.7) ## round down to nearest integer
round(5.7) ## round to nearest integer
round(5.727,2) ##round to 2 decimal places
```

1.7.1 Documentation

Typing `?` and then the name of a function will open up the help page (called R documentation) associated with that function. The help page will appear under the Help tab in the bottom-right pane. Type `?log` to read the help page for the `log()` function.

The sections of the help page that I find useful:

- Description: describes what the function does.
- Usage: tells us what arguments need to be provided, and if arguments have defaults.
- Arguments: describes the arguments.
- Examples: some examples.

Practice: after reading the documentation for `log()`, use R to find the value of $\log_3(5)$.

Note: In the top-left corner of the help page, the name of the function is shown, along with the package it is from in braces. So the `log()` function comes a package called `base`.

1.7.2 Working with common probability distributions

Some common probability distributions (e.g. normal, binomial, etc.) are defined in R. Each distribution has a general function name, for example:

- norm: normal distribution
- binom: binomial distribution
- t: t distribution
- F: F distribution
- chisq: χ^2 distribution

Each distribution has 4 functions associated with it in R. The letter that precedes the distribution name tells R what specifically to do with that distribution. These letters are:

- `d`: gives the value of the probability density (or mass) function.
- `p`: gives the cumulative probability of the distribution at the point specified.
- `q`: determines the value within the distribution associated with the cumulative probability specified.
- `r`: generates random numbers from the distribution.

1.7.2.1 Example 1: Normal distribution

Suppose $X \sim N(1, 9)$, i.e. a normal distribution with mean 1 and variance 9.

- To find $P(X \leq 2)$, we use:

```
pnorm(2, mean=1, sd=3) ##note that SD is specified, not variance
```

```
## [1] 0.6305587
```

- To find the value of X associated with the 25th percentile:

```
qnorm(0.25, mean=1, sd=3)
```

```
## [1] -1.023469
```

- To simulate 10 random draws from X, we use:

```
rnorm(10, mean=1, sd=3)
```

```
##  [1]  0.9286397 -1.2022344  1.9727145 -2.8727213  6.7066808  1.5544307
##  [7] -0.9310952  7.0063098  0.7505008  0.9708035
```

Note: you are likely to get a vector of 10 numbers that are different than the ones shown.

1.7.2.2 Example 2: Binomial distribution

Suppose $Y \sim Bin(10, 0.6)$, i.e. a binomial distribution with $n = 10$ and $p = 0.6$.

- To find $P(Y = 3)$, we use:

```
dbinom(3, size=10, prob=0.6)
```

```
## [1] 0.04246733
```

To find $P(Y \leq 3)$, we use:

```
pbinom(3, size=10, prob=0.6)
```

```
## [1] 0.05476188
```

Practice: Suppose $S \sim \chi^2_5$, i.e. a chi-squared distribution with 5 degrees of freedom.

- Find $P(S > 0.80)$.
- What is the value of S that corresponds to the 90th percentile?

1.8 Packages

The functionality of R is broken up into various packages. There are a number of packages included in an R download, such as base, graphics, stats, utils. These packages are sometimes collectively called "base R packages". An R **package** is a collection of R functions, corresponding documentation, and datasets.

However, statistical methodologies advance, and additional functions beyond those that exist in base R packages are needed. Independent statisticians create packages to solve more modern problems and have contributed them for everyone to use. These packages are sometimes called "contributed packages".

In order to use what is in a contributed package, that package needs to be installed first, and then loaded. You do not need to install or load base R packages.

In order to install packages, you must have an internet connection. Packages only need to be installed once on a computer unless the version of R is updated. We use the function `install.packages()`. As an example, to install the package called `palmerpenguins`:

```
install.packages("palmerpenguins")
```

You can then load the package in order to able to use the functions from that package, by using the `library()` function:

```
library(palmerpenguins)
```

Key idea: packages only need to be installed once (unless you update R). Packages must be loaded prior to an R session using `library()`.

1.9 Types, Data Structures, Classes

In R, **type** refers to how an object is stored in memory. You can refer to this table for a list of types in R. To find the type of an R object, use the `typeof()` function:

```
typeof(total1)
```

```
## [1] "double"
```

Data structures are used to store and organize values. They are classified based on dimensionality and homogeneity.

- A single value is considered to have zero dimensions and is called a scalar.
- A data structure that is either a single column or a single row is considered to have one dimension.
  - If everything recorded in a one-dimensional data structure is of the same type, then it is called a **vector**.
  - If the items recorded in a one-dimensional data structure have varying types, then it is called a **list**.
- A data structure that contains multiple rows and multiple columns is considered to have two dimensions.
  - If the contents of a two-dimensional data structure are homogeneous, then it is called a **matrix**.
  - If the contents of a two-dimensional data structure are heterogeneous, then it is called a **data frame**.
- An **array** is a data structure that is homogeneous with one or more dimensions.

This information is summarized in Table 1.1 below:

Table 1.1: Classification of R Data Structures

Dimension	Homogeneous (Same Type)	Heterogeneous (Diff Type)
One	Vector	List
Two	Matrix	Data Frame
>2	Array	-

Data structures are created either when data are read in from external files or when data are entered manually. Next, we will see how to create them manually.

1.9.1 Vectors

The concatenate function, `c()`, is commonly used to create a vector. The inputs for this function are the data that you would like stored in the vector in the order in which you would like them stored. For example:

```
x <- c(5,10,15,20,25)
x
```

```
## [1]  5 10 15 20 25
```

```
y <- c(3,6,9,12,15)
y
```

```
## [1]  3  6  9 12 15
```

The concatenate function can also be used to add a value to a vector or to combine preexisting vectors.

```
x2 <- c(x,30)
x2
```

```
## [1]  5 10 15 20 25 30
```

```
z <- c(x,y)
z
```

```
##  [1]  5 10 15 20 25  3  6  9 12 15
```

It is important to remember that the order in which the objects are specified is the order in which they will be saved in the vector.

```
c(y,x)
```

```
##  [1]  3  6  9 12 15  5 10 15 20 25
```

1.9.1.1 Pattern vectors

There are times when you may need to create a vector that contains a sequence of numbers. There are a few ways to create this type of vector.

1. If you are wanting to create a vector that contains a sequence of consecutive integers, then you can use the syntax a:b to create a vector that starts at a and increases or decreases by one until b.

```
1:5
```

```
## [1] 1 2 3 4 5
```

```
5:1
```

```
## [1] 5 4 3 2 1
```

```
-1:-5
```

```
## [1] -1 -2 -3 -4 -5
```

2. If you are wanting a sequence where the values have a constant difference other than one, then you can use the sequence function, `seq()`. This function takes three arguments: from, to, by. These inputs specify the first and last number of the sequence as well as the difference between each value in the sequence.

```
seq(from=1, to=5, by=0.5)
```

```
## [1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0
```

There is another optional input for the sequence function: length. The length option specifies the length that you want your vector to be or how many values you want contained in the vector. The sequence function will use this value to determine a constant difference between each of the values that yields the number of values that you specify.

```
seq(1,5, length=9)
```

```
## [1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0
```

3. Another function that creates a useful type of vector is the replicate function, `rep()`. This function takes two arguments: vector and number of replicates. These specify what you would like R to repeat and how many times. Depending on the format of what you provide in the times option, this function will either repeat the given vector a specified number of times or repeat the elements of the vector a specified number of times.

```
rep(1:5, 2)
```

```
##  [1] 1 2 3 4 5 1 2 3 4 5
```

```
rep(1:5, c(1,2,3,2,1))
```

```
## [1] 1 2 2 3 3 3 4 4 5
```

```
rep(1:5, c(2,2,2,2,2))
```

```
##  [1] 1 1 2 2 3 3 4 4 5 5
```

```
rep(1:5, each=2)
```

```
##  [1] 1 1 2 2 3 3 4 4 5 5
```

1.9.2 Matrices

The `matrix()` function can be used to create a matrix. This function takes four arguments: vector, nrow, ncol, and byrow. These arguments instruct R what values to put in the matrix, how many rows and columns the matrix should have, and whether values should be assigned across the rows or down the columns. The default for the byrow option is false, meaning that the values will be assigned down the columns if the byrow option is not provided.

```
matrix(c(5,10,15,20,25,3,6,9,12,15), nrow=2, ncol=5, byrow=TRUE)
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    5   10   15   20   25
## [2,]    3    6    9   12   15
```

Another way to create a matrix is to use either the `cbind()` or `rbind()` function. Both of these functions use vectors as inputs. The `cbind()` function will bind these vectors together as columns and the `rbind()` function will bind these vectors together as rows.

```
rbind(x,y)
```

```
##   [,1] [,2] [,3] [,4] [,5]
## x    5   10   15   20   25
## y    3    6    9   12   15
```

```
cbind(x,y)
```

```
##       x  y
## [1,]  5  3
## [2,] 10  6
## [3,] 15  9
## [4,] 20 12
## [5,] 25 15
```

As with the concatenate function, R will bind the vectors together in the order you specify.

1.9.3 Classes

Before going into the remaining data structures, you need to learn what classes are. In R, **class** refers to how an object behaves with generic functions. There are 3 main classes in R: character, numeric, and logical.

- Character: Represents text strings, symbols, or words. Must always be enclosed in quotes.
- Numeric: Represents real numbers.
- Logical: Represents Boolean values for binary conditions.

The vectors and matrices created in sections 1.9.1 and 1.9.2 are numeric.

```
class(y)
```

```
## [1] "numeric"
```

To create a logical vector:

```
log_vect <- c(TRUE, FALSE, T, F)
## Note that T and F are equivalent to TRUE and FALSE resp
```

To create a character vector:

```
char_vect <- c("ByOne", "ByTwo", "ByThree", "ByFour", "ByFive")
```

The `class()` function can be used to find out the class of an object, for example:

```
class(log_vect)
```

```
## [1] "logical"
```

Alternatively, functions such as `is.character()`, `is.numeric()`, `is.logical()` can be used to test for the class:

```
is.numeric(log_vect)
```

```
## [1] FALSE
```

If a vector or matrix contains multiple types of data, an error message is unlikely to show up. R will modify the data to all be of the same type according to the following hierarchical structure:

1. character
2. numeric
3. logical

```
c(TRUE, 5)
```

```
## [1] 1 5
```

```
c(TRUE, "ByFive", 5)
```

```
## [1] "TRUE"   "ByFive" "5"
```

```
mult_mat <- cbind(char_vect,x,y)
mult_mat
```

```
##      char_vect x    y
## [1,] "ByOne"   "5"  "3"
## [2,] "ByTwo"   "10" "6"
## [3,] "ByThree" "15" "9"
## [4,] "ByFour"  "20" "12"
## [5,] "ByFive"  "25" "15"
```

1.9.3.1 Example 1: Mixing numeric with character

The most common kind of mixing of classes usually involve a data structure containing a mix of numeric and character, for example:

```
mixed <- c(1, 2, "3")
mixed ## notice all characters, since they are in quotes
```

```
## [1] "1" "2" "3"
```

```
class(mixed)
```

```
## [1] "character"
```

Trying to add the numbers in the variable `mixed` results in an error:

```
sum(mixed)
```

In this example, we can **coerce** all entries to become numeric:

```
mixed <- as.numeric(mixed) ## overwrite existing variable
sum(mixed)
```

```
## [1] 6
```

A situation like this example sometimes happens when working with an external data file and the person who entered the data made a mistake. This is a common reason why a mathematical operation fails as one character changes the entire vector to be character.

Note: Coercing an object to change will not always yield a result that makes sense.

1.9.3.2 Example 2: Math operations with logical

Interestingly, math operations can be performed on logicals. `TRUE` and `FALSE` are considered to be 1 and 0 respectively.

```
sum(log_vect)
```

```
## [1] 2
```

```
mean(log_vect)
```

```
## [1] 0.5
```

This will be extremely useful later on when working with simulations, as we often want to evaluate the proportion of times certain conditions are met.

1.9.4 Data Frames

Data frames are commonly used to store data. There is typically a row for each observation and a column for each variable measured on the observations

(think of an EXCEL spreadsheet). A data frame can be created using the `data.frame()` function. The arguments to this function are the vectors that you would like to be in the data frame.

```
mult_data <- data.frame(char_vect,x,y)
mult_data
```

```
##   char_vect  x  y
## 1     ByOne  5  3
## 2     ByTwo 10  6
## 3   ByThree 15  9
## 4    ByFour 20 12
## 5    ByFive 25 15
```

Notice you can also view the data frame by clicking on its name in the Environment tab in the top-right.

1.9.5 Lists

Lists are the most flexible type of object in R. Lists are similar to vectors in that there is one row containing multiple values. However, in a list, those values can be objects of varying types and sizes. A list is created using the list function, `list()`. The arguments specify what you would like in each element of the list.

```
mult_list <- list(mult_data, z, log_vect, 7)
mult_list
```

```
## [[1]]
##   char_vect  x  y
## 1     ByOne  5  3
## 2     ByTwo 10  6
## 3   ByThree 15  9
## 4    ByFour 20 12
## 5    ByFive 25 15
##
## [[2]]
##  [1]  5 10 15 20 25  3  6  9 12 15
##
## [[3]]
## [1]  TRUE FALSE  TRUE FALSE
##
## [[4]]
## [1] 7
```

There are testing and coercion functions for all of the types of data structures. As with the classes of data, coercion may not yield a valid result. Some examples are shown below:

```
is.vector(x)
as.vector(mult_list)
as.data.frame(mult_mat)
```

1.10 Other Practical Advice with R

1.10.1 The Workspace

When you are ready to exit R, you may be asked whether you want to save the workspace image into a `.RData` file. The workspace is not the same as the R script. It refers to the objects in the current environment. My recommendation is not to save the workspace.

This keeps your environment fresh the next time you open R. You will then re-run your R script to reproduce your results. Such practice is consistent with principles associated with **reproducible research**. An environment that contains objects from previous workspaces will interfere with the current workspace and must be avoided.

A situation where you may want to save the workspace is if your code takes a long time to run, and you want to avoid having to run your code again. This is unlikely to be a situation in this class.

To remove this prompt, go to Tools, then Global Options. Under the General tab on the left, and the Basic tab on top, uncheck all boxes listed under R Sessions, Workspace, and History. For the option Save workspace to .RData on exit, choose never.

If you have saved the workspace before, find the .RData file in your working directory, and delete it.

1.10.2 The Environment

Always check that your environment is empty (clear the environment) whenever you start working with R. This will ensure that previously declared variables will not influence the results from your code.

You should also clear the environment if you edited your code after executing it. You do not want previous mistakes or version of the code to influence your results.

To clear the environment to make it empty:

1. Execute the code `rm(list = ls())`, or
2. Click on the broom icon on the top-right under the Environment tab.

These will only delete user-created variables and functions, but do not unload packages or reset global options, which can be problematic if you are sharing the R script with others.

To work in a fully clean environment, start a new R session and be sure to never save or restore the R session.

Chapter 2

R Markdown

2.1 Why Use R Markdown

Imagine you are in this scenario: you have a homework assignment where you are supposed to produce data visualizations and answer some questions based on a data set using R. Your teacher needs to assess if you know what code to write and that you are providing your narrative on the R output. Therefore, you need to produce a document that show the following:

1. R code.
2. Relevant graphs.
3. Your interpretations of the graphs in narrative form.

These have to be presented in a cohesive and organized manner.

Providing an R script will not help, as it only shows your R code and you could provide your narrative as comments. However, your graphs are not shown. Your teacher does not want to produce the graphs by running your R code as it will take a lot of time to run code for all students in the class.

You might think of copying and pasting your R code, your graphs from the output, and type you narrative into a word document. This is doable but it will take you more time than you think, and the document is likely to look sloppy.

Is there a tool that will help you weave your code, R output, and your narrative into a neat document? This is what R Markdown does!

Note: R Markdown is not the only tool that allows you to do this. Quarto is another tool. R Markdown and Quarto can be thought of as variants of Markdown. Markdown is a language for formatting plain text. R Markdown and Quarto expand on Markdown.

2.1.1 Reproducible Research

Another reason for using R Markdown is to adhere to **reproducible research** principles. According to Claerbout and Karrenbach (1992), the definition of reproducible research is "Authors provide all the necessary data and the computer codes to run the analysis again, re-creating the results."

Since your document provides the R commands, readers can simply execute the commands to verify your output. Reproducible research improves transparency and builds trust in your work.

Note: The Turing Way provides an overview of the terms reproducible, replicable, robust, and generalizable for research. We will not go into the details for this class but it is worth your time to think about these.

2.1.2 Prerequisites

While you technically need the `rmarkdown` package, you do not need to explicitly install and load it, if you are using RStudio. RStudio performs these in the background when you knit (render) your document.

2.2 Getting Started

To get started with producing your document with R Markdown, open RStudio, then click on File, New File, R Markdown. Select the Document option, and fill in the entries for title, author, and date. For the time being, use HTML as the default output format.

You will see an untitled tab created. Save it. The file will have a `.Rmd` extension, which is the extension for an R Markdown file. There are some directions provided in this file. Feel free to read it.

In general, there are three essential components of an `.Rmd` file:

1. YAML header.
2. Narrative text.
3. Code chunks.

2.2.1 YAML Header

The **YAML header** controls the settings associated with the document. At this point, your YAML header looks like this:

```
---
title: "Untitled"
author: "Jeffrey Woo"
date: "2026-07-23"
output: html_document
---
```

The YAML header is fenced by three dashes, `---`. These three dashes are also called a **delimiter**. A delimiter is a specific character or symbol used to indicate the boundary between separate, independent regions of the file. You can easily change the title, name, and date by changing the values inside the quotes in the YAML.

2.2.2 Narrative Text

You can type your narration below the YAML.

2.2.2.1 Section Headers

Use hashtags `#` followed by a space to create a section header. The more hashtags the smaller the header.

```
# 1st level header
## 2nd level header
### 3rd level header
```

2.2.2.2 Text Styles

- For italics, fence the word(s) with `*`.
- For bold, fence the word(s) with `**`.
- To display short code in a text, fence the code with “‘.

```
*italicize these words*
**bold these words**
`mean(c(4,6,10))`
```

2.2.2.3 Hyperlinks

For hyperlinks, fence the text with `[ ]` then add the URL in parentheses. For example:

```
Check out the official [R Markdown Documentation](https://rmarkdown.rstudio.com).
```

2.2.2.4 Lists

For an unordered bullet point list, start each line with a dash `-`, asterisk `*`, or plus `+` followed by a space.

```
- First item
- Second item
- Third item
```

For a numbered list, start each line with a number followed by a period and a space.

```
1. First item
2. Second item
3. Third item
```

For a nested list, indent the line with two or four spaces (or one tab) and use the list marker.

```
- Main item 1
    - Sub-item 1a
    - Sub-item 1b
```

2.2.3 Code Chunks

Code chunks allow you to embed R code within your document. There are a few ways to insert a code chunk:

1. Ctrl + Alt + I
2. Click the insert code chunk icon (its a green square with a white C). Click on the down arrow and select R.
3. Type the chunk delimiters ```` ```{r} ```` and ```` ``` ````.

You can type in your R commands inside the code chunks.

2.2.3.1 Chunk options

You can customize how the code runs and displays by using chunk options inside the curly braces. There a lot of options; the ones you may use are:

- `message = FALSE`: which hides messages that R produces from your document.
- `warning = FALSE`: which hides warnings that R produces from your document.
- `echo = FALSE`: which hides the code, but allows the results from R to be displayed in your document. This should be used if readers do not want the R code.
- `eval = FALSE`: which shows the code but does not execute the code in your document. This is used for displaying example R code. (I use it a lot in these notes)

2.2.3.2 Inline Code

To display the result from R directly inside the narrative text, use inline R code by fencing the R code with single backticks starting with the letter `r`, for example:

```
The average of 2, 5, and 9 is`r mean(c(2, 5, 9))`.
```

You may want to consider using the `round()` function as R tends to print out a lot of decimal places

2.3 Cross-Referencing with R Markdown

Cross-referencing is done in a document to help connect related information to readers. You will often that you will want to connect a narrative with a graphical output, information presented in a table, a math equation, or a citation.

Cross-referencing is not provided directly within the `rmarkdown` package, but via the `bookdown` extension package. The output format in the YAML needs to be `html_document2`, `pdf_document2`, or `word_document2`. For example

```
---
title: "Untitled"
author: "Jeffrey Woo"
date: "2026-07-23"
output: bookdown::html_document2: default
---
```

R Markdown does cross-referencing and automatically numbers them for you. Below, we will go over a simple example of how to label a figure produced by R, and then refer to it in a narrative.

For this example, we will use the `pressure` data set that is loaded in base R. Per its documentation, the data are the temperature in degrees Celsius and vapor pressure of mercury in millimeters (of mercury).

In the code chunk below, notice a couple of items are added inside the curly braces:

1. A label, `plot-pressure`, for this example.
2. A caption via `fig.cap`.

```
```{r plot-pressure, fig.cap="Vapor Pressure of Mercury in mm Hg Against Temp in Celcius"}
plot(pressure)  # a scatterplot
```
```

An example narrative about this plot could be: we can see from Figure 2.1 that vapor pressure increases exponentially with temperature.

To cite the figure, type `\@ref(fig:plot-pressure)` in the narrative, using the label in the code chunk.

This is just a simple example. Referencing tables and equations are similar. Details can be found in Xie et al. (2020) Chapter 4.7.

Note: you will learn more about creating plots later in Section 8.

2.4 Knitting Your Document

Once you have your document ready, you can **knit** your document. Knitting transforms your plain text `.Rmd` file into a neat document. Click on the Knit

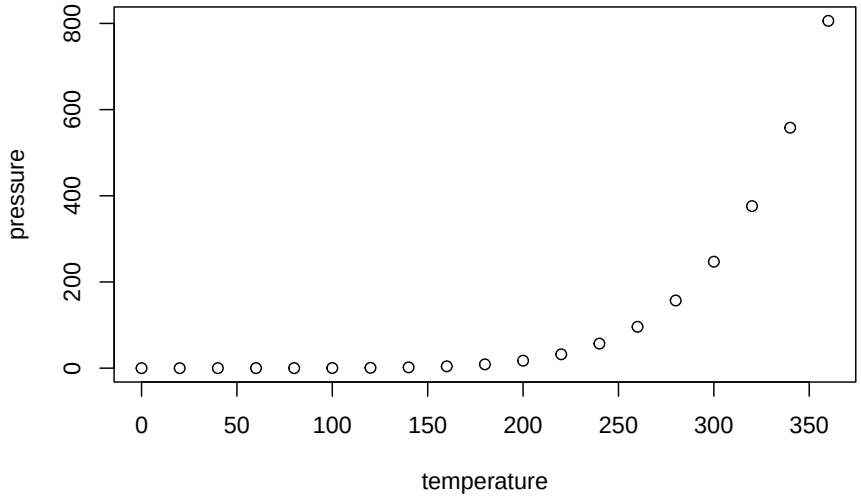


Figure 2.1: Vapor Pressure of Mercury in mm Hg Against Temp in Celcius

button, and your document is produced in the output format chosen at the beginning. This knit button also installs the `rmarkdown` package (if needed) and loads it. This is why you do not have to load it by using the `library()` function.

2.5 Things to Consider

2.5.1 R Script or R Markdown

You might be wondering when should you use an R script and when you should use R Markdown. Personally:

- If I am not going to be sharing the work, or only sharing the work with a small group of collaborators, I am likely to only use an R script.
- If I want to share my work broadly, then I will use an R Markdown. I also start the analysis in an R script, and then transfer the code to an R Markdown, as I find this makes troubleshooting easier.

2.5.2 Troubleshooting

I have seen some students work directly using an `.Rmd` file and bypass an R script. I find troubleshooting error messages difficult in this scenario, as there

are two potential sources of the error:

1. Error in R code inside code chunks.
2. Error in delimiters or options for the YAML, code chunks, inline code.

Personally, I type up and execute my R code in an R script first. Once there are no more errors, I then create my `.Rmd` file and then copy and paste the relevant R code into the code chunks. If an error pops up when knitting, I can rule out errors in the R code and focus on the delimeters and options for the YAML and code chunks.

2.5.3 PDF Output

You will need to produce a PDF file for the project. To do so, you can:

1. Change the default output format to PDF when creating the `.Rmd` file at the beginning.

- To knit the file into a PDF, you need LaTeX, which can be installed by running the following two commands in the R Console:

```
install.packages("tinytex")
tinytex::install_tinytex()
```

2. If this previous step does not work, change the default output format to word when creating the `.Rmd` file at the beginning. Then, convert the word document to a PDF using microsoft word.

2.5.3.1 More Resources for R Markdown

There are a lot more options available with R Markdown. We have only covered the very basics in this unit to get you started with this class. A few other resources with more details can be found below:

- R For Data Science, Ch 27
- R Markdown Cheat Sheet from RStudio
- Cross-Referencing Figures, Equations, and Tables Within Documents.

Chapter 3

Working with Data Structures

3.1 Motivation

Imagine you are working on a data frame in R. For most analyses, common tasks will include subsetting the data frame, performing math operations on the data frame, and computing summaries of some columns of the data frame. You will learn how to perform these tasks in this section.

We will create the same data structures in section 1.9 of the notes to use as examples in this section.

```
x <- c(5,10,15,20,25)
y <- c(3,6,9,12,15)
z <- c(x,y)
log_vect <- c(TRUE, FALSE, T, F)
char_vect <- c("ByOne", "ByTwo", "ByThree", "ByFour", "ByFive")
mult_mat <- cbind(char_vect,x,y)
mult_data <- data.frame(char_vect,x,y)
mult_list <- list(mult_data, z, log_vect, 7)
```

3.2 Subsetting Data Structures

Once a data structure is created and stored, it may be useful in subsequent analyses to subset that data structure. You will learn how to subset data structures using base R functions in this section. Similar operations can be performed using functions from the `dplyr` package, and you will learn these in Section 9.

3.2.1 Subsetting with Brackets

3.2.1.1 Subsetting Vectors

The typical notation to **subset** vectors is a set of **brackets**, `[ ]`, used directly after the variable name of the vector. The information given inside the brackets specifies which elements (in terms of index) should be extracted from the vector.

The position index of any vector begins at 1 and uses consecutive positive integers up to the length of the vector. For example, to obtain the 3rd element of the vector `x`:

```
x[3]
```

```
## [1] 15
```

```
x ## for comparison
```

```
## [1]  5 10 15 20 25
```

Note: Indexing in R starts from 1. R is designed for data analysis, statistics, and mathematics, where standard notations begin counting at 1 (e.g. statisticians think in terms of sample size). This is unlike indexing from 0 for other software such as Python and C, whose audiences tend to be programmers and software engineers.

Multiple positions can be specified by using a vector in the brackets.

```
x[c(1,3)] ##1st and 3rd elements
```

```
## [1]  5 15
```

```
x[2:3] ##2nd and 3rd elements
```

```
## [1] 10 15
```

If a position is duplicated in the brackets, the corresponding value in the vector will be subset twice.

```
x[c(1,1)] ##1st elements displayed twice
```

```
## [1] 5 5
```

If a negative sign is used in the brackets, the specified value(s) will be omitted instead of included in the subset.

```
x[-1] ##omit element 1
```

```
## [1] 10 15 20 25
```

```
x[-(2:3)] ##omit elements 2, 3
```

```
## [1]  5 20 25
```

```
x[-c(1,4)] ##omit elements 1, 4
```

```
## [1] 10 15 25
```

Instead of specifying the positions that you would like included or excluded from the subset, you can specify a logical vector in the brackets. R will include the elements corresponding to TRUE and exclude the elements corresponding to FALSE.

```
x[c(F,T,T,F,T)]
```

```
## [1] 10 15 25
```

If the number of logical inputs does not match the length of the vector, R will repeat the logical inputs until there is a logical input corresponding to each vector element.

```
x[c(F,T)]
```

```
## [1] 10 20
```

The resulting subset vectors will only be stored if you define a variable name and assign the subset vector to it.

```
evens <- x[c(F,T)]
x
```

```
## [1]  5 10 15 20 25
```

```
evens
```

```
## [1] 10 20
```

The brackets following a vector can also be used to replace or add one or more values in a vector.

```
x2 <- x

x2[6] <- 30 ## add 30 as element 6
x2
```

```
## [1]  5 10 15 20 25 30
```

```
x2[7:9] <- c(35,40,45) ##then add 3 values
x2
```

```
## [1]  5 10 15 20 25 30 35 40 45
```

```
x2[c(1,3)] <- 0 ##change value of elements 1, 3 to 0
x2
```

```
## [1]  0 10  0 20 25 30 35 40 45
```

3.2.1.2 Subsetting Matrices and Data Frames

To subset a matrix or data frame, a similar bracket notation is used. However, because matrices and data frames are two-dimensional structures, two arguments are required in the brackets. The first argument references the row index and the second input references the column index.

Note: subsetting data frames is very common when analyzing data, as data are usually stored in data frames.

The examples below are applied to a data frame. Subsetting matrices work in the same way.

```
mult_data
```

```
##   char_vect  x  y
## 1     ByOne  5  3
## 2     ByTwo 10  6
## 3   ByThree 15  9
## 4    ByFour 20 12
## 5    ByFive 25 15
```

```
## Subsetting the value in the first row and second column
mult_data[1,2]
```

```
## [1] 5
```

```
## Subsetting the values in the first two rows and first two columns
mult_data[1:2,1:2]
```

```
##   char_vect  x
## 1     ByOne  5
## 2     ByTwo 10
```

```
## Subsetting the values in rows 1, 4 and columns 2, 3
mult_data[c(1,4),2:3]
```

```
##    x  y
## 1  5  3
## 4 20 12
```

```
## Subsetting the entire first row.
## Comma as the 2nd argument means want all columns
mult_data[1,]
```

```
##   char_vect x y
## 1     ByOne 5 3
```

```
## Subsetting the entire second column
## Comma as the 1st argument means want all rows
mult_data[,2]
```

```
## [1]  5 10 15 20 25
```

```
## Subsetting all but the first row
mult_data[-1,]
```

```
##   char_vect  x  y
## 2     ByTwo 10  6
## 3   ByThree 15  9
## 4    ByFour 20 12
## 5    ByFive 25 15
```

```
## Subsetting all but the third column
mult_data[,-3]
```

```
##   char_vect  x
## 1     ByOne  5
## 2     ByTwo 10
## 3   ByThree 15
## 4    ByFour 20
## 5    ByFive 25
```

```
## Subsetting the third column twice
mult_data[,c(3,3)]
```

```
##    y y.1
## 1  3   3
## 2  6   6
## 3  9   9
## 4 12  12
## 5 15  15
```

```
## Subsetting columns 1, 2, 3 and duplicating the third column
mult_data[,c(1:3,3)]
```

```
##   char_vect  x  y y.1
## 1     ByOne  5  3   3
## 2     ByTwo 10  6   6
## 3   ByThree 15  9   9
## 4    ByFour 20 12  12
## 5    ByFive 25 15  15
```

3.2.1.3 Subsetting Lists

A list is subset to any of its elements by using **double brackets**, `[[ ]]`.

```
mult_list[[3]]
```

```
## [1]  TRUE FALSE  TRUE FALSE
```

Double brackets are limited to one value. In order to subset multiple elements of a list, single brackets can be used.

```
mult_list[2:3]
```

```
## [[1]]
##  [1]  5 10 15 20 25  3  6  9 12 15
##
## [[2]]
## [1]  TRUE FALSE  TRUE FALSE
```

3.2.2 Subsetting with Labels

3.2.2.1 Label Names with Data Structures

Take a look at the output for `mult_data`. Notice there is an extra row on top (as if they are headings or labels for the columns) and an extra column (as if they are labels for the rows).

```
mult_data
```

```
##   char_vect  x  y
## 1     ByOne  5  3
## 2     ByTwo 10  6
## 3   ByThree 15  9
## 4    ByFour 20 12
## 5    ByFive 25 15
```

Descriptive names can be added to columns, rows, or values within data structures. For data frames and lists, column names can be added while creating the data structure.

```
mult_data2 <- data.frame(Factor=char_vect, Fives=x, Threes=y)
mult_data2
```

```
##    Factor Fives Threes
## 1   ByOne     5      3
## 2   ByTwo    10      6
## 3 ByThree    15      9
## 4  ByFour    20     12
## 5  ByFive    25     15
```

```
mult_list2 <- list(Factor.Table=mult_data, FiveThree=z,
                   Logical.values=log_vect, DaysPerWeek=7)
mult_list2
```

```
## $Factor.Table
##   char_vect  x  y
## 1     ByOne  5  3
## 2     ByTwo 10  6
## 3   ByThree 15  9
## 4    ByFour 20 12
```

```
## 5    ByFive 25 15
##
## $FiveThree
##  [1]  5 10 15 20 25  3  6  9 12 15
##
## $Logical.values
## [1]  TRUE FALSE  TRUE FALSE
##
## $DaysPerWeek
## [1] 7
```

Column, row, or value **labels** can also be added to an existing data structure using one or more of the following functions: `names()`, `rownames()`, and `colnames()`. The appropriate function(s) to use depends on the type of data structure needing label names. This information is summarized in Table 3.1 below:

Table 3.1: Labels for R Data Structures

Data Structure	`names()`	`colnames()`	`rownames()`
Vector	Elements	-	-
List	List items	-	-
Matrix	-	Column names	Row names
Data frame	Column names	Column names	Row names

My advice: use `names()` for one-dimensional data structures, `colnames()` and `rownames()` for two-dimensional data structures. Examples are displayed below:

```
x ## For comparison
```

```
## [1]  5 10 15 20 25
```

```
## Labeling values in a vector
names(x) <- char_vect
x
```

```
##   ByOne   ByTwo ByThree  ByFour  ByFive
##       5      10      15      20      25
```

```
## Labeling rows and columns in a matrix
mult_mat2 <- cbind(x,y)
rownames(mult_mat2) <- char_vect
colnames(mult_mat2) <- c("Fives","Threes")
mult_mat2
```

```
##         Fives Threes
## ByOne       5      3
## ByTwo      10      6
```

```
## ByThree    15      9
## ByFour     20     12
## ByFive     25     15
```

```
## Labeling columns in a data frame
colnames(mult_data) <- c("Factor","Fives","Threes")
mult_data
```

```
##    Factor Fives Threes
## 1   ByOne     5      3
## 2   ByTwo    10      6
## 3 ByThree    15      9
## 4  ByFour    20     12
## 5  ByFive    25     15
```

```
## Labeling values in a list
names(mult_list) <- c("Factor.Table", "FiveThree", "Logical.values",
                      "DaysPerWeek")
mult_list
```

```
## $Factor.Table
##   char_vect  x  y
## 1     ByOne  5  3
## 2     ByTwo 10  6
## 3   ByThree 15  9
## 4    ByFour 20 12
## 5    ByFive 25 15
##
## $FiveThree
##  [1]  5 10 15 20 25  3  6  9 12 15
##
## $Logical.values
## [1]  TRUE FALSE  TRUE FALSE
##
## $DaysPerWeek
## [1] 7
```

The `names()` function can also be used to display the names of a data structure.

```
names(x)
```

```
## [1] "ByOne"   "ByTwo"   "ByThree" "ByFour"  "ByFive"
```

```
names(mult_data)
```

```
## [1] "Factor" "Fives"  "Threes"
```

```
names(mult_list)
```

```
## [1] "Factor.Table"   "FiveThree"      "Logical.values" "DaysPerWeek"
```

3.2.2.2 Subsetting with Labels

Data structures with labels can be subsetted using their labels. The name of the column or element can be used within the brackets or after the $ symbol.

```
mult_data[,"Fives"]
```

```
## [1]  5 10 15 20 25
```

```
mult_data$Fives
```

```
## [1]  5 10 15 20 25
```

```
mult_list[["Logical.values"]]
```

```
## [1]  TRUE FALSE  TRUE FALSE
```

```
mult_list$Logical.values
```

```
## [1]  TRUE FALSE  TRUE FALSE
```

3.2.3 Subsetting with Logical Expressions

3.2.3.1 Logical Expressions

It may be useful to use a **logical expression** to determine which, if any, elements in a vector have certain qualities. When a logical expression is applied to a vector, the result will be a vector containing TRUE where the condition is satisfied and a FALSE where the condition is not satisfied.

```
## Are any elements greater than 15?
x > 15
```

```
##   ByOne   ByTwo ByThree  ByFour  ByFive
##   FALSE   FALSE   FALSE    TRUE    TRUE
```

```
## Are any elements greater than or equal to 15?
x >= 15
```

```
##   ByOne   ByTwo ByThree  ByFour  ByFive
##   FALSE   FALSE    TRUE    TRUE    TRUE
```

```
## Are any elements exactly equal to 15?
x == 15
```

```
##   ByOne   ByTwo ByThree  ByFour  ByFive
##   FALSE   FALSE    TRUE   FALSE   FALSE
```

```
## Are any elements not equal to 15?
x != 15
```

```
##   ByOne   ByTwo ByThree  ByFour  ByFive
##    TRUE    TRUE   FALSE    TRUE    TRUE
```

Multiple logical expressions can be combined using either **and** or **or**.

```
## Are any elements in x greater than 15 AND the same element in y is equal to 12?
x > 15 & y == 12
```

```
##   ByOne   ByTwo ByThree  ByFour  ByFive
##   FALSE   FALSE   FALSE    TRUE   FALSE
```

```
## Are any elements greater than 15 OR less than 7?
x > 15 | x < 7
```

```
##   ByOne   ByTwo ByThree  ByFour  ByFive
##    TRUE   FALSE   FALSE    TRUE    TRUE
```

3.2.3.2 Subsetting with Logical Expressions

The logical vectors that result from logical expression statements can be used in the brackets to subset a vector just to the values that satisfy the condition.

```
x[x>15]
```

```
## ByFour ByFive
##     20     25
```

Instead of yielding a logical vector for each entry in a vector, the `which()` function yields the indexes in the vector where the condition is satisfied. The `which()` function can also be used in the brackets to subset a vector to just the values that satisfy the condition.

```
which(x>15)
```

```
## ByFour ByFive
##      4      5
```

```
x[which(x>15)]
```

```
## ByFour ByFive
##     20     25
```

Two-dimensional data frames can be subsetting similarly, using the an argument for the row and another for the column.

3.3 Loading Data Sets in R

We will cover two main ways to load data sets in R:

1. Data sets from R.
2. Data set from a local file.

3.3.1 Data Sets from R

Base R comes pre-loaded with some data sets. You can view them by typing `data()`. You can use the specific data set by typing out its name in the list. To view the `airquality` data set:

```
head(airquality)
```

```
##   Ozone Solar.R Wind Temp Month Day
## 1    41     190  7.4   67     5   1
## 2    36     118  8.0   72     5   2
## 3    12     149 12.6   74     5   3
## 4    18     313 11.5   62     5   4
## 5    NA      NA 14.3   56     5   5
## 6    28      NA 14.9   66     5   6
```

You can use the `head()` function to print out the first 6 rows of a data frame instead of displaying the entire data frame. This gives the reader a good idea of the data without cluttering the document.

Notice that this data frame does not appear in the environment. Personally, I like to assign such data frames to an object so it appears on my environment.

```
Data.air <- airquality
```

Data sets could also come from an R package. For example, the `Davis` data set comes from the `carData` package. So install (if needed) and load the package, and type the name of the data set. Again, I prefer to assign the data set to an object so it appears on the environment.

```
library(carData)
Data.Davis <- carData::Davis
```

Notice that I typed `carData::Davis`. This command tells R to go into the `carData` package and use the data set called `Davis`.

You could have typed `Davis` as well. R would have searched all loaded packages and find the data set called `Davis`. This would work fine if there is no other data set or function with the same name. If another package has a data set or function with the same name, there will be a conflict, and R will use the package that was most recently loaded. This is why I recommend typing the name of the package as well.

3.3.2 Data Sets from Local File

A file containing a data set can be easily loaded into the workspace. Be sure the file is in the working directory (see Section 1.3 on how to set the working directory).

If the data file is comma separated (`.csv`), the function `read.csv()` is used to read in the data.

```
read.csv("Data1.csv", header=TRUE)
```

The argument `header=TRUE` tells R that the first row of the file is a label for the columns (header). If the file does not have a header, specify `header=FALSE`.

Another common type of data file is a `.txt` file. In this case, use `read.table()`.

```
## file has headers
read.table("Data1.txt", header=TRUE)
```

3.3.2.1 Factor Variables

If you read the documentation for the `read.table()` function, you may notice an argument called `stringsAsFactors = FALSE`. This prevents text columns from being automatically converted into **categorical factor variables**, ensuring they are imported as plain character vectors instead.

Factor variables group categorical data into sets containing values with the same string. Each set is given a numerical code that represents the corresponding categorical value. Dummy coding is an example of this. When performing some statistical modeling such as linear regression, categorical data need to be factors.

However, the default is set to `FALSE`, as this saves memory, and it is easier to wrangle with data when they are characters and not factors. Wrangling with data is performed prior to modeling; therefore categorical data are converted to factors after data wrangling and when we are ready for modeling.

To coerce a vector to a factor, use the `as.factor()` or `factor()` functions.

3.4 Math Operations on Data Structures

The standard arithmetic operators can be applied to data structures containing numeric values. To perform the same constant arithmetic operation on each element of a numeric data structure, you will use the object name, the arithmetic operation symbol, and the constant.

```
x <- unname(x) ## remove the name for x
x
```

```
## [1]  5 10 15 20 25
```

```
x*2
```

```
## [1] 10 20 30 40 50
```

```
x/5
```

```
## [1] 1 2 3 4 5
```

```
x^3
```

```
## [1]   125  1000  3375  8000 15625
```

You may have related numeric information stored in two data structures of the same length / dimensions and you may want to perform an arithmetic operation on each pair of related values.

```
x
```

```
## [1]  5 10 15 20 25
```

```
y
```

```
## [1]  3  6  9 12 15
```

```
x+y
```

```
## [1]  8 16 24 32 40
```

```
x*y
```

```
## [1]  15  60 135 240 375
```

```
y/x
```

```
## [1] 0.6 0.6 0.6 0.6 0.6
```

3.5 Applying Functions on Data Structures

Functions can be applied to data structures, for example

```
sum(x)
```

```
## [1] 75
```

```
mean(mult_mat2)
```

```
## [1] 12
```

When working with a data frame, we often want to compute some numerical summaries of some or all of the columns. The `apply()` function applies a pre-existing or user-defined function to the data structure in a manner specified by the user. In this unit, we will just cover the most basic application of the `apply()` function. Variants of this function and user-defined functions will be covered in Section 7.

Suppose you are working on the `airquality` data set, that is stored in the object `Data.air`. The first four columns represents different numeric characteristics associated with air quality and weather conditions. You want to find the average value in each column.

The `apply()` function is used for a two-dimensional data structure. It uses three arguments: data, margin, and function. The data argument specifies the matrix or data frame to which the function should be applied, margin specifies whether

you want the function applied to the rows, columns, or individual values, and function specifies the function to be applied.

```
## 2 since we want to apply the mean to the columns
## use 1 if want to apply mean to the row
## applying to columns is more common as each column
## in a data frame represents a characteristic
apply(Data.air[,1:4], 2, mean)
```

```
##     Ozone   Solar.R      Wind      Temp
##        NA        NA  9.957516 77.882353
```

Notice the NA for two of the columns? This happens when at least one entry in those columns has a missing value. Missing values in R are denoted by `NA`. If you look at the data frame, you can find some NAs in these columns.

If you want R to ignore values and exclude them in calculating the mean, add an argument `na.rm=TRUE` in the `apply()` function.

```
apply(Data.air[,1:4], 2, mean, na.rm=TRUE)
```

```
##      Ozone    Solar.R       Wind       Temp
##  42.129310 185.931507   9.957516  77.882353
```

Suppose you want to count the number of missing values in the column `Ozone`. The `is.na()` function returns a logical identifying whether an entry is missing or not.

```
## the head function returns only the first 6 entries of a vector
## or first 6 rows of a data frame
head(is.na(Data.air$Ozone))
```

```
## [1] FALSE FALSE FALSE FALSE  TRUE FALSE
```

Recall that `TRUE` and `FALSE` are considered to be 1 and 0 respectively. So you can find the number of missing values for `Ozone` with

```
sum(is.na(Data.air$Ozone))
```

```
## [1] 37
```

Chapter 4

Functions

4.1 Motivation

Imagine you are using R to work on a project. You may notice that you have copied and pasted blocks of code in a number of places of your code, with minor tweaks in the value of a particular variable. You will likely be able to replace these blocks of code with a function. A **function** is a reusable block of code designed to perform a specific task. Copying and pasting blocks of code in a number of places is sometimes called **hard-coding**. A benefit of using functions is reducing copying and pasting errors. Manually changing the value of a variable across different blocks of code often leads to mistakes.

You have been using functions that come from base R, or come with other packages. You will now learn how create your own functions.

Functions are defined with three main components:

- the function **name**,
- the **arguments** that the function will take,
- and the **body**, which is the code that should be executed when the function is called.

4.2 Basic Syntax

Suppose you want to convert temperature from Celsius to Fahrenheit in your code. You can create a function in the following manner:

```
CtoF <- function(celsius) {
  celsius * (9/5) + 32
}
```

In this particular example, the function name is `CtoF()`. It only takes in one argument named `celsius`. The value of the argument is the value that can change each time the function is used. We then use an open brace, press enter to enter the body of the function, and then press enter and then use a close brace. The body of the function are the commands that are executed when the function is used.

To use this function to convert 27 degrees Celsius to Fahrenheit

```
CtoF(27)
```

```
## [1] 80.6
```

In this example, you can think of the following sequence of events happening when executing the function named `CtoF(27)`:

- The value of 27 is assigned to `celsius`,
- the math operations as written in the body are performed on `celsius`,
- the value after the operations is returned from the function,
- the value of 27 is no longer stored in `celsius`. In fact, `celsius` is not stored in the environment.

Note: when creating your own function, it is best to avoid using naming your function or arguments with words that are associated with functions in base R.

4.3 Returning a Value

A function in R returns the value of its last evaluated expression. You could explicitly use `return()` in the body to clearly state what you want returned. The function below performs identically to `CtoF()`:

```
## explicit return
CtoFexp <- function(celsius) {
  fahrenheit <- celsius * (9/5) + 32
  return(fahrenheit)
}

CtoFexp(27)
```

```
## [1] 80.6
```

In R, `return()` is not needed when returning a single object, which is unlike other programming languages. If returning multiple objects, `return()` should be used, see Section 4.6.

4.4 Arguments and Default Values

A function can take more than one argument (or even zero arguments such as `getwd()`), and any number of arguments can have a **default** value. Suppose

we want to convert Celsius to Fahrenheit, but also want to function to round to the nearest whole number, by default.

```
CtoFrounded <- function(celsius, decimals = 0) {
  fahrenheit <- celsius * (9/5) + 32
  round(fahrenheit, decimals)
}

CtoFrounded(27)
```

```
## [1] 81
```

```
CtoFrounded(celsius = 27, decimals = 1)
```

```
## [1] 80.6
```

In the function `CtoFrounded()`, a second argument appears, `decimals`, and is set to 0 by default with the equal sign. This argument does not need to be entered if you want to use the default value when using the function. The first argument `celsius` is not set to any default, and must be specified when calling the function.

You can also use `CtoFrounded(27, 1)` to get 27 degrees Celsius converted to 1 decimal place. The values for the arguments entered in the function must be in the same order as the order of the arguments when the function is created.

```
CtoFrounded(27, 1)
```

```
## [1] 80.6
```

Alternatively, you can use `CtoFrounded(decimals = 1, celsius = 27)`. The order does not matter if you specify the names of the arguments.

```
CtoFrounded(decimals = 1, celsius = 27)
```

```
## [1] 80.6
```

You can also decide to have no defaults for any argument at all

```
CtoFnodef <- function(celsius, decimals) {
  fahrenheit <- celsius * (9/5) + 32
  round(fahrenheit, decimals)
}

CtoFnodef(27, 1)
```

```
## [1] 80.6
```

4.5 Argument Validation

Ideally, your function should check if the argument entered satisfies certain conditions, and outputs either an error message if the argument does not work, or a warning message if the argument is suspicious. This is called **argument validation**. Consider our function `CtoF()`. To convert Celsius to Fahrenheit, the argument entered must be numeric, and the numeric should be within a reasonable range for temperatures.

```
CtoFcheck <- function(celsius) {
  if (!is.numeric(celsius)) {
    stop("celsius must be numeric ")
  }
  if (celsius < -100 | celsius > 100){
    warning("Is celsius within acceptable range?")
  }
  celsius * (9/5) + 32
}

CtoFcheck("blah")
```

```
## Error in CtoFcheck("blah"): celsius must be numeric
```

```
CtoFcheck(250)
```

```
## Warning in CtoFcheck(250): Is celsius within acceptable range?
```

```
## [1] 482
```

The `stop()` function stops execution, while `warning()` will print out the warning message but still executes the code and prints out the result.

Notice `if()` statements are used here. Basically, an `if()` statement checks whether a certain condition is TRUE, if so, the code in the curly braces get executed. If FALSE, the code in the curly braces is skipped, and code outside the curly braces get executed. We will cover `if()` statements in more detail in in Section 5.

When I write functions, I normally add the parts regarding argument validation last. I would test that my function works with valid arguments, then add the argument validation parts, then test with invalid arguments.

4.6 Return Multiple Objects

R functions return a single object. If you need to return multiple objects, you can return those objects into a named list. For example,

```
# Returning multiple items using a list
get_summary_stats <- function(vec) {
```

```
  my_mean <- mean(vec)
  my_sd   <- sd(vec)

  # Return objects as a list and name them
  return(list(mean = my_mean, sd = my_sd))
}

get_summary_stats(c(4,3,1,2,8))
```

```
## $mean
## [1] 3.6
##
## $sd
## [1] 2.701851
```

```
get_summary_stats(c(4,3,1,2,8))$mean
```

```
## [1] 3.6
```

4.7 Scope

Variables inside a function are **local**. This means that the value of a variable is not stored in the environment after the function is run (see Section 4.2 where I described the sequence of events when the function is executed). This means that if there is a variable with the same name outside the function, the variable outside is not overwritten. For example,

```
a <- 3 ## global

toy1 <- function(){
  a <- 10 ## local
  return(a + 20)
}

toy1() ## function uses local a
```

```
## [1] 30
```

```
a ## global untouched
```

```
## [1] 3
```

This means that we do not have to be concerned with names of variables inside a function that we create, as these variables will not overwrite variables with the same name outside the function.

Chapter 5

Control Flow

5.1 Motivation

You may realize that you may need your code to make decisions based on criteria (e.g. argument validation in Section 4.5) or repeat (a set number of times or an unknown amount of times). **Control flow** tools will allow you to do so. The main tools and their functions are:

- Branching with if statements: If a condition is true, execute a block of code. Extensions include the possibility of running another block of code if the condition is false.
- Iterate with for loops: repeat a block of code a fixed, known number of times.
- Iterate with while loops: repeat unknown number of times until a condition is no longer met.

It may be worth reviewing logical expressions in Section 3.2.3.1 as they are used a lot in control flows. Pay attention to the `&` and `|` operators. In these examples, the operators were performed element-wise, and are useful when subsetting or modifying a data structure.

However, in control flows, we want logical expressions to return an output of length 1 (a scalar), so you should use the **scalar operators** `&&` for and and `||` for or.

5.2 Branching

5.2.1 If Statement

Branching with an **if statement** allows our code to execute only if a certain condition is met. If the condition is not met, nothing is executed. For example

```
a <- 1

if (a > 0 || a == 0){
  print("a is positive or 0")
}
```

```
## [1] "a is positive or 0"
```

The parenthesis after the `if` statement is the condition that has to be met for the code to be executed. In this example, the condition is met since `a` is equal to 1, so `a > 0` returns `TRUE`, and the code in the braces is executed. However, if we run the following, notice nothing gets printed.

```
a <- -9

if (a > 0){
  print("a is positive")
}
```

Since `a` is equal to -9, the condition `a > 0` is `FALSE`, so the code inside the braces is not executed.

5.2.2 If-Else Statement

If you want to run another block of code when the condition is not met, add an `else` statement that runs when the condition is not met. This is called an **if-else statement**: it branches out based on two conditions.

```
## a is still -9

if (a > 0){
  print("a is positive")
} else {
  print("a is 0 or negative")
}
```

```
## [1] "a is 0 or negative"
```

Since the condition `a > 0` is not met, the code inside the other set of braces (after the `else` statement) gets executed.

So, add an `else` statement if you have two conditions or branches.

5.2.3 Vectorized If-Else

The `ifelse()` function is a vectorized version of an if-else statement. It applies an if-else statement to each element in a vector, and returns a vector with the same length. For example, I have a number of students in class, and they pass if they score at least 60 on a test, otherwise they fail.

```
scores <- c(80, 90, 40, 95, 55)

ifelse(scores >= 60, "pass", "fail")
```

```
## [1] "pass" "pass" "fail" "pass" "fail"
```

```
## condition applied to vector first, then
## what to return if condition is true, then
## what to return if condition is false
```

This is a very common way to convert a quantitative variable into a binary variable during data wrangling.

5.2.4 Else If

If there are more than two conditions or branches, use `else if` statements between the `if` statement at the beginning and an `else` statement at the end.

```
## a is still -9

if (a > 0){
  print("a is positive")
} else if (a == 0) {
  print("a is 0")
} else {
  print("a is negative")
}
```

```
## [1] "a is negative"
```

The condition `a > 0` is not met, so the code in its braces is not run. The next condition `a == 0` is not met, so the code in its braces is not run. Therefore, the code in the braces after the `else` is executed.

5.2.4.1 Example

What do you think gets printed in this example?

```
## a is still -9

if (a > 0){
  print("a is positive")
} else if (a == 0) {
  print("a is 0")
} else if (a < 0 && a > -2){
  print("a is betwwen -2 and 0")
}
```

5.3 Iterating

5.3.1 For Loops

A **for loop** iterates a block of code for each element in a sequence. As a simple example, suppose I want to print out the values of n^2 for $n = 1, 2, 3, \cdots, 10$:

```
for (n in 1:10){
  print (n^2)
}
```

```
## [1] 1
## [1] 4
## [1] 9
## [1] 16
## [1] 25
## [1] 36
## [1] 49
## [1] 64
## [1] 81
## [1] 100
```

Take a closer look at this for loop. A for loop is made up of 5 components:

1. The for statement.
2. The index variable. In this example, this is `n`. This value changes at each iteration of the loop.
3. The in statement.
4. The vector of values for which the for loop should run, i.e. the values of the variable `n` that should be used. In this example, the values of `n` should be integers 1 to 10.
5. the code to be repeated.

The way to think about how this loops runs, is to:

- Run the code for the first value of the index variable, which is 1, so $1^2 = 1$ is printed.
- When this code if finished for the value of 1, move on to the next value of the index variable, which is 2, so $2^2 = 4$ is printed.
- When this code if finished for the value of 2, move on to the next value of the index variable, which is 3, so $3^2 = 9$ is printed.
- Keep iterating until the last value of the index variable, which is 10, so $10^2 = 100$ is printed.

If values are being calculated in the loop that need to be saved, it is best practice to create a data structure of an appropriate size to record the values during the loop. This practice is called **pre-allocation**. This is preferred to creating a data structure with unknown size and have the data structure grow as the loop is run, as pre-allocation is more efficient.

```
## create a vector to store the 10 squared values
## initialize each entry in the vector to be NA to start
## we have to fill the vector with something at the start
squaredVals <- rep(NA, 10)

for (n in 1:10){
  squaredVals[n] <- (n^2)
  ## store the value of n^2 in each entry of the vector
  ## replacing the original value of 0 in each entry
}
```

Ideas associated with a for loop, i.e. iterating a block of code a set number of times, are used a lot in simulations. As mentioned in the preface, simulations are used to estimate statistical quantities that may be difficult to compute. It turns out that there are more efficient ways to run simulations in R, using either vectorization (see Section 5.4) or functions in the apply family (see Section 7).

5.3.2 While Loops

A **while loop** iterates a block of code until some condition is met. Typically, number of iterations is unknown. While loops are much less common in practice though. We will just use a simple example to illustrate this idea.

Suppose you play a gambling game where you start with $5. Each time you play the game, the amount of money you lose is a random number that comes from a normal distribution with mean 2, and standard deviation 1. You keep playing the game until you run out of cash. We want to simulate the number of times you play the game, and record the amount of cash after each time you play the game.

```
## set.seed() for reproducibility due to random nature
## of random loss each time you play
set.seed(10)

cash <- 5 ## initial amount of cash

##store amount of cash at each play.
##start with empty vector as we don't know
##how many times you will play
##don't do this with for loop as pre-allocation
##is more efficient than starting with empty vector
storeCash <- c()
numPlays <- 0 ## counter to count number of times you play

while (cash > 0) {
  cash <- cash - rnorm(1, mean = 2, sd =1)
  numPlays <- numPlays + 1
```

```
  storeCash[numPlays] <- cash
}

numPlays
```

```
## [1] 4
```

```
storeCash
```

```
## [1]  2.9812538  1.1655064  0.5368369 -0.8639954
```

5.4 Vectorization in R

Vectorization in R is a feature of R where operations are applied to an entire vector or data structure at once, rather than iterating through each individual element one by one using a traditional loop. You have seen this feature in the examples in Section 3.4. Vectorization makes performing math operations and functions on data structures efficient.

Consider the example in Section 5.3.1. We have a vector `1:10` and we want to square each value. Instead of squaring each entry in the vector one by one in a for loop, we can easily type `(1:10)^2` to get the same result.

As your code gets longer and more complex, making it more efficient with vectorization and pre-allocation will make a difference in how long your code runs. Another way of making code more efficient in R is using the `apply` family of functions: you have seen a simple example in Section 3.5, and you will learn more about this family of functions in Section 7.

Chapter 6

Simulations

6.1 Motivation

When we are analyzing data, there is always going to be some degree of uncertainty, as there is **randomness** in a lot of phenomena that we observe in our world. An example that is common in statistics comes from the field of sample surveys. A poll is conducted to see what proportion of voters will vote each candidate in an upcoming election. Even with a well-designed poll and a large sample size, the value of the sample proportion that will vote for a candidate has some randomness to it. If we get another random sample, the sample proportion is likely to be different (but hopefully close). The act of obtaining a random sample, or **sampling**, from a population introduces some randomness.

Statisticians and data scientists often want to investigate the degree of randomness involved in what is being studied. Sometimes, this can be done mathematically, but oftentimes, the mathematics is extremely challenging. You may also realize in the previous paragraph that one way of studying the randomness in polls is to repeatedly obtain random samples. But it is practically unfeasible to repeatedly go out in the field to collect data.

This is why simulations are used. We do not need to solve complex mathematics, and we can simulate the studies many times (in the thousands, at least) with computers, saving time and money than if we physically ran these studies many times.

You will see how sampling introduces randomness and then learn how to create simulations to quantify the degree of randomness or uncertainty.

6.2 Sampling

The `sample()` function in R randomly draws a selected number of items from a list of items (the list is usually a vector), and the probabilities of each item being selected are the same for all items. A simple example of randomness is tossing a fair coin a set number of times (equal chance of landing heads or tails).

```
fairCoin <- c("heads", "tails")
sample(fairCoin, size = 5, replace = TRUE)
```

```
## [1] "heads" "heads" "heads" "tails" "heads"
```

The `sample()` function has three main arguments:

- A vector of values from which to sample.
- How many values from the vector to draw.
- If you want to sample with or without replacement.

Sampling with replacement means that when we sample a value from the vector, the value gets returned to the vector and can be sampled again.

Sampling without replacement means that means that when we sample a value from the vector, the value does not get returned to the vector and cannot be sampled again.

Run the code in the block above a few times. You will notice that the returned vector is not always the same, so the result is unpredictable. This is unlike most of the earlier code you have seen in the course notes.

Another use of sampling is in model validation (i.e. assessing model accuracy). The original data set is randomly split into two mutually exclusive data sets: one called training and the other called test. The training set is used to build the model, and model assessment is done on the test set. Suppose we want to split the `Davis` data set from the `carData` package.

```
library(carData)
Data.Davis <- carData::Davis
n <- dim(Data.Davis)[1]
## when dim() is applied to a dataframe, two values get returned
## the num of rows and number of columns. The number of rows is
## equal to the sample size

trainInd <- sample(1:n, size = ceiling(n/2), replace = FALSE)
## each observation belongs to a row. So randomly select half of
## row indexes without replacement so the two sets are mutually ## exclusive

## split data into training and test
trainData <- Data.Davis[trainInd,]
testData <- Data.Davis[-trainInd,]
```

You have also seen sampling before in Section 1.7.2, where functions such as `rnorm()` and `rbinom()` draw random numbers from known probability distributions such as the normal and binomial distributions. Try typing `rnorm(10)` a few times. You should get a vector containing 10 random draws from a standard normal, and the vectors are not always the same.

6.3 Replicate

You have seen how sampling is inherently random. In order to quantify the uncertainty introduced through sampling, we want to be able to repeat, or **replicate**, the sampling a lot of times.

The `replicate()` function repeatedly evaluates a given function or expression, returning a matrix where each column contains the output of each replicate. There are two arguments: the number of replications and the function to be replicated.

Going back to the coin tossing example, suppose we want to toss a fair coin 5 times, and replicate this experiment for 6 replicates.

```
replicate(6, sample(fairCoin, size = 5, replace = TRUE))
```

```
##      [,1]    [,2]    [,3]    [,4]    [,5]    [,6]
## [1,] "heads" "tails" "heads" "heads" "tails" "tails"
## [2,] "heads" "tails" "heads" "heads" "tails" "tails"
## [3,] "tails" "heads" "tails" "heads" "heads" "heads"
## [4,] "tails" "heads" "tails" "tails" "tails" "tails"
## [5,] "heads" "tails" "tails" "tails" "heads" "heads"
```

From section 5, you may think of using a for loop to execute these steps. A for loop could do the same, but the code is longer.

6.4 Simulation

According to Bonnell and Ogihara (2024), a simulation consists of three main steps:

1. Decide what to simulate. What process do we want to imitate?
2. Figure out how to simulate one value.
3. Replicate step 2 many times.

In the previous block of code, we see these steps in action:

- We want to imitate the results from tossing a fair coin 5 times.
- We use the `sample()` function to simulate one "value", which is a vector storing the outcomes from 5 tosses of the coin.
- We use the `replicate()` to repeat the previous step for a total of 6 replicates.

6.4.1 Worked Example: Sampling Distribution of the Sample Mean

In introductory statistics, you learned that if $X_i \sim N(\mu, \sigma^2)$, then $\bar{X} \sim N(\mu, \frac{\sigma^2}{n})$ (where $\bar{X} = \frac{X_1+X_2+\cdots+X_n}{n}$ and X_i are independent. Independence is achieved via random sampling). This result is sometimes called the sampling distribution of the sample mean, and can be proved mathematically. However, such proofs are not the scope of this class.

This means that if we have data from a $N(\mu, \sigma^2)$ distribution, then the distribution of the sample means from a large number of random samples each of size n is $N(\mu, \frac{\sigma^2}{n})$. We will see how we can use simulation to illustrate the sampling distribution of the sample mean.

Suppose the data follow a standard normal distribution, i.e. $X \sim N(0, 1)$. The sampling distribution of the sample mean tells us that based on samples each of size 10, $\bar{X}_{10} \sim N(0, \frac{1}{10})$. So the three main steps of this simulation would be:

1. We want to simulate 10 random draws from a standard normal distribution, compute their sample mean, and store this value.
2. Use `rnorm(10)` to simulate 10 draws from a standard normal distribution, then compute the sample mean of these 10 draws, and store this value.
3. Repeat step 2 for a large number of times, say for a total of 10,000 replicates.

Step 3 involves using the `replicate()` function, where we supply the number of replicates and a function that gives the result from step 2, i.e. the sample mean from 10 random draws from a standard normal. While there is no function in base R that does this, we can easily write our own function to do this (see Section 4 on how to write functions)!

```
sampMean <- function(number){
  mean(rnorm(number))
}
```

Our created `sampMean()` computes the sample mean of a certain number of draws (that is supplied) from a standard normal distribution. `sampMean()` executes step 2.

For step 3, we can just use `replicate()` on `sampMean()`. Suppose we want 10,000 replicates. We store the value of the sample mean from 10,000 random samples, each with size 10.

```
Reps <- 10000
storeSampMeans <- replicate(Reps, sampMean(10))
```

Intuitively, we know the sample means from each sample are different. We can print out a few of these values to confirm this idea.

```
head(storeSampMeans)
```

```
## [1] -0.28824148  0.56248186  0.02602954  0.30351791  0.29413886  0.92128312
```

If the sampling distribution of the sample mean is correct, we know that $\bar{X}_{10} \sim N(0, \frac{1}{10})$, theoretically. Our simulation should be consistent with this theory.

We can create a density plot of the 10,000 sample means to visualize the uncertainty from sampling. We can see in figure 6.1 that most of the sample means are close to 0, and values further from 0 are less likely to occur. This also looks like a normal distribution.

```
plot(density(storeSampMeans), main="")
```

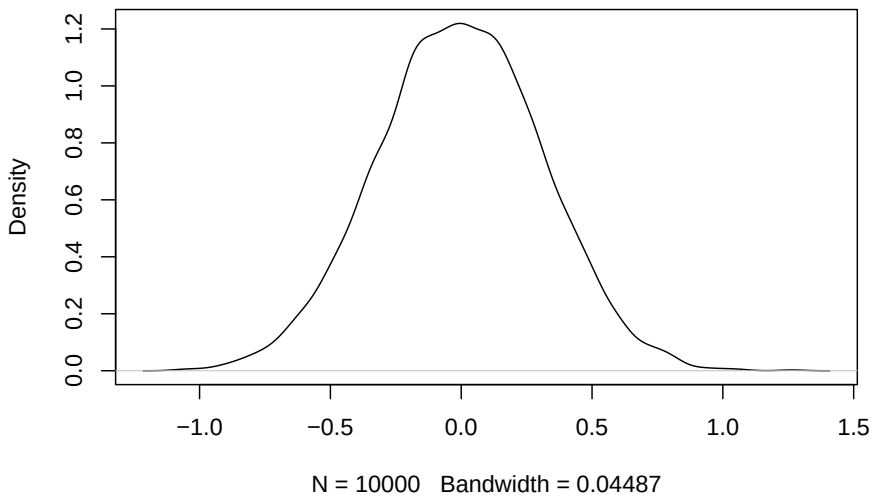


Figure 6.1: Density Plot of Sample Means from 10,000 Random Samples of Size 10

We know that the mean and variance of the 10,000 sample means should be close to 0 and 0.1 theoretically.

```
mean(storeSampMeans)
```

```
## [1] -0.005047077
```

```
var(storeSampMeans)
```

```
## [1] 0.09895951
```

The variance measures how spread out the sample means are, and is considered a measure of the uncertainty associated with the sample mean from samples of size 10.

Pay attention to the difference between the number of replicates, which is 10,000, and the sample size (number of data points drawn) of each replicate, which is 10. People often confuse these two.

We have just created a simulation that imitates the sampling distribution of the sample mean when the data are standard normal and each sample has size 10!

Hopefully, this example convinces you that with a large number of replicates, results from a simulation match very closely with theory.

Note: you could have gotten similar results by using a for loop, using the code below. However, the for loop is a bit longer as we need to pre-allocate, and is less efficient.

```
## Repeat for total 10,000 times
Reps <- 10000
## pre allocate vector to store the 10,000 sample means
sampleMeansLoop <- c(NA, Reps)

for (i in 1:Reps){
  ## draw 10 numbers from standard normal, find the mean, and store
  sampleMeansLoop[i] <- mean(rnorm(10))
}

plot(density(sampleMeansLoop), main="")
mean(sampleMeansLoop)
var(sampleMeansLoop)
```

In many other situations, mathematical theory can be difficult to prove. For example, what is the sampling distribution of the median from a non-normal population? While finding this result will be challenging mathematically, we can use simulation to get very close to the result.

6.4.2 Considerations with Simulations

In the example above in Section 6.4.1, we compared results from a simulation with the corresponding theoretical values, so we could see that simulations work. However, if we do not know the theoretical values, two questions will come to mind:

- How many replicates do we need? We only know the values from the simulation converge (get close) to the theoretical values as we increase the number of replicates. Using more replicates will make the simulation run longer on your computer.

- Related to the previous question, how close is close enough? How do you know your value from the simulation is close enough to the theoretical value? There is no way of knowing if the theoretical value is unknown.

6.5 Random Seeds

Some of the functions introduced in this section (for example `sample()` and `rnorm()`) **generate random numbers**. Evaluating the same command multiple times will yield different results. The long-run properties are somewhat predictable (e.g. the sample mean when drawing a large number of random numbers from `rnorm()` will be close to the true mean of the normal distribution), but individual outcomes are not predictable. For example, if we run the command `rnorm(1)` twice, we get two different numbers

```
rnorm(1)
```

```
## [1] -0.07546817
```

```
rnorm(1)
```

```
## [1] -1.177601
```

To stay consistent with principles of reproducible research as mentioned in Section 2.1.1, you may need to reproduce the exact results from a simulation. To generate the same random values each time a command is run, the random generation seed can be set by using the `set.seed()` function. There is one required argument, the seed number.

Seeds operate in a defined order, so once the seed is set, the next random generation commands will use the next seeds in that order. Thus, setting the seed once at the beginning of a program will give the same random values each time the program is run, but each command within the program will generate different results. To yield the same values each time a random generation command is run, the `set.seed()` function must be rerun prior to each command.

```
## With a set seed specified once
set.seed(5)
rnorm(3)
```

```
## [1] -0.8408555  1.3843593 -1.2554919
```

```
rnorm(3)
```

```
## [1]  0.07014277  1.71144087 -0.60290798
```

```
## With a set seed specified each time
set.seed(5)
rnorm(3)
```

```
## [1] -0.8408555  1.3843593 -1.2554919
```

```
set.seed(5)
rnorm(3)
```

```
## [1] -0.8408555  1.3843593 -1.2554919
```

If you copy and paste the code of the simulation from the worked example in Section 6.4.1, you will realize that the density plot and the sample mean and variance of the 10,000 sample means are close, but not exactly the same. This is because I did not set the seed. The overall observation from the simulation would be similar regardless.

Being able to reproduce the same exact results is likely to be required if you want to get your work published.

You may want to consider using `set.seed()` in homework questions that involve generating random numbers, so that you can reproduce your result if needed. Be careful if you clear your environment:

- Running the command `rm(list = ls())` does not remove the seed.
- Clicking the broom icon removes the seed.

Chapter 7

Apply Family

7.1 Overview

We often want to apply the same function separately to each element of a vector. R has the `apply` family of functions that implement loops in specialized use cases. This family of functions is useful since it:

- is compact,
- can iterate over different data structures and return collated results, and
- use anonymous functions to improve performance.

This family of functions include:

- `apply()`: Apply a function over the margins of an array.
- `tapply()`: Apply a function over subsets of a vector.
- `lappy()`: Loop over a list and evaluate a function on each item.
- `sapply()`: Similar to `lapply()` but attempt to simplify the result.

7.2 `apply()`

You learned about the `apply()` function in Section 3.5. It is to apply a function over the margins of an array. To reiterate: it uses three arguments: data, margin, and function. The data argument specifies the matrix or data frame to which the function should be applied, margin specifies whether you want the function applied to the rows, columns, or individual values, and function specifies the function to be applied. For example, to find the average weight and height all respondents in the `Davis` data set from the `carData` package:

```
library(carData)
Data.Davis <- carData::Davis
apply(Data.Davis[,2:3], 2, mean)
```

```
## weight height
##  65.80 170.02
```

```
##2 for columns, 1 for rows
```

You may realize that one could use a loop to obtain these values. The `apply()` function is more succinct, although it does not really run faster than a loop (Peng, 2015).

We often compute sums or means of columns or rows that there functions that produce the same output as equivalent `apply()` versions:

- `rowSums(X)` equivalent to `apply(X, 1, sum)`.
- `rowMeans(X)` equivalent to `apply(X, 1, mean)`.
- `colSums(X)` equivalent to `apply(X, 2, sum)`.
- `colMeans(X)` equivalent to `apply(X, 2, mean)`.

These functions are faster than `apply()` especially when `X` is large (Peng, 2015).

7.3 tapply()

The `tapply()` function is used for a vector of values that can be categorized by a second vector of values. It uses three arguments: data, categorizations, and function. The data input specifies the vector of values that should be grouped and to which the function should be applied, categorizations specifies the category of each corresponding value in the data vector, and function specifies the function to be applied to each resulting group. For example, to find the average weight of female and male respondents in the `Davis` data set.

```
tapply(Davis$weight, Davis$sex, mean)
```

```
##        F        M
## 57.86607 75.89773
```

7.4 lapply() and sapply()

The `lapply()` and `sapply()` functions are both used for lists. Once the specified function is applied, `lapply()` returns a list and `sapply()` attempts to simply the result. They use two arguments: data and function. The data argument specifies the list of elements to which the function should be applied and function specifies the function. If the function has other arguments, they can be specified as additional arguments in `lapply()` and `sapply()`. For example:

```
v1 <- list(a = 1:5, b = rnorm(10))
lapply(v1, mean)
```

```
## $a
## [1] 3
```

```
##
## $b
## [1] -0.07310692
```

```
##get the mean of 1:5 and the mean for 10 draws from standard normal
```

You can use `lapply()` to evaluate a function multiple times each with a different argument. This can be useful if you want to vary the value of an argument in a simulation. For example, to replicate drawing n values from a standard uniform distribution, with $n = 1, 2, 3, 4$.

```
v2 <- 1:4
lapply(v2, runif)
```

```
## [[1]]
## [1] 0.4374119
##
## [[2]]
## [1] 0.9659641 0.1419139
##
## [[3]]
## [1] 0.95497889 0.44473056 0.05936247
##
## [[4]]
## [1] 0.27514918 0.03114826 0.01448234 0.48716903
```

When you pass a function to `lapply()`, `lapply()` takes elements of the list and passes them as the first argument of the function you are applying. In the above example, the first argument of `runif()` is `n`, the number of random draws, and so the elements of the sequence `1:4` are all passed to the `n` argument of `runif()`.

Functions that you pass to `lapply()` may have other arguments. For example, the `runif()` function has a `min` and `max` argument. When these are unspecified, the default values are 0 and 1 respectively, which gives the standard uniform distribution. If these arguments need to be specified, they are passed to `lapply()` next. For example, to replicate drawing n values from a uniform distribution with `min=3` and `max=10`, with $n = 1, 2, 3, 4$.

```
lapply(v2, runif, min = 3, max = 10)
```

```
## [[1]]
## [1] 7.166061
##
## [[2]]
## [1] 7.184971 5.783764
##
## [[3]]
## [1] 5.777813 8.712532 4.646391
##
```

```
## [[4]]
## [1] 8.788092 6.734332 9.502599 6.848563
```

You can also pass your own function to `lapply()`.

Anonymous functions can also be passed to `lapply()` (and other functions in the apply family). Anonymous functions are functions you create that have no variable name and are not stored in the environment. For example, you have a couple of matrices and you want to extract the first row of each one.

```
twoMats <- list(a = matrix(1:4, 2, 2), b = matrix(1:6, 3, 2))

twoMats
```

```
## $a
##      [,1] [,2]
## [1,]    1    3
## [2,]    2    4
##
## $b
##      [,1] [,2]
## [1,]    1    4
## [2,]    2    5
## [3,]    3    6
```

```
lapply(twoMats, function(extract) { extract[1,] })
```

```
## $a
## [1] 1 3
##
## $b
## [1] 1 4
```

I recommend using anonymous functions only if the function is simple and if you are going to only use it once. Otherwise, create a function and name it before `lapply()`, and pass the function to `lapply()`. For example

```
row1 <- function(extract){
  extract[1,]
}

lapply(twoMats, row1)
```

```
## $a
## [1] 1 3
##
## $b
## [1] 1 4
```

Notice that the function `row1` now appears in the environment.

7.5 sapply()

`sapply()` is similar to `lapply()`, but `sapply()` attempts to simplify the result of `lapply()`.

- If the result is a list where every element is length 1, then a vector is returned.
- If the result is a list where every element is a vector of the same length and greater than 1, a matrix is returned, where each column represents each vector.
- Otherwise, a list is returned, just like `lapply()`.

Look at the output from the following, and compare with the output from `lapply()`.

```
sapply(v1, mean)
```

```
##           a           b
##  3.00000000 -0.07310692
```

```
sapply(twoMats, row1)
```

```
##      a b
## [1,] 1 1
## [2,] 3 4
```

I find working with vectors and matrices easier than working with lists, so I prefer using `sapply()` to `lapply()`.

Chapter 8

Data Visualizations (ggplot2)

8.1 Introduction

Data visualizations are tools that summarize data. Consider the visualization in this article from Pew Reasearch. Without actually having access to the actual data, we have a sense of the audience associated with 30 major news sources. Good visualizations are easy to interpret for a wide variety of audiences, and are easier to explain than statistical models.

In this section, you will learn how to create common data visualizations. The choice of data visualization is almost always determined by whether the variable(s) involved is **categorical** or **quantitative**. **Discrete** variables are interesting because depending on the circumstance, they can be viewed as either categorical or quantitative in the context of data visualizations. With discrete variables, the consideration is how many distinct values exist. The more distinct values in the discrete variable, the more likely we want to use visualizations associated with quantitative variables.

Note: some practitioners use the term discrete variable interchangeably with categorical variable. For example, this cheat sheet provided by RStudio. I disagree with this. Discrete variables are quantitative variables where the number of values can be counted, as opposed to continuous variables which are quantitative variables where the number of values cannot be counted. My view is consistent with most academics, but some practitioners interchange discrete with categorical.

We will be using functions from the `ggplot2` package to create visualizations. The `ggplot2` package enables users to create various kinds of data visualizations, beyond the visualizations that can be made in base R.

This section is heavily based on R for Data Science (Wickham et al., 2023), Chapter 1. We will be covering the basics of these visualizations in this section. Feel free to consult the book for a more thorough explanation of how the functions from the `ggplot2` package work. I have added some of my own commentary, as well as some examples on how to create similar visualizations using base R.

You will learn the basics in this section. For further reading, I will suggest Chapters 9 and 11 of Wickham et al. (2023). We also have a full course, STAT 3280, that is dedicated to data visualizations.

8.1.1 Motivating Example

We will use the `penguins` data set from the `palmerpenguins` package (Horst et al., 2020). The data set contains data on penguins near Antarctica, such as measurements on flipper length, body mass, sex, the species, and which island they live in. Suppose we want to see how flipper lengths are related to body mass. Are the body masses of these penguins closely related to their flipper lengths? If so, how strong is this relationship. Is the relationship linear or exponential? Do the relationships vary by species and island?

We will load the `tidyverse` package, which automatically loads the `ggplot2` package, as well as the `palmerpenguins` package and the `ggthemes` package which uses colorblind safe colors.

```
library(tidyverse)
library(palmerpenguins)
library(ggthemes)
```

Notice that loading the `tidyverse` package prints out a couple of messages informing us of functions in the package that conflict with functions in base R (functions with the same name in different packages). As mentioned in Section 3.3, R will use the function from the package that was most recently loaded when the function name is called. You will have to type the name of the package and function if you want to use the function from the package that was loaded earlier, e.g. `stats::filter()` to use `filter()` from the `stats` package.

8.2 Visualizing Two Quantitative Variables

We take a quick peak at the contents of the data frame:

```
Data <- palmerpenguins::penguins
head(Data)
```

```
## # A tibble: 6 x 8
##   species island    bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##   <fct>   <fct>              <dbl>         <dbl>             <int>       <int>
## 1 Adelie  Torgersen           39.1          18.7               181        3750
```

```
## 2 Adelie  Torgersen           39.5          17.4               186        3800
## 3 Adelie  Torgersen           40.3          18                 195        3250
## 4 Adelie  Torgersen           NA            NA                  NA          NA
## 5 Adelie  Torgersen           36.7          19.3               193        3450
## 6 Adelie  Torgersen           39.3          20.6               190        3650
## # i 2 more variables: sex <fct>, year <int>
```

Each row of the data frame should contain measurements on an **observation** (penguin), and each column of the data frame should contain a **variable** (characteristic). We are using the term variable in the context of statistics, and not in the context of computer science (see Section 1.6).

- An **observation** is each object that we collect data on. Often called a data point.
- A **variable** is a characteristic that we measure on each observation.

As mentioned earlier, suppose we want to see how flipper lengths are related to body mass. Notice that both variables, flipper length and body mass, are **quantitative**, and not **categorical**.

- A variable is **quantitative** if its value is numeric and arithmetic operations can be performed.
- A variable is **categorical** if arithmetic operations cannot be performed. In this example, the variables species and island are categorical.

A **scatterplot** is the most common visualization that is used to display two quantitative variables. Often times, one variable can be designated as the **response variable** (often denoted as Y), and the other as the **predictor** (often denoted as X). The predictor is used to forecast or explain changes in the **response variable**. In this context, the body mass is the response variable, and flipper length is the predictor.

To create a scatterplot of these two variables, we can use

```
ggplot(Data, aes(x = flipper_length_mm, y = body_mass_g)) +
  geom_point()
```

```
## Warning: Removed 2 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

From these two lines we code, we can see the basic structure of creating data visualizations with the `ggplot()` function:

1. Use the `ggplot()` function, and supply the name of the data frame, and the x- and/or y- variables via the `aes()` (aesthetics) function. End this line with a `+` operator, and then press enter.

2. In the next line (also called layer), specify the type of visualization we want to create (called `geoms`). For a scatterplot, type `geom_point()`. By convention, the response variable goes on the vertical axis and the predic-

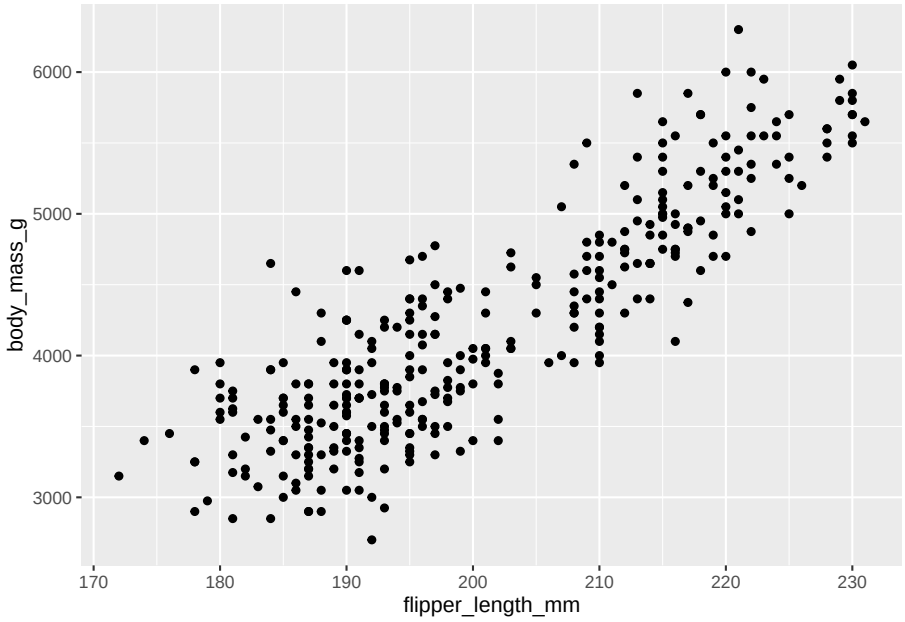


Figure 8.1: Basic Scatterplot with ggplot

tor goes on the horizontal axis. `ggplot()` takes care of this automatically when these are supplied as x- and y- variables via `aes()`.

Notice the output prints a message informing us that 2 observations are missing and are removed. This is unlike some previous functions which output `NA` when missing values are present.

From the scatterplot in Figure 8.1, it looks like flipper length and body mass have a fairly strong positive linear relationship. As flipper length increases, body mass generally increases. If we were to draw an imaginary line through the plots, the plots fall fairly close to the line.

This is the most basic scatterplot that can be created. However, we note in this particular data set that we know that there are 3 species of penguins being measured, so perhaps the relationship between flipper length and body mass may vary across species, we want to add an additional variable `species`. One common way to do this is a scatterplot is to use colors. Also notice the axes have labels that are not the most descriptive, and a title is missing. These can be added via additional arguments in `aes()` or by supplying extra layers.

8.2.1 Adding Aesthetics and Layers

8.2.1.1 Color for adding a categorical variable

To use different colored plots for each species:

```
ggplot(Data, aes(x = flipper_length_mm, y = body_mass_g, color = species)) +
  geom_point()
```

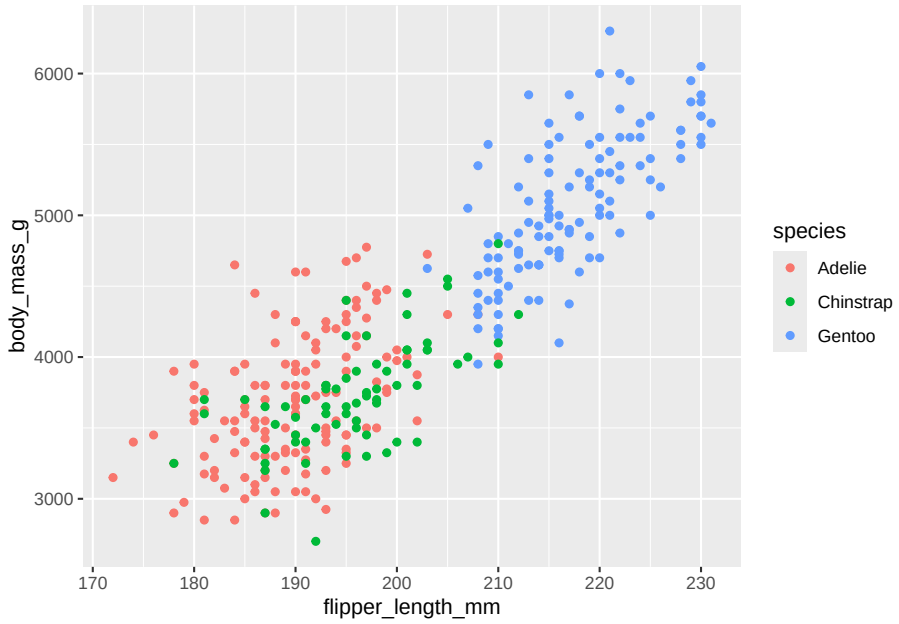


Figure 8.2: Scatterplot using Color via ggplot

Notice that a legend is automatically created as well. To use a colorblind safe palette, add an extra layer `scale_color_colorblind()` (this function comes from the `ggthemes` package that we loaded earlier):

```
ggplot(Data, aes(x = flipper_length_mm, y = body_mass_g, color = species)) +
  geom_point() +
  scale_color_colorblind()
```

Notice that each layer ends with `+` and the next layer starts on a new line.

Based on Figures 8.2 and 8.3, it looks like there are a few clusters in our plots. The Gentoo penguins appear in one cluster, while the Adelie and Chinstrap penguins have their measurements in one cluster. For each species, the body mass is linearly related to the flipper length; although Gentoo penguins have larger body masses and longer flipper lengths than the other two species.

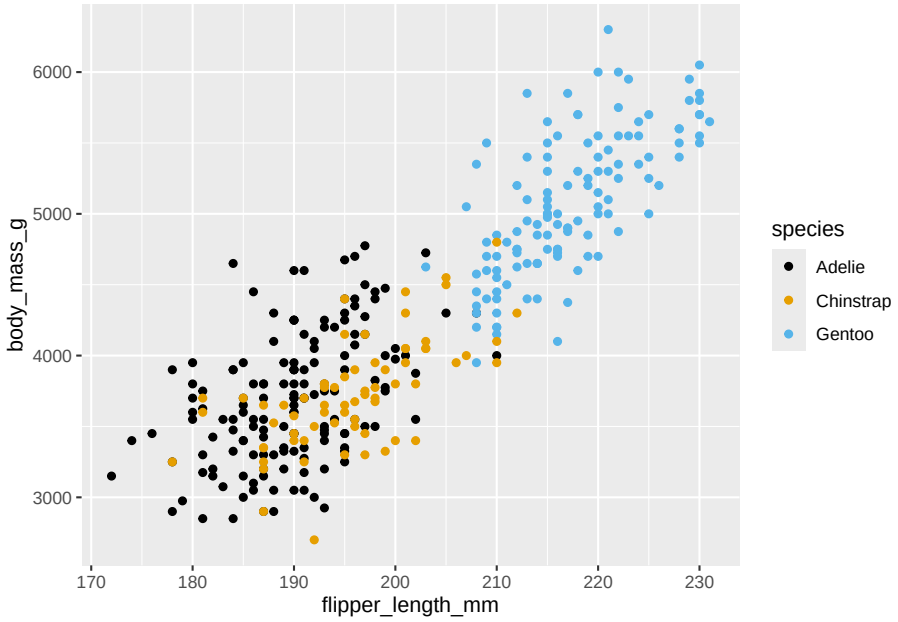
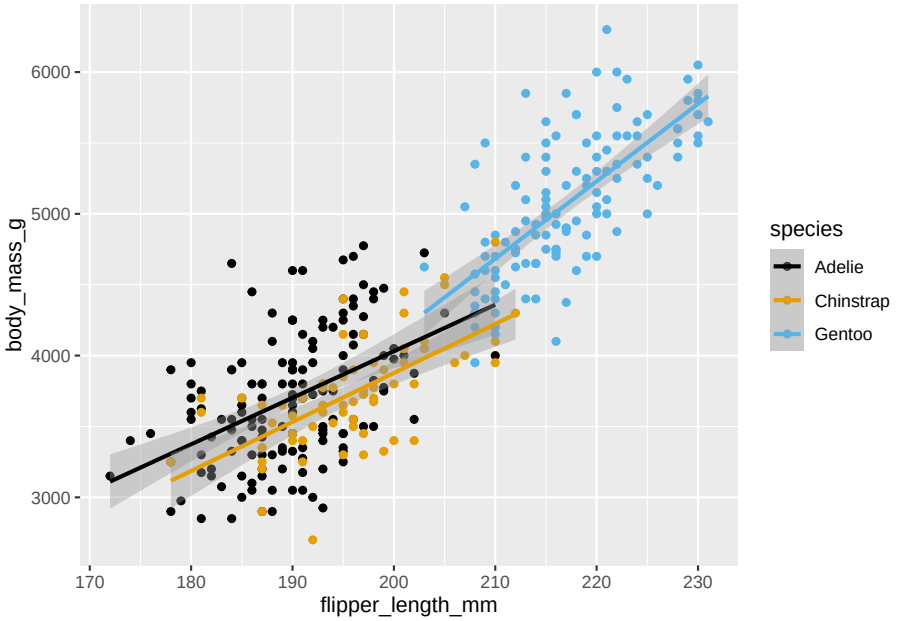


Figure 8.3: Scatterplot using Colorblind Safe Palette via ggplot

8.2.1.2 Overlaying lines and curves

A way to help visualize the relationship between the quantitative variables is to overlay a line of best fit.

```
ggplot(Data, aes(x = flipper_length_mm, y = body_mass_g, color = species)) +
  geom_point() +
  scale_color_colorblind() +
  geom_smooth(method = "lm")
```



We are overlaying a line over each species, where represents the regression line of body mass against flipper length through `geom_smooth` and `method = "lm"`. Notice in Figure **??** that each line has a grey band around them? These are confidence intervals for the regression line. These can be removed if you mainly want to assess if the relationship is linear

```
ggplot(Data, aes(x = flipper_length_mm, y = body_mass_g, color = species)) +
  geom_point() +
  scale_color_colorblind() +
  geom_smooth(method = "lm", se = FALSE)
```

If the observations fall evenly on both sides of the line, the relationship can be approximated by a line. Overlaying regressions lines only make sense if the relationship is linear. If the relationship is nonlinear, we should overlay loess curves, by changing `method` inside `geom_smooth`:

```
ggplot(Data, aes(x = flipper_length_mm, y = body_mass_g, color = species)) +
  geom_point() +
  scale_color_colorblind() +
  geom_smooth(method = "loess", se = FALSE)
```

With real data, relationships are rarely perfectly linear, and so a judgment call has to be made if loess curves or linear regression lines are more appropriate. In this example in Figure 8.5, the curves are pretty close to being straight lines, so using linear regression lines may be chosen as they are simpler than curves.

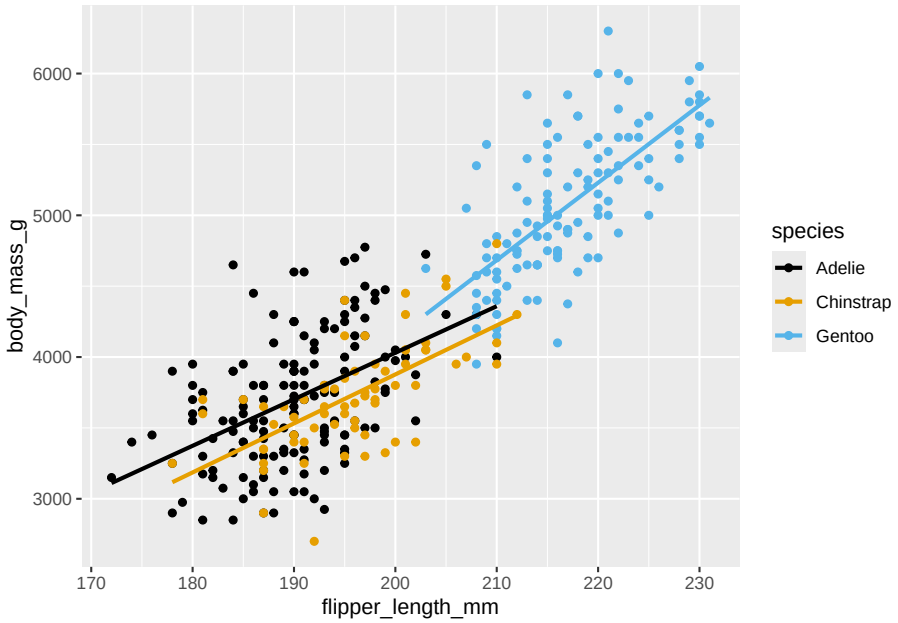


Figure 8.4: Scatterplot with Regression Lines Overlaid (CIs Removed) via ggplot

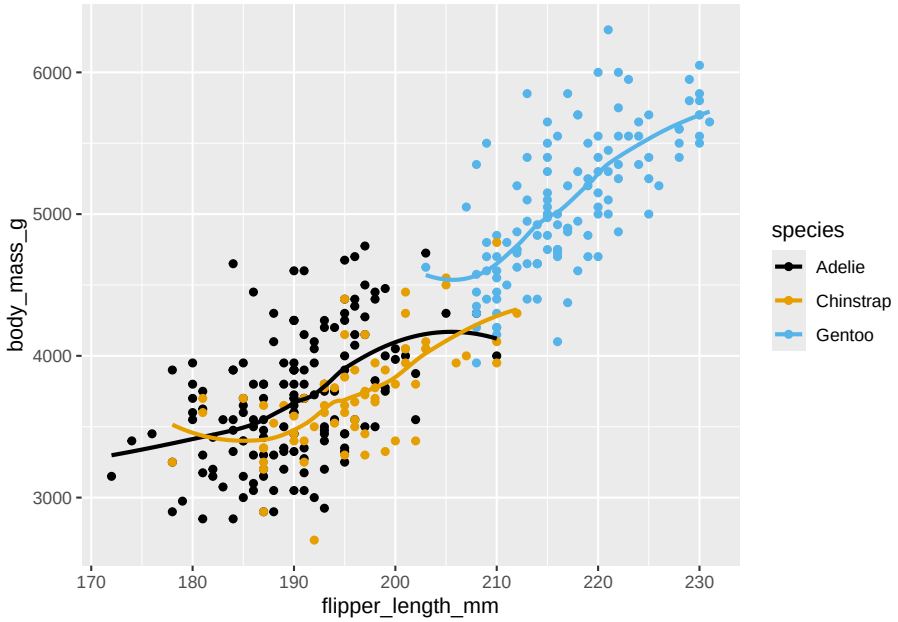


Figure 8.5: Scatterplot with Loess Curves Overlaid via ggplot

8.2.1.3 Adding titles and labeling axes

Now, we are ready to add the a layer to label the axes and give a title to the plot via a layer called `labs`:

```
ggplot(Data, aes(x = flipper_length_mm, y = body_mass_g, color = species)) +
  geom_point() +
  scale_color_colorblind() +
  geom_smooth(method = "lm", se = FALSE) +
  labs(title = "Body Mass and Flipper Length of Penguins",
    x = "Flipper length (mm)", y = "Body mass (g)",
    color = "Species" )
```

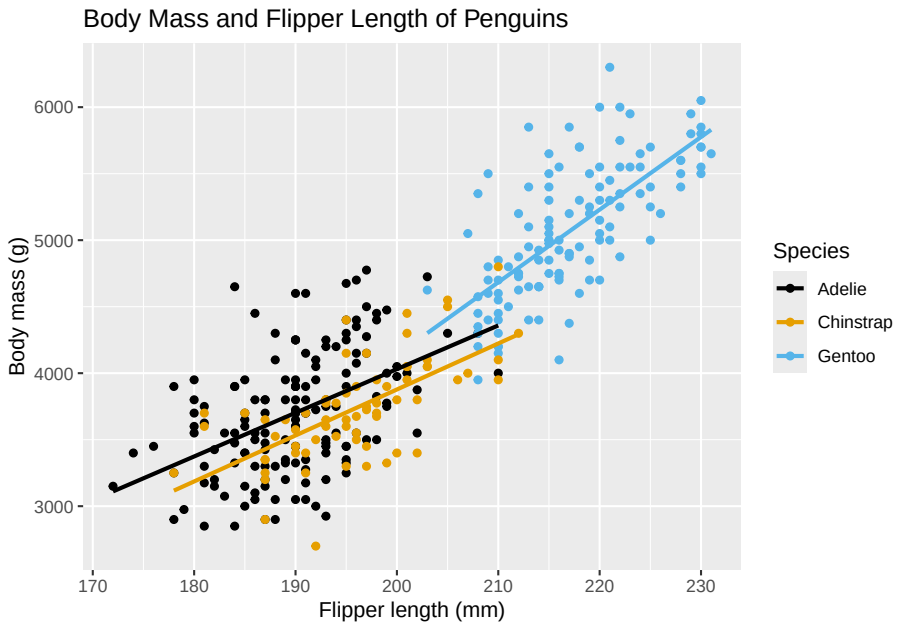


Figure 8.6: Scatterplot with Title and Axes Labeled via ggplot

8.2.2 Base R

To create a scatterplot using base R functions, use the `plot()` function, then supply a vector for the x-variable, then a vector for the y-variable:

```
plot(Data$flipper_length_mm, Data$body_mass_g)
```

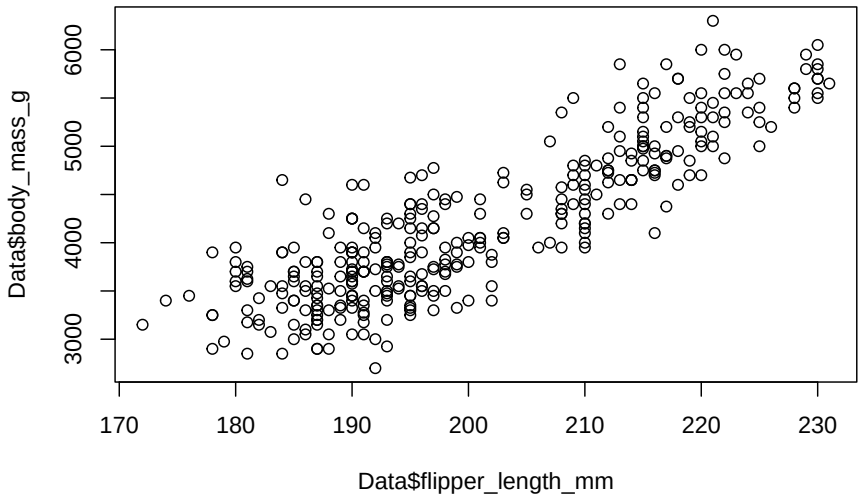


Figure 8.7: Scatterplot via Base R

8.3 Visualizing Two Categorical Variables

A bar chart is often used to visualize the relationship between two categorical variables. Perhaps we want to see if the different species of penguins tend to live in different islands. The most basic bar chart can provide the number of each species in each island:

```
ggplot(Data, aes(x = island, fill = species)) +
  geom_bar() +
  scale_fill_colorblind()
```

```
## Warning: `scale_fill_colorblind()` was deprecated in ggthemes 5.2.0.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

The predictor is `island` and the response variable is `species`. For bar charts, place the response variable in the `fill` argument. Notice the extra layer for the color blind palette is a bit different.

Notice we use `geom_bar()` instead of `geom_point()`. The `geom` tells the `ggplot()` function what kind of visualization we want.

From Figure 8.8, we can see there are an unequal number of penguins in each

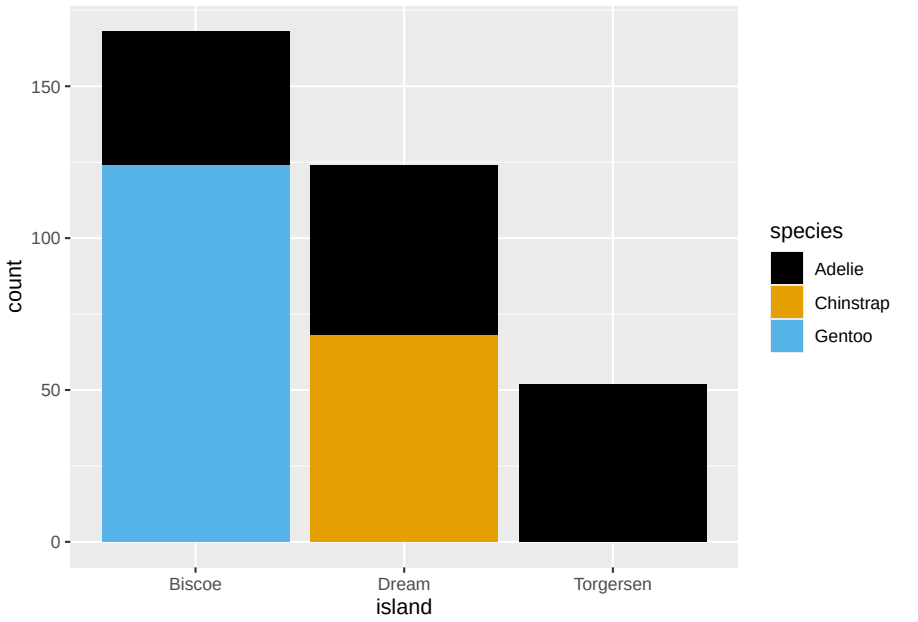


Figure 8.8: Bar Chart via ggplot

island. So using proportions, rather than counts, will be more useful:

```
ggplot(Data, aes(x = island, fill = species)) +
  geom_bar(position = "fill") +
  scale_fill_colorblind()
```

From Figure 8.9, we see that Biscoe island has mostly Gentoos and some Adelies. In Dream island, we have a fairly even proportion of Adelies and Chinstrap. Torgensen island is the island of Adelies. Since the proportions of species are so different across the islands, it appears that the island and species are related. If there is no relationship, the proportion of each species would be the same across the islands.

Notice the label of the y-axis is weird? How would you add a title and change the labels of each axes so they make more sense?

8.3.1 Tables

We can also create **two-way tables** to summarize the data from two categorical variables. These functions are from base R.

```
table(Data$island, Data$species)
```

```
##
```

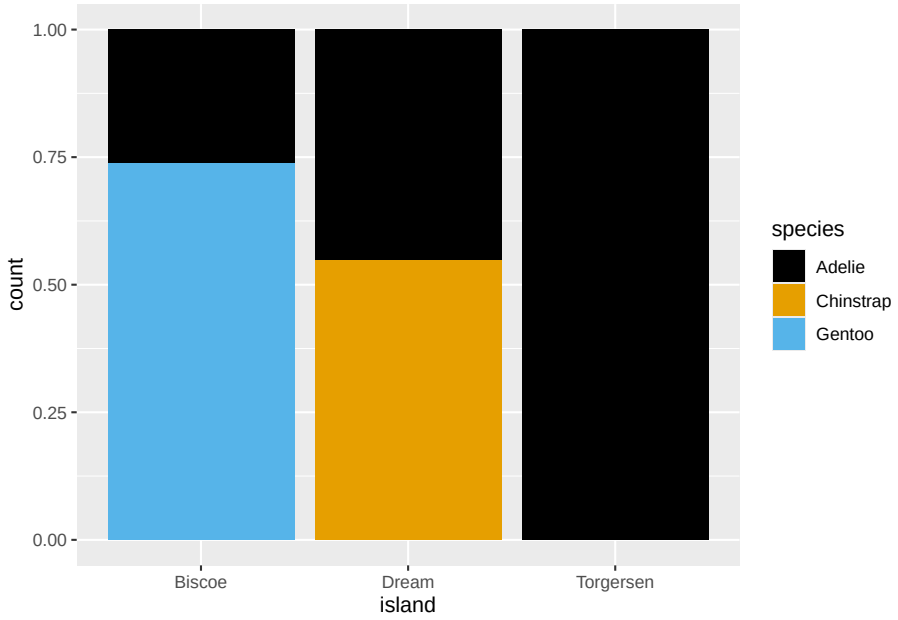


Figure 8.9: Bar Chart with Proportions via ggplot

```
##             Adelie Chinstrap Gentoo
##   Biscoe        44         0    124
##   Dream         56        68      0
##   Torgersen     52         0      0
```

To find proportions of each species in each island:

```
prop.table(table(Data$island, Data$species), margin = 1)
```

```
##
##                Adelie Chinstrap    Gentoo
##   Biscoe    0.2619048 0.0000000 0.7380952
##   Dream     0.4516129 0.5483871 0.0000000
##   Torgersen 1.0000000 0.0000000 0.0000000
```

`margin = 1` means we want the proportions in each row to add up to 1. Use `2` for columns. We have placed the predictor in the rows here.

8.3.2 Base R

As a quick example using base R:

```
barplot(table(Data$species, Data$island))
```

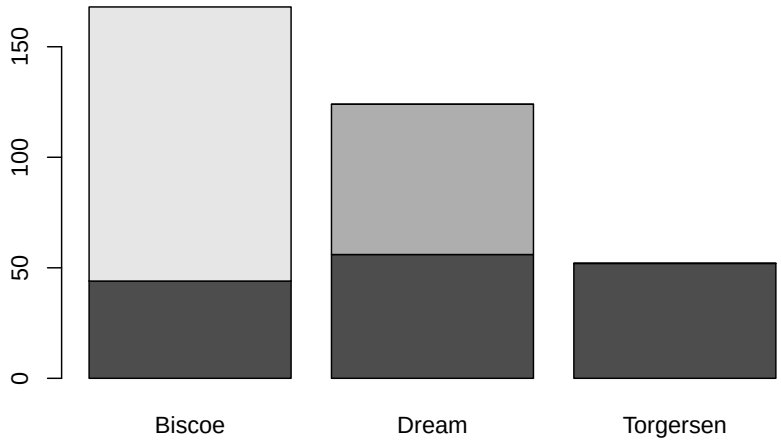


Figure 8.10: Bar Chart via Base R

8.4 Visualizing a Quantitative Variable Across a Categorical Variable

Suppose we want to see if the body mass of the penguins differ across the species. We can create **side by side boxplots**

```
ggplot(Data, aes(x = species, y = body_mass_g)) +
  geom_boxplot()
```

A boxplot represents these values: minimum (lower end of whisker), first quartile (lower end of box), median (line in box), third quartile (upper end of box), and maximum (upper end of whisker). Dots represent outliers based on some rule ($1.5\times$ IQR rule). From Figure 8.11 we see that an outlier that is bigger than the rest and another outlier that is smaller than the rest for Chinstraps. For the Chinstraps, the lower and upper ends of the whiskers represent the values that are the minimum and maximum that are not considered outliers. We can see that Gentoos tend to be larger than Adelies and Chinstraps. The body mass of Adelies and Chinstraps are similar.

How would you add a title and change the labels of each axes so they make more sense?

We can also use density plots of body mass and use different colors for each

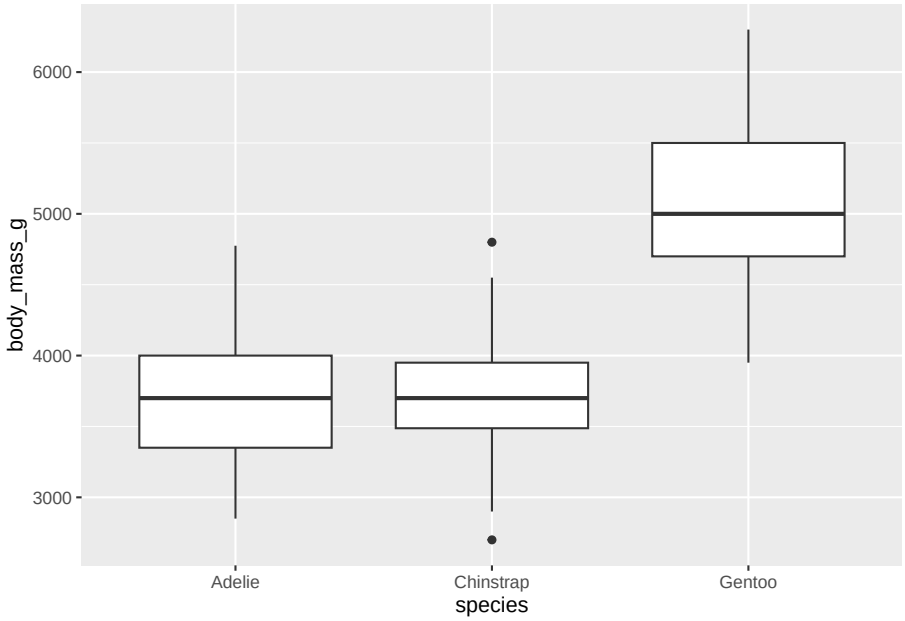


Figure 8.11: Boxplot via ggplot

species and arrive at a similar conclusion.

```
ggplot(penguins, aes(x = body_mass_g, color = species)) +
  geom_density() +
  scale_color_colorblind()
```

Areas under the density curve represent proportion of observations in that range. Based on Figure 8.12, most Chinstraps have body masses between 3000g and 4500g.

How would you add a title and change the labels of each axes so they make more sense?

8.4.1 Base R

As a quick example using base R. For side by side boxplots:

```
boxplot(Data$body_mass_g ~ Data$species) ##y ~ x
```

Using density plots for each species is cumbersome in base R.

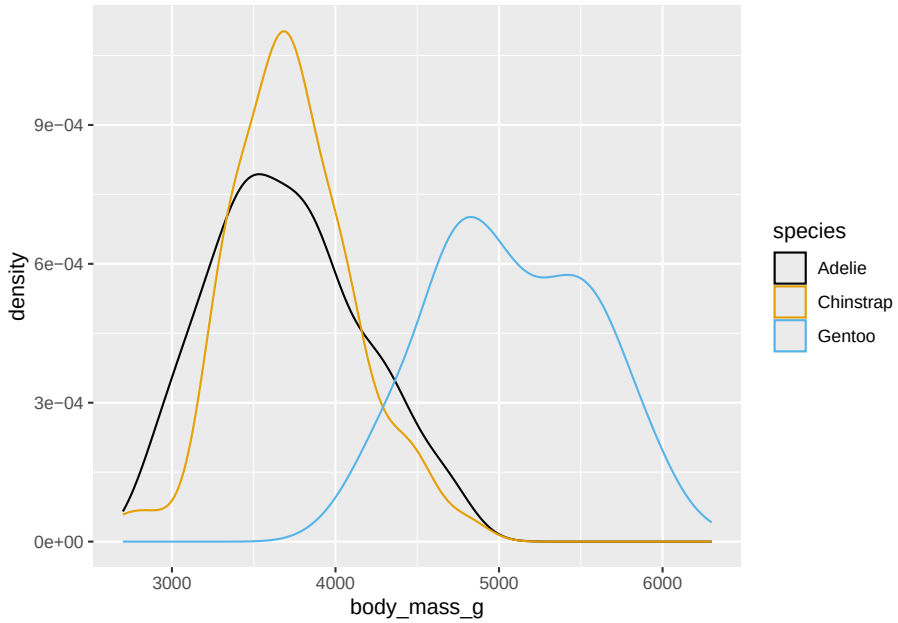


Figure 8.12: Density Plot via ggplot

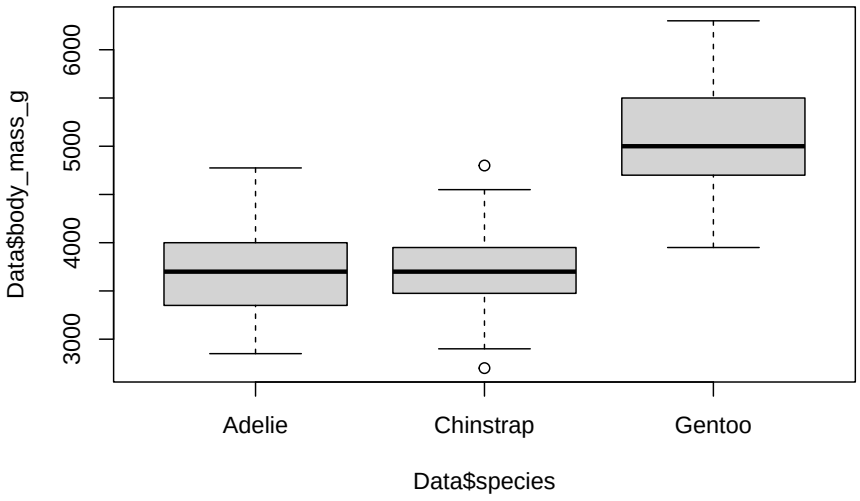


Figure 8.13: Boxplot via Base R

8.5 Adding an Additional Categorical Variable

Most of these plots visualize two variables. In Section 8.2.1.1, we have a scatterplot which incorporates color to highlight a third, categorical variable.

To incorporate another categorical variable, we can use what `ggplot()` calls a `facet_wrap`. It will create the visualization across each value of the categorical variable in the `facet_wrap`. For example,

```
ggplot(Data, aes(x = flipper_length_mm, y = body_mass_g, color = species)) +
  geom_point() +
  scale_color_colorblind() +
  geom_smooth(method = "lm", se = FALSE) +
  labs(title = "Body Mass and Flipper Length of Penguins",
    x = "Flipper length (mm)", y = "Body mass (g)",
    color = "Species" ) +
  facet_wrap(~island)
```

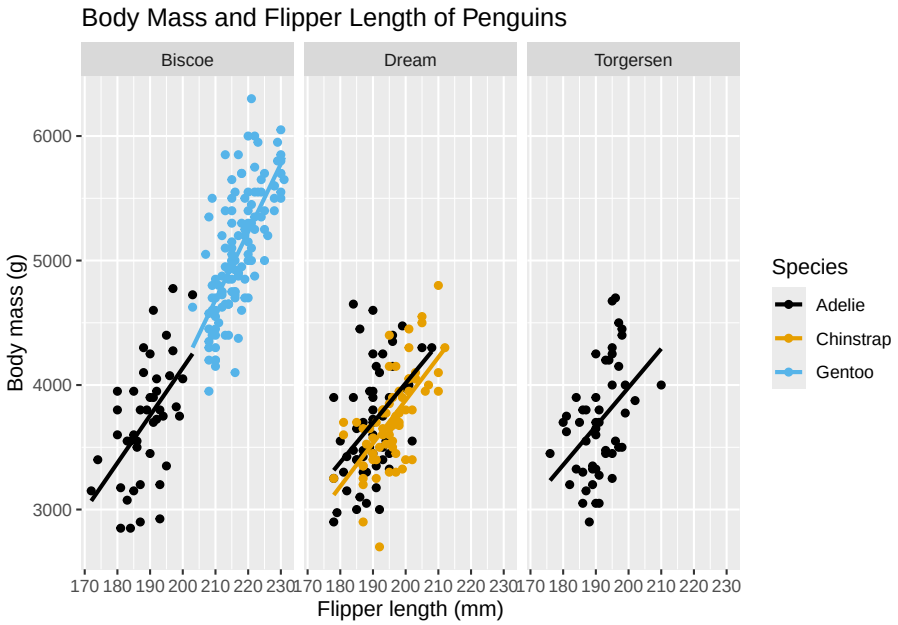


Figure 8.14: Using Facet Wrap with ggplot

The `facet_wrap` essentially subsets the data frame based on the variable supplied to it.

8.6 Visualizing Single Variables

Thus far, we have looked at visualizing two (or more) variables. In such visualizations, we usually want to see two (or more) variables are related.

Oftentimes, we need to see how are the values of a single variable distributed. Again, our choice of visualization depends on whether the variable is categorical or quantitative.

8.6.1 Single Categorical Variable

A bar chart can also be used for a single categorical variable:

```
ggplot(Data, aes(x = species)) +
  geom_bar()
```

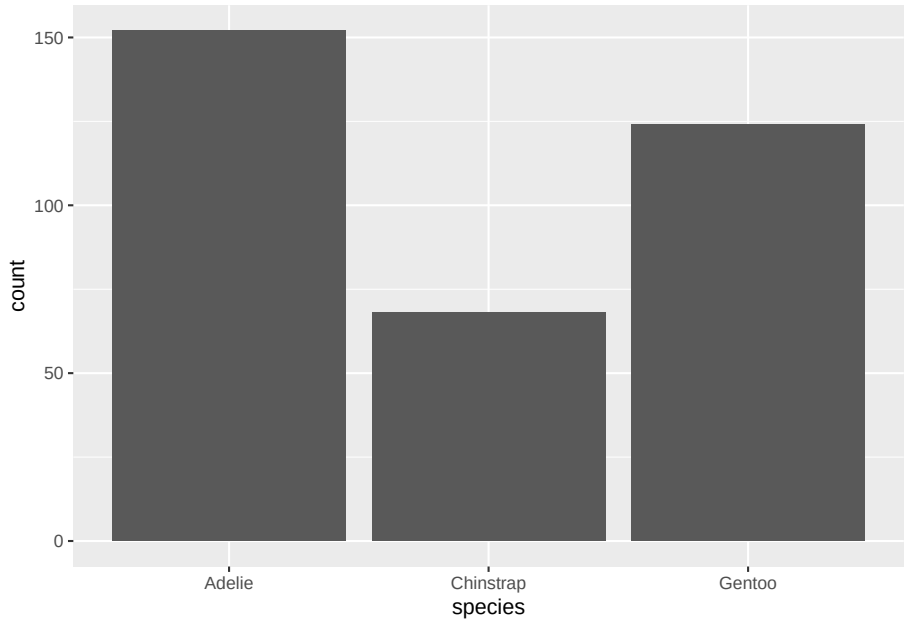
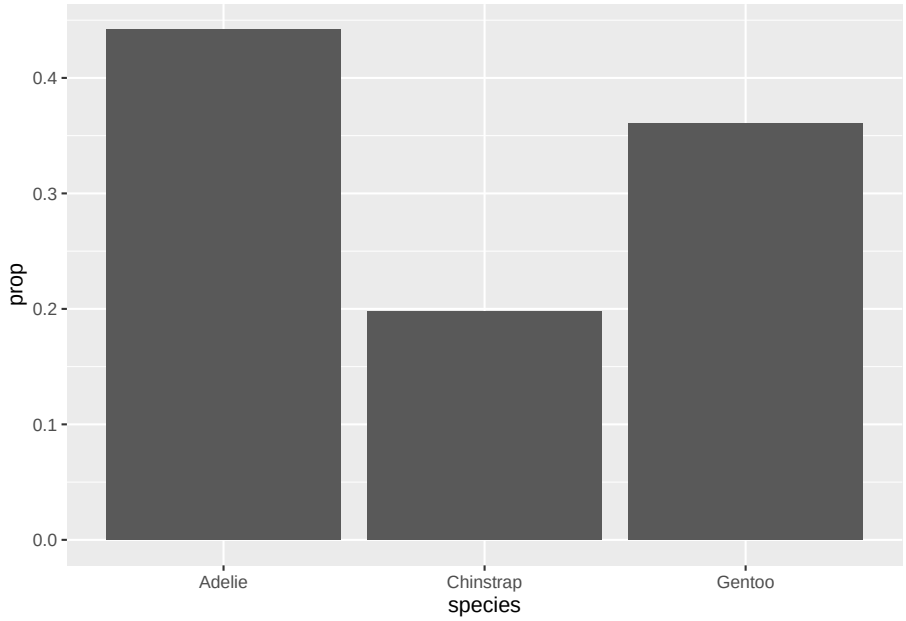


Figure 8.15: Barchart via ggplot

By default, a bar chart showing counts is displayed. If we want proportions instead:

```
ggplot(Data, aes(x = species, y = after_stat(prop), group = 1)) +
  geom_bar()
```

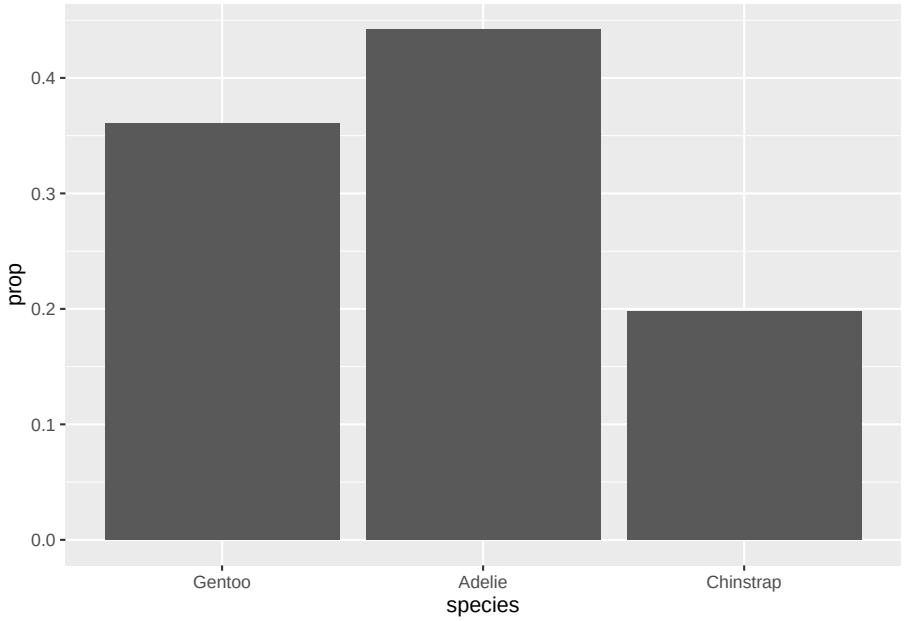


The argument `y = after_stat(prop)` tells the function that we want the proportions on the y axis. `group = 1` means we want the proportions across all bars to add up to 1.

If you want to change the order of the species that are displayed:

```
Data$species <- factor(Data$species, levels=c("Gentoo","Adelie","Chinstrap"))

ggplot(Data, aes(x = species, y = after_stat(prop), group = 1)) +
  geom_bar()
```



Changing the order may not make much sense in this example, but they will if the categorical variable is **ordinal**. In such variables, there is an order in the values of the categories, for example, shirt size.

Tables can also be easily created:

```
table(Data$species)
```

```
##
##    Gentoo    Adelie Chinstrap
##       124       152        68
```

```
prop.table(table(Data$species))
```

```
##
##    Gentoo    Adelie Chinstrap
## 0.3604651 0.4418605 0.1976744
```

8.6.2 Single Quantitative Variable

A boxplot or a density plot are commonly used to summarize the distribution of a single quantitative variable.

```
ggplot(Data, aes(y = body_mass_g)) +
  geom_boxplot()
```

```
ggplot(Data, aes(x = body_mass_g)) +
  geom_density()
```

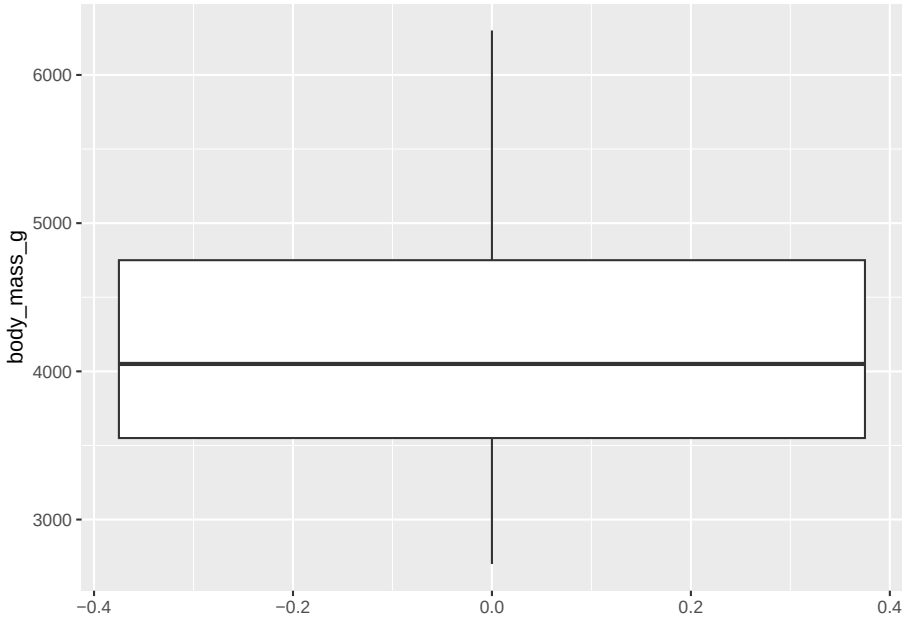


Figure 8.16: Boxplot via ggplot

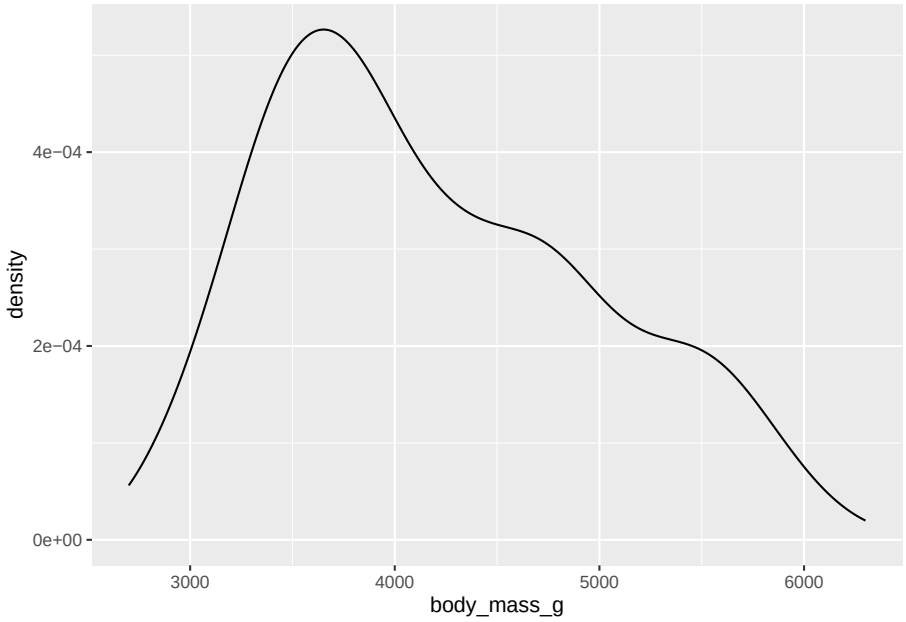


Figure 8.17: Density Plot via ggplot

Or with base R

```
boxplot(Data$body_mass_g)
```

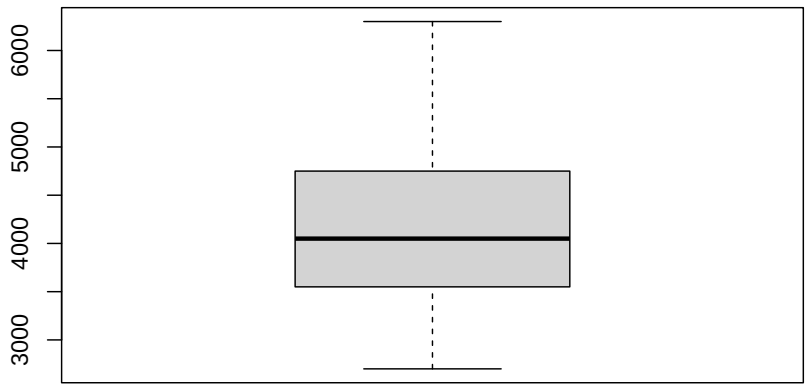


Figure 8.18: Boxplot via Base R

```
plot(density(Data$body_mass_g, na.rm=TRUE))
```

8.7 Cheat Sheet

We have covered basic visualizations with `ggplot`. Feel free to refer to this cheat sheet provided by RStudio for more.

8.8 Visualizations from Simulations

You may realize that `ggplot()` assumes you are working with a data frame. However, when running your own simulations, results are often stored in other kinds of data structures such as vectors and matrices. In such instances, using a base R function to create visuals may be more efficient, as we can skip a step by not needing to create a data frame. For example, we simulate 20 values from a standard normal and want a plot to show their values. Using base R:

```
plot(rnorm(20))
```

Using `ggplot()` involves more code to type:

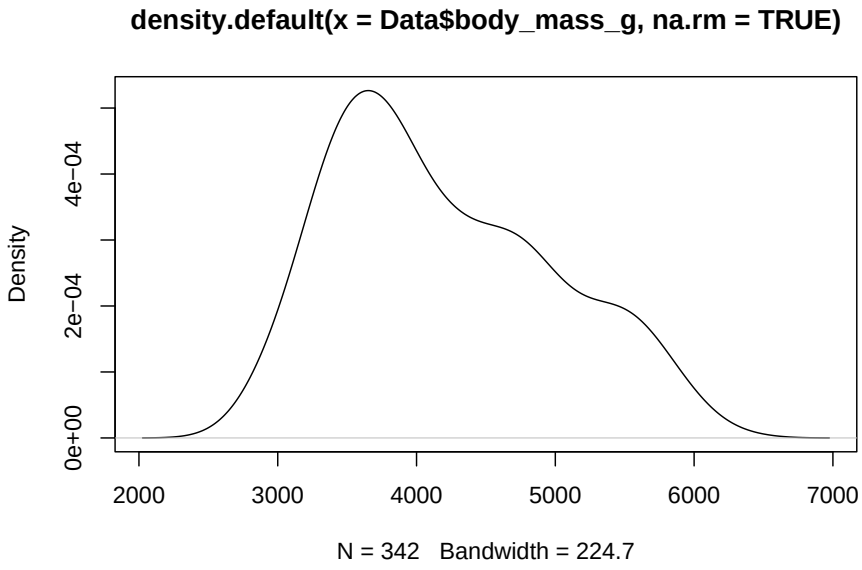


Figure 8.19: Density Plot via Base R

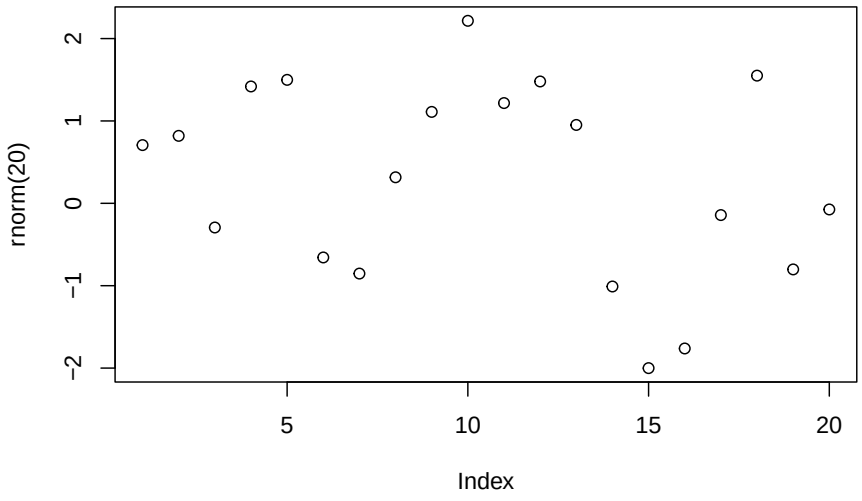


Figure 8.20: Visualization Simulations with Base R

```
y <- rnorm(20)
x <- 1:20
newDF <- data.frame(x,y)
ggplot(newDF, aes(x = x, y = y)) +
  geom_point()
```

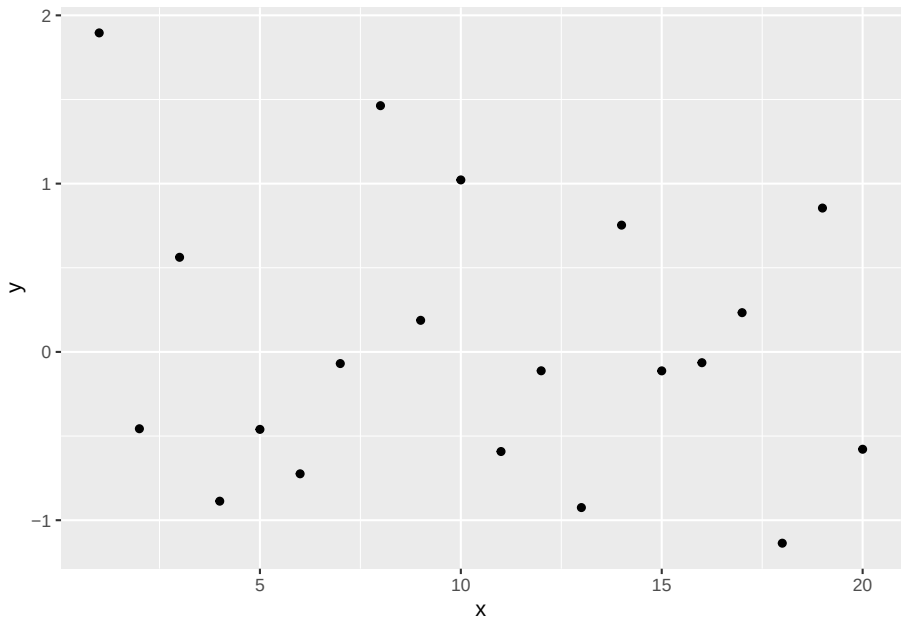


Figure 8.21: Visualization Simulations with ggplot

8.9 ggplot vs Base R

So which to use? `ggplot()` produces nicer visuals with much more customization, but base R can be quicker to type up.

- If I am starting my analysis, I usually start with base R.
- For simple visualizations with no or little customization needed, use base R.
- Use `ggplot()` for more complicated visualizations.
- Use `ggplot()` if presenting to an audience.

Note: I did not add titles and labels to axes in a lot of these plots. Feel free to practice adding them on your own. Visualizations should always be titled and axes should always be labeled.

Chapter 9

Data Wrangling (dplyr)

9.1 Motivation

The data structure that we will be dealing with most often when working with real data will be data frames. A data frame can be viewed like an EXCEL spreadsheet, where data are stored in rows and columns. As you saw in Section 8, we want the data frame to have this basic structure:

- Each row of the data frame corresponds to an **observation**.
- Each column of the data frame corresponds to a **variable**.

Sometimes, data are not structured in this way, and we will have to transform the data to take on this basic data structure. This process is called **data wrangling**. The most common and basic operations to transform the data are:

- Select a subset of rows of the data frame based on some criteria.
- Rearrange rows in some order.
- Select a subset of columns of the data frame.
- Create new columns based on existing variables.
- Change column names.
- Change the order of columns.

You may realize that you can already perform some of these operations in Sections 1 and 3. In this section, you will learning how to perform these operations using another core package from the `tidyverse` called `dplyr`, which has gained popularity. Its advocates say the syntax associated with `dplyr` is more readable for the human eye than the syntax with base R, although its detractors will say that `dplyr` introduces many function names and its syntax strays from base R.

This section is heavily based on R for Data Science (Wickham et al., 2023), Chapter 3. We will be covering the basics of data wrangling with `dplyr`. Feel free to consult the book for a more thorough explanation of how these functions

work. I have added some of my own commentary.

9.1.1 Motivating Example

For this section, we will use a data set on flights that departed from New York City in 2013. It comes from the `nycflights13` package. It contains all 336,776 flights that departed from New York City in 2013 and comes from the US Bureau of Transportation Statistics.

```
library(tidyverse)
library(nycflights13)
Data <- nycflights13::flights
```

9.1.2 dplyr Basics

`dplyr` functions typically work in the following manner:

1. Specify data frame.
2. Select which rows and columns to work on.
3. Output data frame.

One of the key operators that we will use is called the **pipe**, `|>`. Pipes pass the result of one function straight into the next function, which makes code clean, linear, and easy to read from left to right and from top to bottom like a sentence. The easiest way to interpret a pipe is "and then". For example

```
Data |>
  filter(dest == "IAH") |>
  group_by(month) |>
  summarize(arr_delay = mean(arr_delay, na.rm = TRUE))
```

```
## # A tibble: 12 x 2
##    month arr_delay
##    <int>     <dbl>
##  1     1     4.16
##  2     2     5.40
##  3     3    -1.19
##  4     4    14.8
##  5     5     0.972
##  6     6    11.1
##  7     7    11
##  8     8     0.705
##  9     9   -10.6
## 10    10     1.81
## 11    11    -1.78
## 12    12    14.5
```

What this block of code does is:

1. Takes the data frame called `Data`, and then
2. Filters the observations with destination being `IAH`, and then
3. Groups the observations by month, and then
4. Computes the mean arrival delay for each month.

Compare with the code using base R without pipes:

```
tapply(Data[Data$dest == "IAH",]$arr_delay, Data[Data$dest == "IAH",]$month, mean, na.rm = TRUE)
```

```
##           1           2           3           4           5           6
##   4.1627907   5.4023904  -1.1875000  14.7666667   0.9722222  11.1023490
##           7           8           9          10          11          12
##  11.0000000   0.7054010 -10.6473684   1.8080645  -1.7763158  14.5000000
```

People tend to like the linearity and flow of pipes.

The keyboard shortcut for a pipe is Ctrl + Shift + M. You may have to change this in RStudio options, via Tools, Global Options, Code and check the box "Use native pipe operator". There is another pipe operator `%>%` which comes from the `magrittr` package. `|>` is part of base R.

9.2 Operations on Rows

9.2.1 Select Rows Based on Condition

The `filter()` function from the `dplyr` package selects rows of a data frame based on a condition. For example, to select flights with arrival delays of more than 120 minutes:

```
Data |>
  filter(dep_delay > 120)
```

```
## # A tibble: 9,723 x 19
##     year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##    <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>
##  1  2013     1     1      848           1835       853     1001           1950
##  2  2013     1     1      957            733       144     1056            853
##  3  2013     1     1     1114            900       134     1447           1222
##  4  2013     1     1     1540           1338       122     2020           1825
##  5  2013     1     1     1815           1325       290     2120           1542
##  6  2013     1     1     1842           1422       260     1958           1535
##  7  2013     1     1     1856           1645       131     2212           2005
##  8  2013     1     1     1934           1725       129     2126           1855
##  9  2013     1     1     1938           1703       155     2109           1823
## 10  2013     1     1     1942           1705       157     2124           1830
## # i 9,713 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
```

```
## #   hour <dbl>, minute <dbl>, time_hour <dttm>
```

You can use an and operator, `&`, for multiple conditions such as arrival delays of more than 120 minutes and departed before 1200:

```
Data |>
  filter(dep_delay > 120 & dep_time < 1200)
```

```
## # A tibble: 1,353 x 19
##     year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##    <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>
##  1  2013     1     1      848           1835       853     1001           1950
##  2  2013     1     1      957            733       144     1056            853
##  3  2013     1     1     1114            900       134     1447           1222
##  4  2013     1     2      126           2250       156      233           2359
##  5  2013     1     2      833            558       155     1018            727
##  6  2013     1     2      923            624       179     1051            758
##  7  2013     1     3       50           2145       185      203           2311
##  8  2013     1     3      235           2359       156      700            437
##  9  2013     1     3      854            630       144     1057            810
## 10  2013     1     3      909            700       129     1103            850
## # i 1,343 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dttm>
```

You can use an or operator, `|`, for example, to find flights in January and February:

```
Data |>
  filter(month == 1 | month == 2)
```

```
## # A tibble: 51,955 x 19
##     year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##    <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>
##  1  2013     1     1      517            515         2      830            819
##  2  2013     1     1      533            529         4      850            830
##  3  2013     1     1      542            540         2      923            850
##  4  2013     1     1      544            545        -1     1004           1022
##  5  2013     1     1      554            600        -6      812            837
##  6  2013     1     1      554            558        -4      740            728
##  7  2013     1     1      555            600        -5      913            854
##  8  2013     1     1      557            600        -3      709            723
##  9  2013     1     1      557            600        -3      838            846
## 10  2013     1     1      558            600        -2      753            745
## # i 51,945 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
```

```
## #   hour <dbl>, minute <dbl>, time_hour <dttm>
```

The `%in%` operator can be used in place of `|` and `==`. It keeps rows where the variable equals one of the values on the right:

```
Data |>
  filter(month %in% c(1, 2))
```

```
## # A tibble: 51,955 x 19
##     year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##    <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>
##  1  2013     1     1      517            515         2      830            819
##  2  2013     1     1      533            529         4      850            830
##  3  2013     1     1      542            540         2      923            850
##  4  2013     1     1      544            545        -1     1004           1022
##  5  2013     1     1      554            600        -6      812            837
##  6  2013     1     1      554            558        -4      740            728
##  7  2013     1     1      555            600        -5      913            854
##  8  2013     1     1      557            600        -3      709            723
##  9  2013     1     1      557            600        -3      838            846
## 10  2013     1     1      558            600        -2      753            745
## # i 51,945 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dttm>
```

Notice in these examples that the resulting data frame is printed but not stored. Remember to assign the output to a variable name if you wish to store it. For example, to store the flights on New Year's day:

```
jan1 <- Data |>
  filter(month == 1 & day == 1)
```

9.2.2 Arrange Rows in Order

The `arrange()` function arranges the rows based on the value in columns. Specifying multiple columns will arrange by the first column, then by the second, and so on. The default order is small to large. To arrange by month, then by day, and then by departure time:

```
Data |>
  arrange(month, day, dep_time)
```

```
## # A tibble: 336,776 x 19
##     year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##    <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>
##  1  2013     1     1      517            515         2      830            819
##  2  2013     1     1      533            529         4      850            830
```

```
##  3  2013     1     1      542            540         2      923            850
##  4  2013     1     1      544            545        -1     1004           1022
##  5  2013     1     1      554            600        -6      812            837
##  6  2013     1     1      554            558        -4      740            728
##  7  2013     1     1      555            600        -5      913            854
##  8  2013     1     1      557            600        -3      709            723
##  9  2013     1     1      557            600        -3      838            846
## 10  2013     1     1      558            600        -2      753            745
## # i 336,766 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dttm>
```

To change the order of just the departure times:

```
Data |>
  arrange(month, day, desc(dep_time))
```

```
## # A tibble: 336,776 x 19
##     year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##    <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>
##  1  2013     1     1     2356           2359        -3      425            437
##  2  2013     1     1     2353           2359        -6      425            445
##  3  2013     1     1     2353           2359        -6      418            442
##  4  2013     1     1     2343           1724       379      314           1938
##  5  2013     1     1     2327           2250        37       32           2359
##  6  2013     1     1     2326           2130       116      131             18
##  7  2013     1     1     2323           2200        83       22           2313
##  8  2013     1     1     2312           2000       192       21           2110
##  9  2013     1     1     2310           2255        15       24             15
## 10  2013     1     1     2307           2245        22       32           2357
## # i 336,766 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dttm>
```

9.2.3 Counting

To count the number of occurrences of each origin and destination combination:

```
Data |>
  count(origin, dest, sort = TRUE)
```

```
## # A tibble: 224 x 3
##    origin dest      n
##    <chr>  <chr> <int>
##  1 JFK    LAX   11262
##  2 LGA    ATL   10263
```

```
##  3 LGA    ORD    8857
##  4 JFK    SFO    8204
##  5 LGA    CLT    6168
##  6 EWR    ORD    6100
##  7 JFK    BOS    5898
##  8 LGA    MIA    5781
##  9 JFK    MCO    5464
## 10 EWR    BOS    5327
## # i 214 more rows
```

`sort = TRUE` sorts from largest to smallest.

9.3 Operations on Columns

When operating on columns, we usually want to

- Create new columns (variables) based on existing columns.
- Select specific columns.
- Change the names of columns.
- Change the position of the columns.

9.3.1 Create New Columns

You can create new columns based on existing columns using `mutate()`. For example, to create a new column called `speed` in miles per hour:

```
Data |>
  mutate(
    speed = distance / air_time * 60
  )
```

```
## # A tibble: 336,776 x 20
##     year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##    <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>
##  1  2013     1     1      517            515         2      830            819
##  2  2013     1     1      533            529         4      850            830
##  3  2013     1     1      542            540         2      923            850
##  4  2013     1     1      544            545        -1     1004           1022
##  5  2013     1     1      554            600        -6      812            837
##  6  2013     1     1      554            558        -4      740            728
##  7  2013     1     1      555            600        -5      913            854
##  8  2013     1     1      557            600        -3      709            723
##  9  2013     1     1      557            600        -3      838            846
## 10  2013     1     1      558            600        -2      753            745
## # i 336,766 more rows
## # i 12 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
```

```
## #   hour <dbl>, minute <dbl>, time_hour <dttm>, speed <dbl>
```

By default, the new column is placed as the last column. You can add `.before = 1` within `mutate()` to place it before column 1.

```
Data |>
  mutate(
    speed = distance / air_time * 60,
    .before = 1
  )
```

```
## # A tibble: 336,776 x 20
##    speed  year month   day dep_time sched_dep_time dep_delay arr_time
##    <dbl> <int> <int> <int>    <int>          <int>     <dbl>    <int>
##  1  370.  2013     1     1      517            515         2      830
##  2  374.  2013     1     1      533            529         4      850
##  3  408.  2013     1     1      542            540         2      923
##  4  517.  2013     1     1      544            545        -1     1004
##  5  394.  2013     1     1      554            600        -6      812
##  6  288.  2013     1     1      554            558        -4      740
##  7  404.  2013     1     1      555            600        -5      913
##  8  259.  2013     1     1      557            600        -3      709
##  9  405.  2013     1     1      557            600        -3      838
## 10  319.  2013     1     1      558            600        -2      753
## # i 336,766 more rows
## # i 12 more variables: sched_arr_time <int>, arr_delay <dbl>, carrier <chr>,
## #   flight <int>, tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>,
## #   distance <dbl>, hour <dbl>, minute <dbl>, time_hour <dttm>
```

To place it after column 1:

```
Data |>
  mutate(
    speed = distance / air_time * 60,
    .after = 1
  )
```

```
## # A tibble: 336,776 x 20
##     year speed month   day dep_time sched_dep_time dep_delay arr_time
##    <int> <dbl> <int> <int>    <int>          <int>     <dbl>    <int>
##  1  2013  370.     1     1      517            515         2      830
##  2  2013  374.     1     1      533            529         4      850
##  3  2013  408.     1     1      542            540         2      923
##  4  2013  517.     1     1      544            545        -1     1004
##  5  2013  394.     1     1      554            600        -6      812
##  6  2013  288.     1     1      554            558        -4      740
##  7  2013  404.     1     1      555            600        -5      913
##  8  2013  259.     1     1      557            600        -3      709
```

```
##  9  2013  405.     1     1      557            600        -3      838
## 10  2013  319.     1     1      558            600        -2      753
## # i 336,766 more rows
## # i 12 more variables: sched_arr_time <int>, arr_delay <dbl>, carrier <chr>,
## #   flight <int>, tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>,
## #   distance <dbl>, hour <dbl>, minute <dbl>, time_hour <dttm>
```

Alternatively, using `.keep` within `mutate()` will only keep the columns that were involved in `mutate()`:

```
Data |>
  mutate(
    speed = distance / air_time * 60,
    .keep = "used"
  )
```

```
## # A tibble: 336,776 x 3
##    air_time distance speed
##       <dbl>    <dbl> <dbl>
##  1      227     1400  370.
##  2      227     1416  374.
##  3      160     1089  408.
##  4      183     1576  517.
##  5      116      762  394.
##  6      150      719  288.
##  7      158     1065  404.
##  8       53      229  259.
##  9      140      944  405.
## 10      138      733  319.
## # i 336,766 more rows
```

9.3.2 Select Columns

When working with data frames with many columns, you may want to only focus on a subset of the columns. To select only the departure and arrival delays:

```
Data |>
  select(dep_delay, arr_delay)
```

```
## # A tibble: 336,776 x 2
##    dep_delay arr_delay
##        <dbl>     <dbl>
##  1         2        11
##  2         4        20
##  3         2        33
##  4        -1       -18
##  5        -6       -25
##  6        -4        12
```

```
##  7        -5        19
##  8        -3       -14
##  9        -3        -8
## 10        -2         8
## # i 336,766 more rows
```

There are a number of helper functions you can use within `select()`. These can be useful if there is a pattern in the naming of the columns:

- `starts_with("abc")`: matches names that begin with "abc".
- `ends_with("xyz")`: matches names that end with "xyz".
- `contains("ijk")`: matches names that contain "ijk".
- `num_range("x", 1:3)`: matches `x1`, `x2` and `x3`.

You can rename variables as you `select()` them.

```
Data |>
  select(tail_num = tailnum)
```

```
## # A tibble: 336,776 x 1
##    tail_num
##    <chr>
##  1 N14228
##  2 N24211
##  3 N619AA
##  4 N804JB
##  5 N668DN
##  6 N39463
##  7 N516JB
##  8 N829AS
##  9 N593JB
## 10 N3ALAA
## # i 336,766 more rows
```

The new name appears on the left-hand side of the `=`, and the old variable appears on the right-hand side.

9.3.3 Rename Columns

If you want to keep all the existing variables and just want to rename a few, you can use `rename()`:

```
flights |>
  rename(depTime = dep_time)
```

```
## # A tibble: 336,776 x 19
##     year month   day depTime sched_dep_time dep_delay arr_time sched_arr_time
##    <int> <int> <int>   <int>          <int>     <dbl>    <int>          <int>
##  1  2013     1     1     517            515         2      830            819
```

```
##  2  2013     1     1      533            529         4      850            830
##  3  2013     1     1      542            540         2      923            850
##  4  2013     1     1      544            545        -1     1004           1022
##  5  2013     1     1      554            600        -6      812            837
##  6  2013     1     1      554            558        -4      740            728
##  7  2013     1     1      555            600        -5      913            854
##  8  2013     1     1      557            600        -3      709            723
##  9  2013     1     1      557            600        -3      838            846
## 10  2013     1     1      558            600        -2      753            745
## # i 336,766 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dttm>
```

9.3.4 Relocate Columns

You can use `relocate()` to move variables around. You may want to place important ones in front, or group related variables together. By default, the variables are moved to the front:

```
Data |>
  relocate(time_hour, air_time)
```

```
## # A tibble: 336,776 x 19
##    time_hour           air_time  year month   day dep_time sched_dep_time
##    <dttm>                 <dbl> <int> <int> <int>    <int>          <int>
##  1 2013-01-01 05:00:00      227  2013     1     1      517            515
##  2 2013-01-01 05:00:00      227  2013     1     1      533            529
##  3 2013-01-01 05:00:00      160  2013     1     1      542            540
##  4 2013-01-01 05:00:00      183  2013     1     1      544            545
##  5 2013-01-01 06:00:00      116  2013     1     1      554            600
##  6 2013-01-01 05:00:00      150  2013     1     1      554            558
##  7 2013-01-01 06:00:00      158  2013     1     1      555            600
##  8 2013-01-01 06:00:00       53  2013     1     1      557            600
##  9 2013-01-01 06:00:00      140  2013     1     1      557            600
## 10 2013-01-01 06:00:00      138  2013     1     1      558            600
## # i 336,766 more rows
## # i 12 more variables: dep_delay <dbl>, arr_time <int>, sched_arr_time <int>,
## #   arr_delay <dbl>, carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
## #   dest <chr>, distance <dbl>, hour <dbl>, minute <dbl>
```

Or you can specify where to place them after or before:

```
Data |>
  relocate(time_hour, air_time, .after = day)
```

```
## # A tibble: 336,776 x 19
##     year month   day time_hour           air_time dep_time sched_dep_time
```

```
##    <int> <int> <int> <dttm>                <dbl>    <int>          <int>
##  1  2013     1     1 2013-01-01 05:00:00     227      517            515
##  2  2013     1     1 2013-01-01 05:00:00     227      533            529
##  3  2013     1     1 2013-01-01 05:00:00     160      542            540
##  4  2013     1     1 2013-01-01 05:00:00     183      544            545
##  5  2013     1     1 2013-01-01 06:00:00     116      554            600
##  6  2013     1     1 2013-01-01 05:00:00     150      554            558
##  7  2013     1     1 2013-01-01 06:00:00     158      555            600
##  8  2013     1     1 2013-01-01 06:00:00      53      557            600
##  9  2013     1     1 2013-01-01 06:00:00     140      557            600
## 10  2013     1     1 2013-01-01 06:00:00     138      558            600
## # i 336,766 more rows
## # i 12 more variables: dep_delay <dbl>, arr_time <int>, sched_arr_time <int>,
## #   arr_delay <dbl>, carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
## #   dest <chr>, distance <dbl>, hour <dbl>, minute <dbl>
```

9.4 Pipes

Pipes are most useful when you want to combine multiple operations. For example, you wanted to find the fastest flights to Houston's IAH airport while also printing out relevant information:

```
Data |>
  filter(dest == "IAH") |>
  mutate(speed = distance / air_time * 60) |>
  select(year:day, dep_time, carrier, flight, speed) |>
  arrange(desc(speed))
```

```
## # A tibble: 7,198 x 7
##     year month   day dep_time carrier flight speed
##    <int> <int> <int>    <int> <chr>    <int> <dbl>
##  1  2013     7     9      707 UA         226  522.
##  2  2013     8    27     1850 UA        1128  521.
##  3  2013     8    28      902 UA        1711  519.
##  4  2013     8    28     2122 UA        1022  519.
##  5  2013     6    11     1628 UA        1178  515.
##  6  2013     8    27     1017 UA         333  515.
##  7  2013     8    27     1205 UA        1421  515.
##  8  2013     8    27     1758 UA         302  515.
##  9  2013     9    27      521 UA         252  515.
## 10  2013     8    28      625 UA         559  515.
## # i 7,188 more rows
```

Without pipes:

```
flights1 <- filter(Data, dest == "IAH")
flights2 <- mutate(flights1, speed = distance / air_time * 60)
```

```
flights3 <- select(flights2, year:day, dep_time, carrier, flight, speed)
arrange(flights3, desc(speed))
```

```
## # A tibble: 7,198 x 7
##     year month   day dep_time carrier flight speed
##    <int> <int> <int>    <int> <chr>    <int> <dbl>
##  1  2013     7     9      707 UA         226  522.
##  2  2013     8    27     1850 UA        1128  521.
##  3  2013     8    28      902 UA        1711  519.
##  4  2013     8    28     2122 UA        1022  519.
##  5  2013     6    11     1628 UA        1178  515.
##  6  2013     8    27     1017 UA         333  515.
##  7  2013     8    27     1205 UA        1421  515.
##  8  2013     8    27     1758 UA         302  515.
##  9  2013     9    27      521 UA         252  515.
## 10  2013     8    28      625 UA         559  515.
## # i 7,188 more rows
```

Notice multiple objects are created in the intermediate steps.

9.5 Grouping

You can group your data frame to set it up for easier analysis. We often want to summarize data across groups. For example, to find the median arrival delay by carrier:

```
Data |>
  group_by(carrier) |>
  summarize(med_arr_delay = median(arr_delay, na.rm = TRUE))
```

```
## # A tibble: 16 x 2
##    carrier med_arr_delay
##    <chr>           <dbl>
##  1 9E                 -7
##  2 AA                 -9
##  3 AS                -17
##  4 B6                 -3
##  5 DL                 -8
##  6 EV                 -1
##  7 F9                  6
##  8 FL                  5
##  9 HA                -13
## 10 MQ                 -1
## 11 OO                 -7
## 12 UA                 -6
## 13 US                 -6
```

```
## 14 VX                -9
## 15 WN                -3
## 16 YV                -2
```

You can also use `n()` to find the number of observations in each group

```
Data |>
  group_by(carrier) |>
  summarize(med_arr_delay = median(arr_delay, na.rm = TRUE),
    n = n()
  )
```

```
## # A tibble: 16 x 3
##    carrier med_arr_delay     n
##    <chr>           <dbl> <int>
##  1 9E                 -7 18460
##  2 AA                 -9 32729
##  3 AS                -17   714
##  4 B6                 -3 54635
##  5 DL                 -8 48110
##  6 EV                 -1 54173
##  7 F9                  6   685
##  8 FL                  5  3260
##  9 HA                -13   342
## 10 MQ                 -1 26397
## 11 OO                 -7    32
## 12 UA                 -6 58665
## 13 US                 -6 20536
## 14 VX                 -9  5162
## 15 WN                 -3 12275
## 16 YV                 -2   601
```

You can also summarize by more than 1 group. For example, to find the number of flights for each combination of departure airports and arrival airports:

```
Data |>
  group_by(origin, dest) |>
  summarize(n = n()
  )
```

```
## `summarise()` has grouped output by 'origin'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 224 x 3
## # Groups:   origin [3]
##    origin dest      n
##    <chr>  <chr> <int>
##  1 EWR    ALB     439
##  2 EWR    ANC       8
```

```
##  3 EWR    ATL    5022
##  4 EWR    AUS     968
##  5 EWR    AVL     265
##  6 EWR    BDL     443
##  7 EWR    BNA    2336
##  8 EWR    BOS    5327
##  9 EWR    BQN     297
## 10 EWR    BTV     931
## # i 214 more rows
```

Note: if you realize that the functions in Section 9.5 are similar to `tapply()` and `table()`, you are correct!

9.5.1 Select Rows within Group

There are five handy functions that allow you to extract specific rows within each group:

- `df |> slice_head(n = 1)` takes the first row from each group.
- `df |> slice_tail(n = 1)` takes the last row in each group.
- `df |> slice_min(x, n = 1)` takes the row with the smallest value of column `x`.
- `df |> slice_max(x, n = 1)` takes the row with the largest value of column `x`.
- `df |> slice_sample(n = 1)` takes one random row.

For example, the following code finds the flights that are most delayed upon arrival at each destination:

```
Data |>
  group_by(dest) |>
  slice_max(arr_delay, n = 1) |>
  relocate(dest)
```

```
## # A tibble: 108 x 19
## # Groups:   dest [105]
##    dest   year month   day dep_time sched_dep_time dep_delay arr_time
##    <chr> <int> <int> <int>    <int>          <int>     <dbl>    <int>
##  1 ABQ    2013     7    22     2145           2007        98      132
##  2 ACK    2013     7    23     1139            800       219     1250
##  3 ALB    2013     1    25      123           2000       323      229
##  4 ANC    2013     8    17     1740           1625        75     2042
##  5 ATL    2013     7    22     2257            759       898      121
##  6 AUS    2013     7    10     2056           1505       351     2347
##  7 AVL    2013     8    13     1156            832       204     1417
##  8 BDL    2013     2    21     1728           1316       252     1839
##  9 BGR    2013    12     1     1504           1056       248     1628
## 10 BHM    2013     4    10       25           1900       325      136
```

```
## # i 98 more rows
## # i 11 more variables: sched_arr_time <int>, arr_delay <dbl>, carrier <chr>,
## #   flight <int>, tailnum <chr>, origin <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dttm>
```

Chapter 10

Exploratory Data Analysis and Tidying Data

10.1 Introduction

This section covers two slightly different topics, but they will be helpful in your project.

1. Exploratory data analysis (EDA) is used by statisticians and data scientists to investigate data sets and summarize their main characteristics, often using data visualizations. A lot of the tools you saw in Sections 8 and 9 will be useful. You will learn more about the way to use these skills to perform EDA. This subsection is heavily based on R for Data Science (Wickham et al., 2023), Chapter 10.

2. Data tidying is an approach to make the raw data that we get into a "nice" format which makes performing EDA and modeling much easier. This subsection is heavily based on R for Data Science (Wickham et al., 2023), Chapter 5.

10.2 EDA

EDA can be viewed as a (fairly) systematic and iterative process, but it should not be viewed as a formal process with a strict set of rules. These steps are typically taken:

1. Generate questions that you want to answer with the data.
2. Generate data visualizations and / or numerical summaries to address these questions.
3. Use what you learned to refine the questions, or generate new questions.

At the beginning, you should feel free to explore your ideas. Not all the ideas will pan out; but you will eventually hone in on some interesting insights.

EDA can also be used to check the quality of your data.

Generally speaking, the main questions that are asked during EDA are

1. What type of variation occurs within the variables?
2. What kind of relationship exists between the variables?

Domain-specific knowledge will be helpful in this process. Statisticians and data scientists often learn about the domain associated with the data set. You should never work on data where you have little domain-specific knowledge!

Note that modeling is rarely done during EDA. We are just creating visualizations and computing numerical summaries, and these should be done prior to modeling.

10.2.1 Variation Within Variables

Variance measures the spread associated with a quantitative variable. If a particular variable is constant, there is no variance, as it always takes on the same value. When we have a quantitative variable, we can use tools from Section 8.6.2, such as density plots, boxplots, and histograms.

We take a look at the `diamonds` data set from the `tidyverse` package. It contains the prices and other attributes of almost 54,000 diamonds. We wish to explore the weight of the diamonds, carat, as it is a characteristic that buyers are most familiar with.

```
library(tidyverse)
Data <- ggplot2::diamonds
```

```
ggplot(Data, aes(x = carat)) +
  geom_histogram(binwidth = 0.5) + ##use binwidth that makes sense
  labs(
    x = "Carat",
    y = "Count",
    title = "Histogram of Diamond Carats"
  )
```

Typical questions are (and the answers based on Figure 10.1):

1. What are the more common values? The bars appear highest around 0.5 to 1.5 carats, so these carats are common. Carats are concentrated among the smaller carats.

2. What values are rare? High carat diamonds are rare. Heavier diamonds are rare and therefore fetch a premium. This is to be expected.

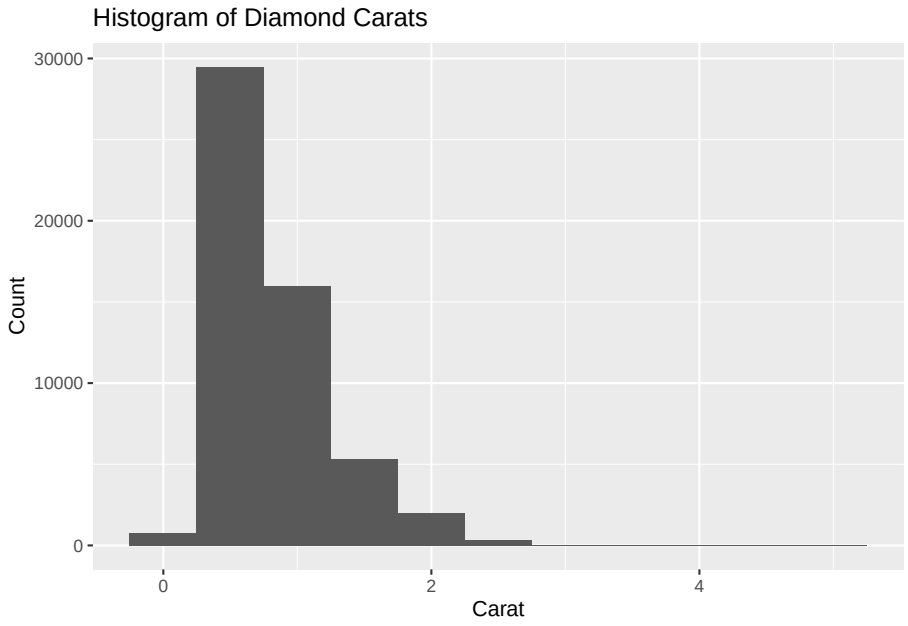


Figure 10.1: Diamond Carats are Right Skewed

3. Are there unusual patterns? Not really, as we expect more diamonds to be on the smaller end of the scale. The bar for the smallest carats being so low may be surprising.

Given that most diamonds are less than 3 carats, we can create a similar histogram, but only for diamonds less than 3 carats. We can also refine the width of the bins to examine the distribution of the carats more closely. You must make it clear that you are only examining a certain portion of the data.

```
smaller <- Data |>
  filter(carat < 3)

ggplot(smaller, aes(x = carat)) +
  geom_histogram(binwidth = 0.01) +
  labs(
    x = "Carat",
    y = "Count",
    title = "Histogram of Diamond Carats (less than 3 carats)"
  )
```

Notice that there are more diamonds with full or half carat sizes. According to Hamilton Jewelers, “this is done to create more precise measurements and because the price of a diamond will jump at each half and full carat”. So this

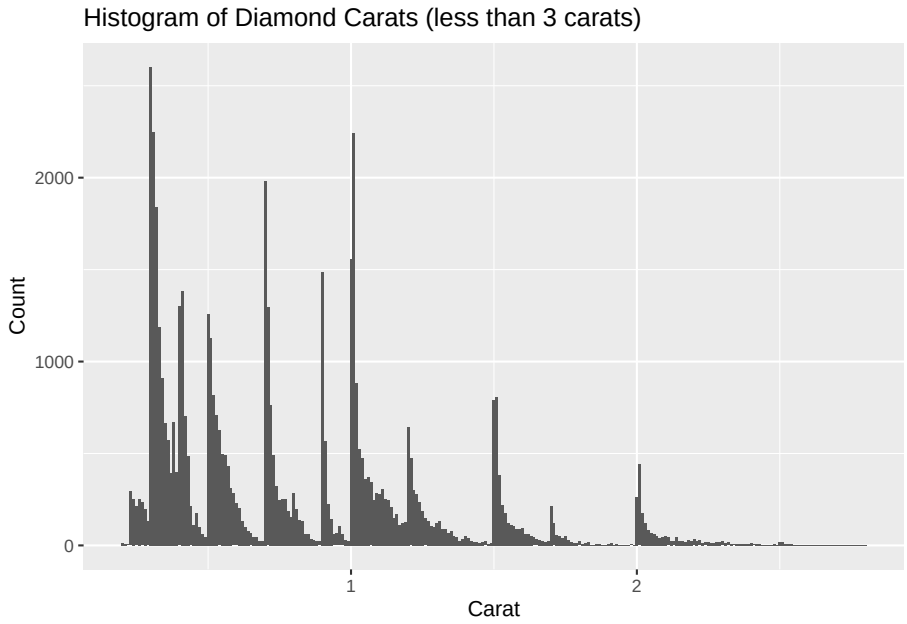


Figure 10.2: Histogram with Big Diamonds Removed

observation is as expected.

10.2.1.1 Unusual values

We can use visualizations created during EDA to help spot unusual values. Sometimes these are data entry errors, sometimes they are correct values that can happen, and sometimes there is something interesting about these values that warrant further investigation.

We take a look at the `y` variable in the diamonds data set, which measures the width of the diamonds in mm.

```
ggplot(Data, aes(x = y)) +
  geom_histogram(binwidth = 0.5) +
  labs(
    x = "Width in mm",
    y = "Count",
    title = "Histogram of Diamond Widths"
  )
```

From Figure 10.3, most diamonds have widths between 5 to 10mm. The fact that the histogram has an x-axis ranging from 0 to 60mm informs us that there exist diamonds with widths in the extremes. You can "zoom-in" on the plot by limiting the limit on the y-axis by supplying the range of the y-axis via `ylim`

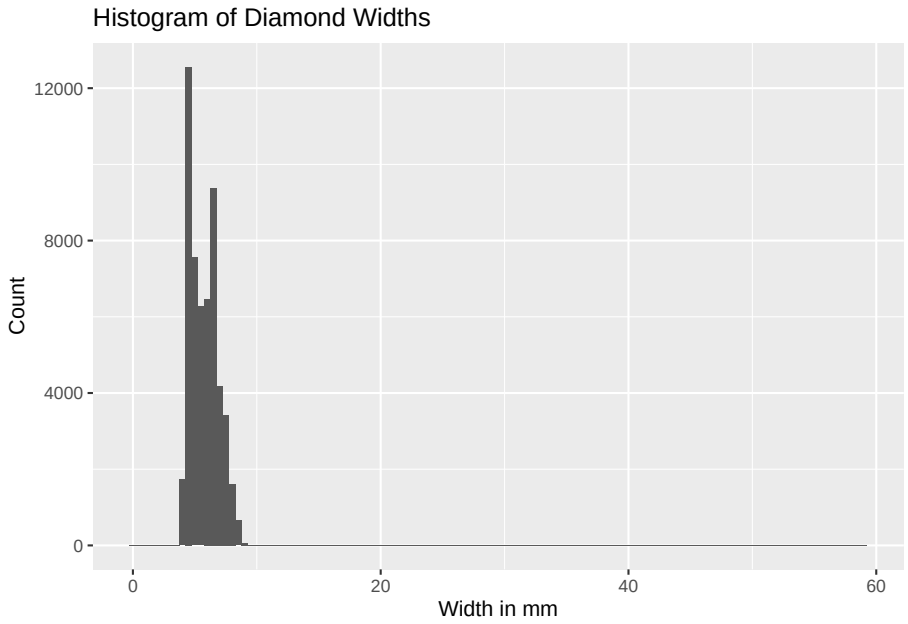


Figure 10.3: Width of Diamonds in mm

inside the layer `coord_cartesian`. You can also limit the range of the x-axis if needed.

```
ggplot(diamonds, aes(x = y)) +
  geom_histogram(binwidth = 0.5) +
  coord_cartesian(ylim = c(0, 50))
```

From Figure 10.4, we see around 7 diamonds with width of 0mm, and 1 around 30mm and 1 around 59mm. The latter two diamonds could be legitimate, but there cannot be diamonds with width 0mm. Are there really diamonds with width recorded 0? To confirm, we can arrange the data frame by width,

```
Data |>
  arrange(y) |>
  relocate(y)
```

```
## # A tibble: 53,940 x 10
##        y carat cut       color clarity depth table price     x     z
##    <dbl> <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl>
##  1  0     1    Very Good H     VS2      63.3    53  5139  0     0
##  2  0     1.14 Fair      G     VS1      57.5    67  6381  0     0
##  3  0     1.56 Ideal     G     VS2      62.2    54 12800  0     0
##  4  0     1.2  Premium   D     VVS1     62.1    59 15686  0     0
```

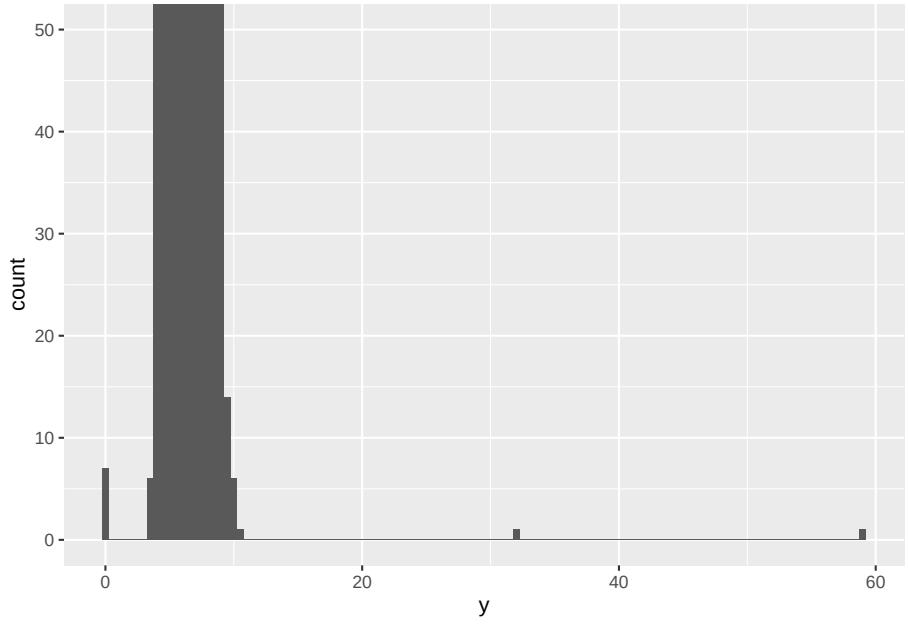


Figure 10.4: Zooming in by Changing Limits of Y Axis

```
##  5  0     2.25 Premium   H     SI2      62.8    59 18034  0     0
##  6  0     0.71 Good      F     SI2      64.1    60  2130  0     0
##  7  0     0.71 Good      F     SI2      64.1    60  2130  0     0
##  8  3.68  0.2  Premium   D     VS2      62.3    60   367  3.73  2.31
##  9  3.71  0.2  Premium   F     VS2      62.6    59   367  3.73  2.33
## 10  3.71  0.2  Very Good E     VS2      63.4    59   367  3.74  2.36
## # i 53,930 more rows
```

or use the `summary()` function to check the minimum value of the widths.

```
summary(Data$y)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.000   4.720   5.710   5.735   6.540  58.900
```

Even though we are not diamond experts, common sense tells us that no diamond can have a width of 0mm. Given that the values were entered using 2 decimal places, these are also unlikely to be values that were rounded down to 0mm. I suspect the 0s were entered to represent missing values, and so we will probably want to replace these 0s with `NA`. Notice their length (x) and depth (z) are also 0. You should always confirm this with the entity that collected these data.

How about the 2 wide diamonds? If they are truly that huge, we'd expect them

to be expensive. We can sort the data set by width (descending order) and also look at the prices of these diamonds:

```
Data |>
  arrange(desc(y)) |>
  relocate(x, y, z, price)
```

```
## # A tibble: 53,940 x 10
##        x     y     z price carat cut       color clarity depth table
##    <dbl> <dbl> <dbl> <int> <dbl> <ord>     <ord> <ord>   <dbl> <dbl>
##  1  8.09 58.9   8.06 12210  2    Premium   H     SI2      58.9    57
##  2  5.15 31.8   5.12  2075  0.51 Ideal     E     VS1      61.8    55
##  3 10.7  10.5   6.98 18018  5.01 Fair      J     I1       65.5    59
##  4 10.2  10.2   6.72 18531  4.5  Fair      J     I1       65.8    58
##  5 10.1  10.1   6.17 15223  4.01 Premium   I     I1       61      61
##  6 10.0   9.94  6.24 15223  4.01 Premium   J     I1       62.5    62
##  7 10.0   9.94  6.31 15984  4    Very Good I     I1       63.3    58
##  8 10     9.85  6.43 17329  4.13 Fair      H     I1       64.8    61
##  9  9.86  9.81  6.13 16193  3.67 Premium   I     I1       62.4    56
## 10  9.66  9.63  6.03 18701  3.51 Premium   J     VS2      62.5    59
## # i 53,930 more rows
```

We expect that prices are driven by the weight (carat), which in turn is driven by the dimensions of the diamonds. We can take a look at the summary of prices:

```
summary(Data$price)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     326     950    2401    3933    5324   18823
```

The price of the diamond with width 31.8mm is below the median. The price of the other diamond is above the 3rd quartile. The former is more suspicious, but the latter may be more legitimate. If possible, consult the entity that collected the data or consult a subject-matter expert to ask if such values could be legitimate.

My recommendation will be to perform subsequent analysis with and without these two diamonds, and see how different the analysis is. If the results are similar, omit them but mention that they were omitted. If the results are different, you really need to find out what makes these values so different.

As mentioned earlier, I would probably replace the 0s with `NA`. Do not remove the entire row, as the row contains information on other columns.

10.2.2 Relationship between Variables

10.2.2.1 One quantitative and one categorical variable

We can use the visualizations from 8.4 to explore the relationship between a quantitative variable and a categorical variable. To explore the relationship between the price and cut of diamonds, we can create a density plot of price with separate colors for each cut:

```
ggplot(Data, aes(x = price, color = cut)) +
  geom_density() +
  labs(
    x = "Price",
    y = "Density",
    color = "Cut",
    title = "Density Plot of Prices across Cut"
  )
```



Figure 10.5: Prices Across Cuts are Right Skewed

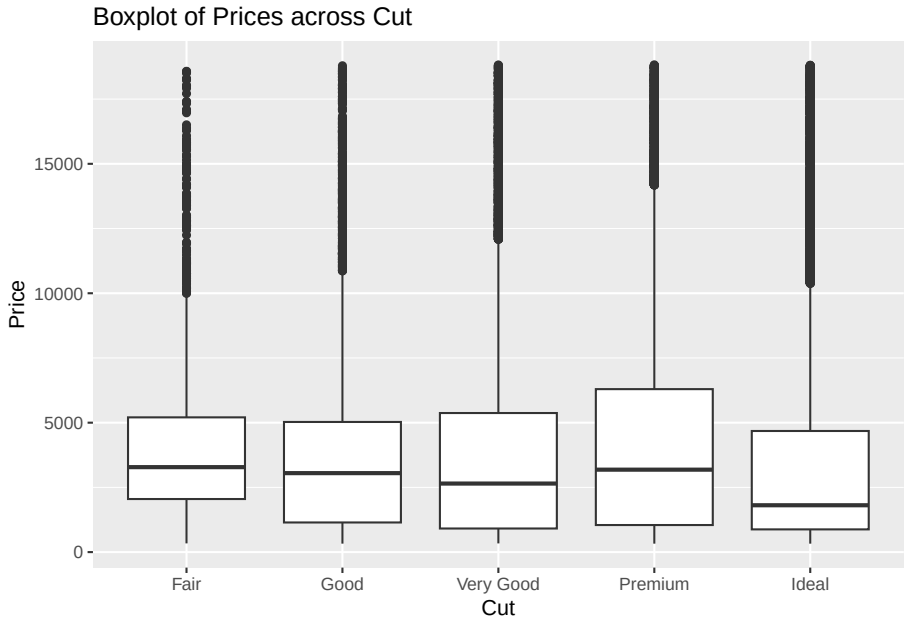
Notice in this example the categorical variable cut is ordinal, there is an order to the values. Cut reflects a measure of quality, in increasing order from fair, good, very good, premium, ideal. You have to make sure that they are ordered correctly (which they are in this data set). If they are not, please see Section 8.6.1 to see how to use the `factor()` function to order the variable.

If prices are similar across each cut, these density curves should be similar to

each other for all cuts. However, we see in Figure 10.5 that the curve for fair cuts appear to be quite different: its prices tend to be a little higher, which may be counter intuitive as fair is the lowest quality.

Alternatively, we can create side by side box plots

```
ggplot(Data, aes(x = cut, y = price)) +
  geom_boxplot() +
  labs(
    x = "Cut",
    y = "Price",
    title = "Boxplot of Prices across Cut"
  )
```



We see a similar conclusion in Figure **??** that diamonds with fair cut may have slightly higher prices.

Notice that the prices across cuts are right skewed? Most prices are on the lower end, and a few prices are really high. Sometimes, log transforming a right skewed variable helps the visualization.

```
Data <- Data |>
  mutate(
    logPrice = log(price)
  )
```

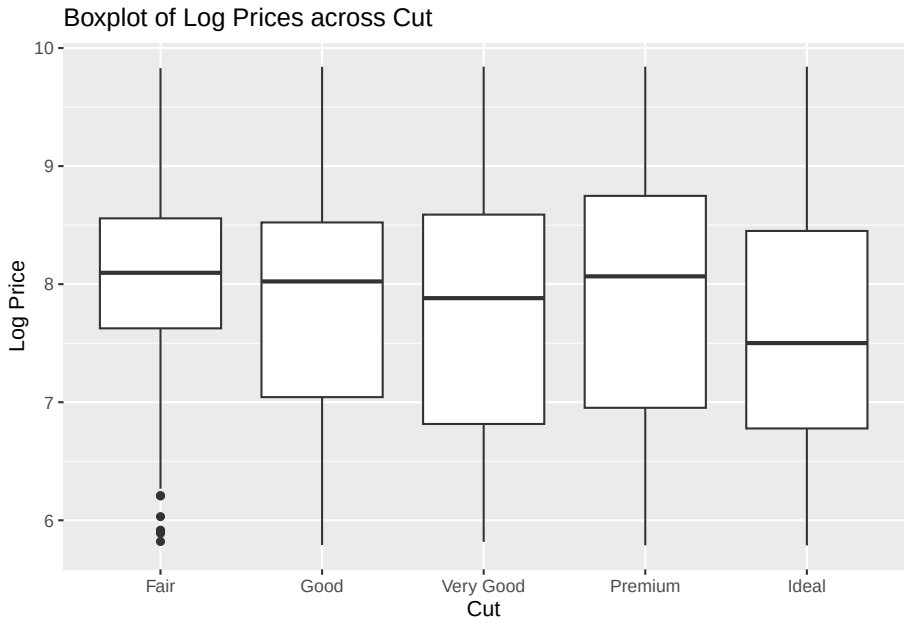
```
ggplot(Data, aes(x = logPrice, color = cut)) +
  geom_density() +
```

```
  labs(
    x = "Log Price",
    y = "Density",
    color = "Cut",
    title = "Density Plot of Log Prices across Cut"
  )
```



Figure 10.6: Density Plot with Log Transformed Prices Across Cut

```
ggplot(Data, aes(x = cut, y = logPrice)) +
  geom_boxplot() +
  labs(
    x = "Cut",
    y = "Log Price",
    title = "Boxplot of Log Prices across Cut"
  )
```

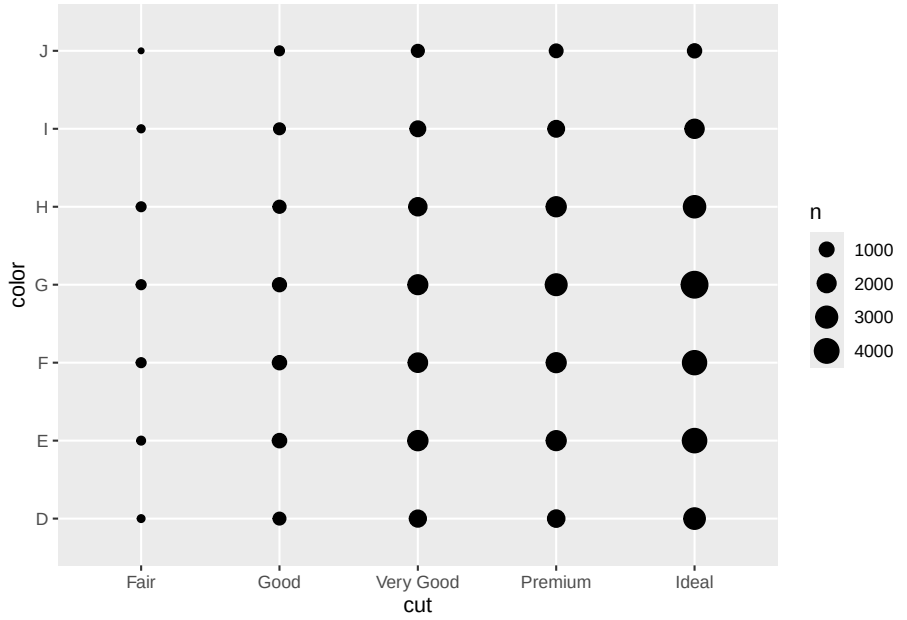


It appears from Figures 10.6 and **??** that cut has the opposite relationship with price than expected. What are some plausible reasons for this observation?

10.2.2.2 Two categorical variables

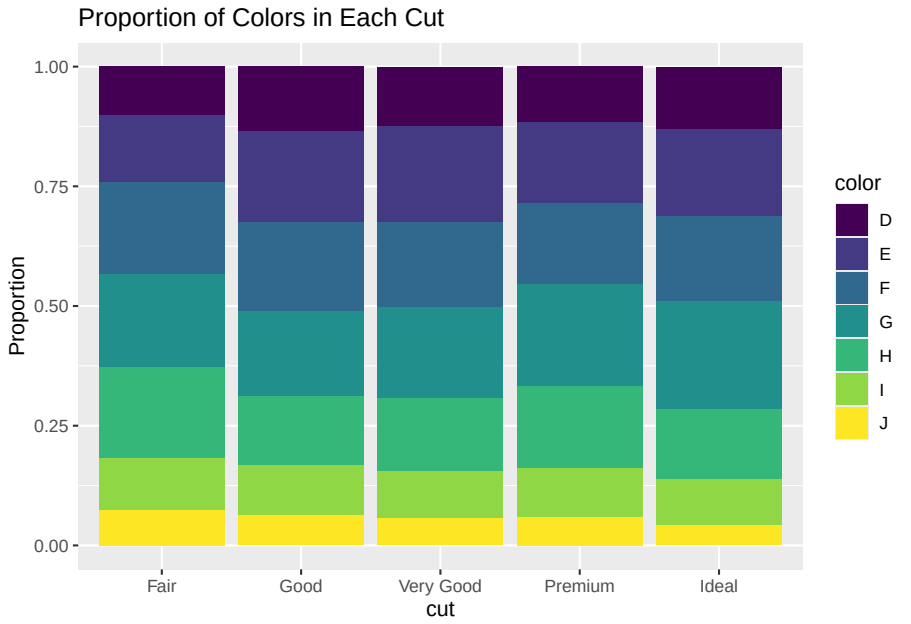
We can use the tools from 8.3 to explore the relationship between two categorical variables. For example, we want to explore if cut and color of diamonds are related. We can create a visual that summarizes the number of diamonds for each combination of cut and color:

```
ggplot(Data, aes(x = cut, y = color)) +
  geom_count()
```



Large circles indicate the specific combination of cut and color that are more prevalent and more likely to occur together. We see in Figure **??** that ideal cut with color G has the most occurrence, but it may be better to look at proportions instead of counts. We can use a bar chart:

```
ggplot(Data, aes(x = cut, fill = color)) +
  geom_bar(position = "fill") +
  labs(title = "Proportion of Colors in Each Cut", y = "Proportion")
```



In Figure **??**, the proportions of colors in each cut, while not identical, are pretty similar. So there is little evidence that color and cut are associated. We can create a two-way table:

```
table(Data$cut, Data$color)
```

```
##
##                D    E    F    G    H    I    J
##   Fair       163  224  312  314  303  175  119
##   Good       662  933  909  871  702  522  307
##   Very Good 1513 2400 2164 2299 1824 1204  678
##   Premium   1603 2337 2331 2924 2360 1428  808
##   Ideal     2834 3903 3826 4884 3115 2093  896
```

Or a two-way table using proportions, due to the different number of diamonds in each cut:

```
round(prop.table(table(Data$cut, Data$color), margin = 1), 2)
```

```
##
##                D    E    F    G    H    I    J
##   Fair      0.10 0.14 0.19 0.20 0.19 0.11 0.07
##   Good      0.13 0.19 0.19 0.18 0.14 0.11 0.06
##   Very Good 0.13 0.20 0.18 0.19 0.15 0.10 0.06
##   Premium   0.12 0.17 0.17 0.21 0.17 0.10 0.06
##   Ideal     0.13 0.18 0.18 0.23 0.14 0.10 0.04
```

For each cut, the proportion of each color is similar, so there is little evidence

of an association between cut and color.

10.2.2.3 Two quantitative variables

Scatterplots are often used to explore the relationship between two quantitative variables. To explore how price is related to carat:

```
ggplot(Data, aes(x = carat, y = price)) +
  geom_point() +
  labs(title = "Price Vs Carat of Diamonds",
    x = "Carat", y = "Price")
```

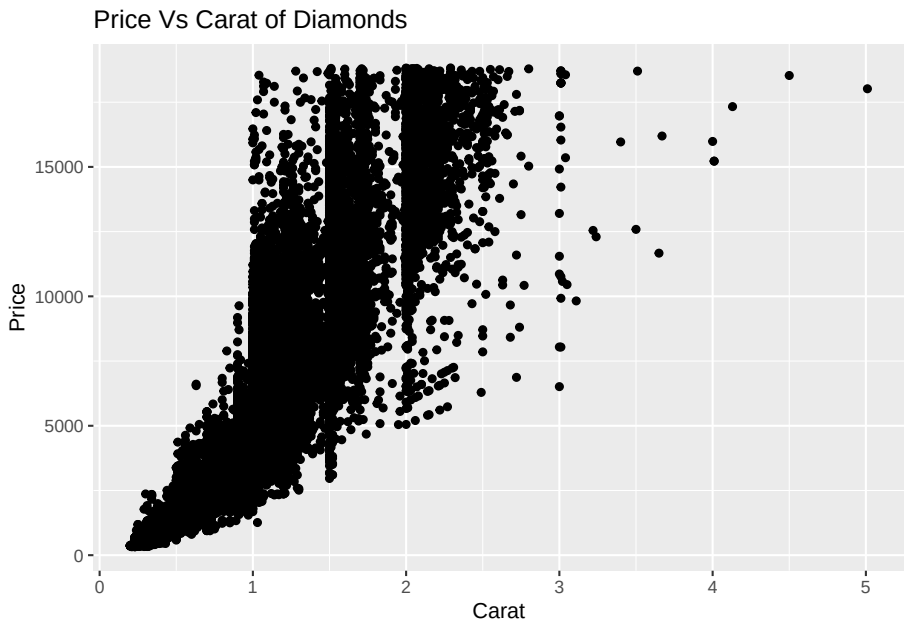


Figure 10.7: Scatterplot of Price Against Carat

Typical questions for scatterplots include (and answers based on Figure 10.7):

1. Is the relationship linear or not? In this case, the relationship in non linear.

2. Does the response variable generally increases as the predictor increases? In this example, generally yes, as the carat increases, price tends to increase.

3. How strong in the relationship? Imagine overlaying a line (if linear) or curve over the plots. How close do the plots fall to the line or curve? The closer they lie, the stronger the relationship. For smaller diamonds

the prices are clustered together, but less so for larger diamonds. So the relationship is stronger for smaller diamonds.

An idea with visualizing quantitative variables is to bin them, i.e. create a categorical variable base on range of values on a quantitative variable, and then use visualizations involving one quantitative variable and one categorical variable.

```
ggplot(Data, aes(x = carat, y = price)) +
  geom_boxplot(aes(group = cut_width(carat, 0.1)))
```

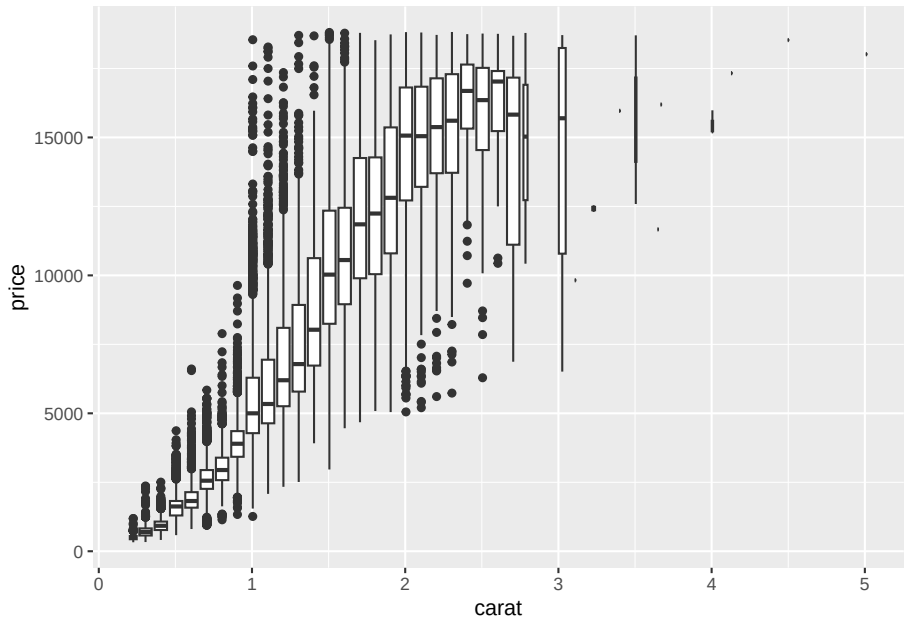


Figure 10.8: Boxplots of Price Against Binned Carat

`cut_width(carat, 0.1)` creates boxplots for each group of carats into bins with width 0.1. The interpretations are pretty similar to the scatterplot.

Another tool to assess the strength of the linear relationship is the correlation, via `cor()`. Note that correlation should only be used if the relationship is linear, and not when the relationship is nonlinear.

10.2.3 A Few Things to Consider

If you notice some patterns or relationships in the data, you can ask:

1. Is this pattern coincidence?
2. How can you describe the relationship?
3. How strong is the relationship?

4. What other variables might affect the relationship?
5. Could the relationship change if you look at subgroups of the data?

10.2.4 Tips for Effective Visualizations

- Provide a title.
- Label axes, avoid using the name of the column. Readers may not have looked at your data frame.
- Right visualization for variable type (categorical, quantitative). Sometimes, an error message is shown and one reason could be the visualization does not work for the type of variable.
- Scaling of axes. Pay attention to the range of the axes and they do not mislead.
- Clear and simple. Avoid unnecessary styling or decorations. The visual needs to convey your key message. Do not cram too many variables. In my opinion, going beyond 3 or 4 becomes questionable. Using `facet_wrap` can be helpful.
- Know your audience. Simplify for non-experts.

10.3 Tidy Data

You may have notice that in Sections 8 and 9, data frames were called tibbles. A tibble is the `tidyverse` version of a data frame. Data sets from the `tidyverse` are tibbles. The main difference is that when typing out the name of a tibble, only the first few rows and columns are printed, whereas all rows of a data frame get printed. Therefore, tibbles are nicer when working with large data sets.

Tidy data have the following characteristics:

1. Each variable is a column; each column is a variable.
2. Each observation is a row; each row is an observation.
3. Each value is a cell; each cell is a single value.

The packages in the `tidyverse` work best with tidy data (as do most other packages). So why may data not be tidy?

1. The data are organized for another purpose, usually for ease of data entry rather than data analysis.

2. Most people are not familiar with the principles of tidy data, and it is difficult to derive them yourself unless you spend a lot of time working with data.

Please read R for Data Science (Wickham et al., 2023), Chapter 5. There are two main classes of non tidy data, and so we lengthen or widen the data. Some of you may have to do these in your project, while most will not.

Chapter 11

Sampling Distributions and the Central Limit Theorem

11.1 Key Ideas and Terms

Consider this simple scenario. We want to find the distribution associated with the systolic blood pressure of American adults. To be able to achieve this goal, we would have to get the systolic blood pressure of every single American adult. This is usually not feasible as researchers are unlikely to have the time and money to interview every single American adult. Instead, a representative sample of American adults will be obtained, for example, 750 randomly selected American adults are interviewed. We can then create density plots, histograms, compute the mean, median, variance, skewness, and other summaries that may be of interest, based on these 750 American adults.

11.1.1 Population Vs. Sample

The above scenario illustrates a few concepts and terms that are fundamental in statistical inference. In any study, we must be clear as to who or what is the population of interest, and who or what is the sample.

The **population** (sometimes called the population of interest) is the entire set of individuals, or objects, or events that a study is interested in. In the scenario described above, the population would be (all) American adults.

The **sample** is the set of individuals, or objects, or events which we have data on. In the scenario described above, the sample is the 750 randomly selected American adults.

In short, we are analyzing data from a sample and then try to make an inference about the population.

Ideally, the sample should be **representative** of the population. A representative sample is often achieved through a simple random sample, where each unit in the population has the same chance of being selected to be in the sample. In this section, we will assume that we have a representative sample. Note: You may feel that obtaining a simple random sample may be difficult. We will not get into a discussion of sampling (sometimes called survey sampling), which is a field of statistics that handles how to obtain representative samples, or how calculations should be adjusted if the sample is not representative. There is still a lot of research that is being done in survey sampling.

11.1.2 Parameter Vs Estimator

Now that we have made the distinction between a population and a sample, we are ready to define parameters and estimators.

A **parameter** is a numerical summary associated with a population. In the scenario described above, an example of a population parameter would be the population mean systolic blood pressure of American adults.

An **estimator** or a **statistic** is a numerical summary associated with samples. An estimator is typically used to estimate a parameter. In the scenario described above, an estimator of the population mean systolic blood pressure of American adults could be the average systolic blood pressure in our sample. So the sample mean is an estimator of the population mean.

An **estimated value**, or **estimate**, is the actual value of the estimator based on a sample. In the scenario described above, suppose the average systolic blood pressure of the 750 American adults is 140 mm Hg. We will say the estimated value of the mean systolic blood pressure of American adults is 140 mm Hg.

So a parameter is associated with a population, while an estimator is associated with a sample. Some other differences between parameters and estimators:

- The value of parameters are unknown, while we can actually calculate numerical values of estimators.
- The value of parameters are considered fixed (as there is only one population), while the numerical values of estimators can vary if we obtain multiple random samples of the same sample size. Using the scenario above again, suppose we obtain a second representative sample of 750 American adults. The average systolic blood pressure of this second sample is likely to be different from the average systolic blood pressure of the first sample. This illustrates that there is **variance associated with estimators due to random sampling**.

11.1.3 Sampling Distributions

Now imagine that we keep obtaining representative samples of 750 American adults. We calculate the average systolic blood pressure from each sample.

They are all likely to be different. We can then create a histogram or density plot (since the averages are numeric) to visualize the distribution of the sample means. Are they all close to each other? Or they spread out? Is the distribution bell-shaped or skewed?

The distribution associated with a statistic (from the same sample size) is called its **sampling distribution**. A sampling distribution is the probability distribution of a given statistic (like a mean or proportion) calculated from multiple random samples of the same size taken from a larger population. Some statistics have known sampling distributions (under certain conditions), for example:

- The sample mean, $\bar{X}$.
- The sample proportion , $\hat{p}$.

The sampling distributions of the statistics $\bar{X}$ and $\hat{p}$ have been proven mathematically. However, what happens when the "certain conditions" are not met, or if the statistic has no known sampling distribution? We can use simulations in these instances! We have seen an example in Section 6.4.1, where we used a simulation to derive the sampling distribution of the sample mean.

It turns out we can use simulations to also estimate expected values and probabilities. When using simulations in these settings, the simulations are called **Monte Carlo** simulations. Monte Carlo simulations work because of the Law of Large Numbers, which we briefly describe next.

11.1.4 Law of Large Numbers

The **Law of Large Numbers** (LLN) states that as n gets larger and approaches infinity, the sample mean $\bar{X}_n = \frac{X_1+\cdots+X_n}{n}$ converges to the true mean (or expected value) μ. This implies that the sample mean tends to get closer to the population mean with larger sample sizes. The key word here is tends to, it is not a guarantee that the sample mean always gets closer to the population mean whenever n gets larger, but it generally does. This explains why we tend to trust results from larger sample sizes.

We use an example to illustrate the LLN, which comes from flipping a fair coin. Let X denote whether the coin lands heads or tails, and let $X = 1$ for heads and $X = 0$ for tails. We can say that $X \sim Bern(0.5)$ since the coin is fair. Imagine flipping the coin n times, and record the outcome after each flip, so $X_1, \cdots, X_n$ denote the outcome of each flip. We know that $E(X) = 0.5$ since $X \sim Bern(0.5)$. The LLN informs us that $\bar{X}_1, \cdots, \bar{X}_n$ should usually get closer to 0.5 as n increases. In other words, the value of the sample proportion after each flip should usually get closer to 0.5 with more flips. The code below simulates this example for $n = 500$, and Figure 11.1 shows how the sample proportions get closer to 0.5, in general, as n increases.

```
n<-500 ##make this big, but not too big otherwise picture is difficult to see

set.seed(23)
```

```
X<-rbinom(n,1,0.5) ##simulate 500 flips of fair coin

totals<-cumsum(X) ##count total number of heads after each flip
index<-1:n
props<-totals/index ##find proportion of heads after each flip

##create visual. LLN says that as n gets larger, the value of the sample proportion te
plot(props, type="l", main="Prop vs Sample Size", ylab="Proportion", xlab="n")
abline(h=0.5, col="blue") ##overlay 0.5 for easy comparison
```

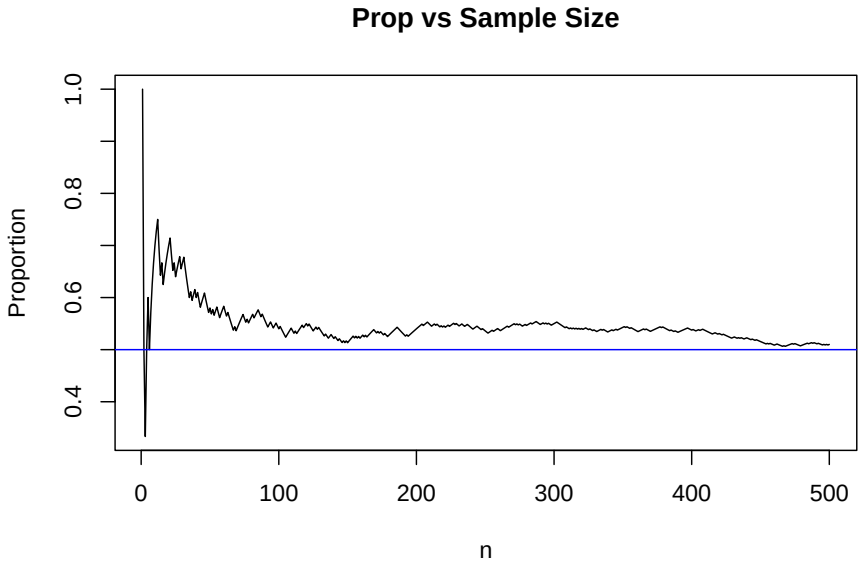


Figure 11.1: LLN Example with Coin Flips

Note: The LLN actually comes in two versions, the Weak Law of Large Numbers (WLLN), and the Strong Law of Large Numbers (SLLN). What I have written gives an intuitive explanation of what the LLN implies.

11.1.4.1 Misconceptions with LLN

One key idea with the LLN is that the sample mean **tends to get closer to the true mean as n gets larger**. The key words here are "tends" and "as n gets larger".

A misunderstanding of the LLN is the **gambler's fallacy**, which erroneously believes that the sample mean must "self correct" and get closer to the population mean with small increments of n.

Using the example from flipping a fair coin. The gambler's fallacy erroneously thinks that:

- The proportion of heads should be close to 0.5, even with small n.
- The results of subsequent flips should self correct, i.e. the proportion of heads get closer to 0.5 with the next flip. For example, if the first 5 flips are heads, the next flip is "due" to be tails since the proportion should get closer to 0.5 with the next flip.

The convergence to 0.5 comes from flipping the coin many times.

11.2 Monte Carlo Simulations

The idea behind Monte Carlo simulations is to use repeated random sampling (and by repeated, we mean repeated a large number of times) to estimate features of data based on expected values. Monte Carlo simulations are used for the following purposes:

1. When the probability or expectation is too complicated to work out by hand. Recall that finding probabilities and expectations by hand involve summations or integrals, and it becomes obvious that working with summations and especially integrals can get onerous.

2. To verify theoretical results involving probability or expectations. While a lot of theory is proved using mathematics, most academic papers include Monte Carlo simulations to verify the theoretical results. We have done these to verify the Central Limit Theorem in Section 6.4.1.

3. To help confirm that you understand the meaning of theoretical results. The only way your code matches the theory is if you understand the theory.

The Law of Large Numbers (LLN) provides the mathematical backbone for Monte Carlo Simulations. LLN guarantees that as you repeat an experiment or random trial more times, your average result tends to get closer to the true expected value.

11.2.1 Monte Carlo Methods for Expected Values

Suppose we want to find some expectation for a continuous random variable X, $E(g(X))$, where g is some function, i.e. $E(g(X)) = \int_{-\infty}^{\infty} g(x)f_X(x)$. Monte Carlo methods avoid doing this integration by simulating $X_1, \cdots, X_M$ from X and estimate $E(g(X))$ with the sample mean of $g(X)$, $\frac{1}{M}\sum_{i=1}^{M} g(X_i)$. The LLN tells us that as M gets larger, this sample mean converges to $E(g(X))$.

Monte Carlo methods replace the integral (or summation) with simulating a random variable repeatedly many times. We use a simple example to illustrate this idea.

Let X be a standard normal distribution. Suppose we want to find the value of $E(X^2) = \int_{-\infty}^{\infty} x^2 \frac{1}{\sqrt{2\pi}} e^{-x^2/2} dx$. Instead of working out this integral by hand, we carry out a Monte Carlo simulation by doing these steps:

- Simulate M random values from a standard normal, where M is large.
- Calculate $X_1^2, \cdots, X_M^2$.
- Find the sample average of $X_1^2, \cdots, X_M^2$.

```
set.seed(5)

reps<-10000 ## this is M

Xs<-rnorm(reps) ##generate M values of X

squared.values<-Xs^2 ##square each X

mean(squared.values) ##sample average of squared values.
```

```
## [1] 1.024546
```

```
##when reps is large, this sample mean should be close to the true E(X^2), which is 1
```

Since X is standard normal, we know $E(X) = 0$ and $Var(X) = E(X^2) - E(X)^2 = E(X^2) = 1$. We see in our simulation that our estimated value for $E(X^2)$ is pretty close to its theoretical value.

11.2.2 Monte Carlo Methods for Probabilities

Suppose we want to find the probability that a random variable satisfies some event E, $P(E)$. We could perform a summation or integral to find this probability, or estimate the probability using Monte Carlo simulations. What we will do is simulate $X_1, \cdots, X_M$ from X, where M is large, in other words, simulate a large number of replicates of X. We then count how many of the X_is correspond to event E happening, and then divide this number by M, the number of replicates. This proportion can also be viewed as the average number of times event E happens, if we encode the event E happening as 1 and the event E does not happen as 0. We use an example to illustrate this idea.

Let X be a standard normal distribution. Suppose we want to find the probability $P(X^2 > 1)$. We carry out a Monte Carlo simulation by doing these steps:

- Simulate M random values from a standard normal, where M is large.
- Calculate $X_1^2, \cdots, X_M^2$.
- Count the number of times X_i^2 is greater than 1.
- Divide this number by M to estimate the probability, since probability can be interpreted as a long-run proportion.

```r
set.seed(5)

reps<-10000 ## this is M

Xs<-rnorm(reps) ##generate M values of X

squared.values<-Xs^2 ##square each X

sum(squared.values>1)/reps ##count the number of times X^2 is greater than 1, and divide by M
```

```
## [1] 0.3178
```

```r
##alternatively, same result as previous line
mean(squared.values>1)
```

```
## [1] 0.3178
```

```r
##when reps is large, this proportion should be close to
1-pchisq(1, df=1)
```

```
## [1] 0.3173105
```

```r
##it turns out that squaring a standard normal gives a chi-squared distribution with 1 df.
```

We see the estimated probability $P(X^2 > 1)$ is close to its theoretical probability.

Why do Monte Carlo simulations work with probabilities? The probability from the Monte carlo simulation finds the proportion of times a certain condition is met, which is the same as finding the average of the 1s (when condition is met) and 0s (when condition is not met), so the proportion is equivalent to a mean. So our results are backed by the LLN.

11.3 Sampling Distribution of the Sample Mean

We will state the sampling distribution of the sample mean, $\bar{X}_n$. Let $X_1, \cdots, X_n$ be i.i.d. from a normal distribution with finite mean μ and finite variance σ^2. Then $\bar{X}_n = \frac{X_1+\cdots+X_n}{n} \sim N(\mu, \frac{\sigma^2}{n})$.

This implies that if the data are originally normally distributed, the sample mean will also be normally distributed, regardless of the sample size.

For example, if the data are i.i.d. standard normal, the sample means follow a normal distribution:

- When $n = 10$, $\bar{X}_n \sim N(0, \frac{1}{10}) = N(0, 0.1)$.
- When $n = 100$, $\bar{X}_n \sim N(0, \frac{1}{100}) = N(0, 0.01)$.
- When $n = 1000$, $\bar{X}_n \sim N(0, \frac{1}{1000}) = N(0, 0.001)$.

We see that when the sample size gets larger, the variance of the sample mean

decreases, so there is less spread and their resulting density plot is more concentrated around the mean.

We can run the following Monte Carlo simulation to display the sampling distributions of the sample means when the data are i.i.d. standard normal when $n = 10, 100, 100$ in Figure 11.2 below (this simulation is similar to what was done in Section 6.4.1):

```
##function to find sample mean of data from standard normal
sampMean <- function(number){
  mean(rnorm(number))
}

Reps <- 10000

##find the sample means of n data points, with 10000 replicates
repSampMean <- function(number){
  return(replicate(Reps, sampMean(number)))
}

##find the sample means when n = 10, 100, 1000 with 10000 replicates each
results <- sapply(c(10, 100, 1000), repSampMean)

##visualize the sampling distributions of the sample means when n=10, 100, 1000

##set the max limit of the y axis
max_y1 <- max(density(results[,1])$y, density(results[,2])$y, density(results[,3])$y)

##n = 10
plot(density(results[,1]), ylim=c(0, max_y1), main="Density Plot of Sample Mean with n
##n = 100
lines(density(results[,2]), col="blue")
##n = 1000
lines(density(results[,3]), col="red")
legend("topright", legend = c("n=10", "n=100", "n=1000"), col = c("black","blue", "red
```

From Figure 11.2, we can see that as n gets larger, the distribution of the sample mean has less variance and the resulting density plot is more concentrated around the mean.

We can also find the average of the sample means when $n = 10, 100, 1000$, which should be 0 theoretically:

```
apply(results, 2, mean)
```

```
## [1] -0.0088640968 -0.0002700293  0.0004294338
```

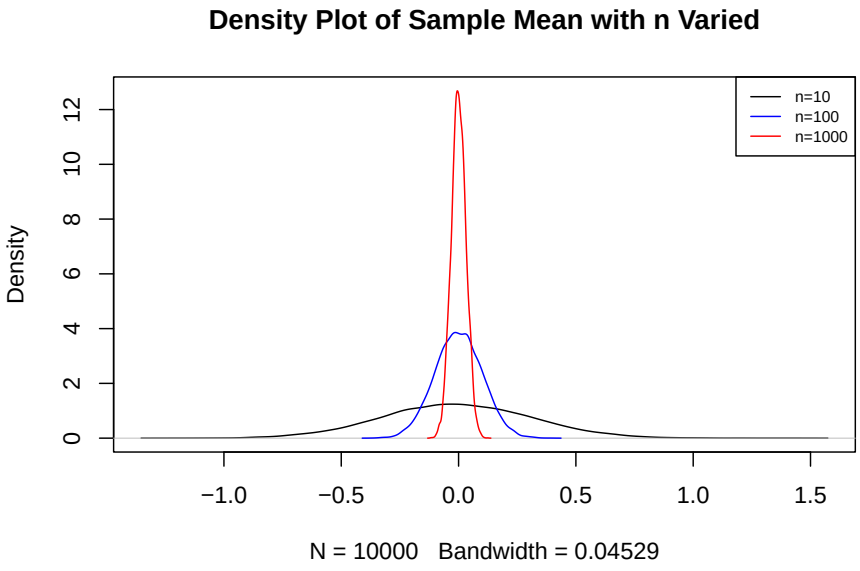


Figure 11.2: Sampling Distribution of Sample Mean when Data are Standard Normal with n = 10, 100, 1000

```
##close to 0
```

As well as the variance of the sample means when $n = 10, 100, 1000$, which should be 0.1, 0.01, and 0.001 respectively:

```
apply(results, 2, var)
```

```
## [1] 0.100813295 0.009996330 0.001000144
```

Thought question: How would you use this simulation to estimate the probability $P(\bar{X} > 0.5)$ when $n = 10$ and the data $X_1, \cdots, X_n$ are i.i.d. standard normal? Compare the answer from the simulation with the answer based on the known sampling distribution of the sample mean.

11.4 Central Limit Theorem

The previous subsection writes about the sampling distribution of the sample means when the data were i.i.d. normal. However, normal data rarely happens in real life. So is the sampling distribution of the sample mean useless? It turns out that under some conditions, the result is still applicable.

Let $X_1, \cdots, X_n$ be i.i.d. from a distribution with finite mean μ and finite variance

σ^2. The Central Limit Theorem (CLT) states that as the sample size n gets larger and tends to infinity, $\bar{X}_n = \frac{X_1+\cdots+X_n}{n}$ is **approximately** $N(\mu, \frac{\sigma^2}{n})$.

An implication of the CLT is that even if our data do not follow a normal distribution, the average value of the data can be approximated by a normal distribution, as long as our sample size is large enough, and if the data are independent and all follow the same distribution. This is useful since a lot of research questions deal with averages, and we do not need the data to be normal as long as our sample size is large enough (e.g. do average test scores improve after the implementation of a new program?).

The question then becomes: how large does the sample size n have to be for the approximation to work? Most books will say at least 25, or 30. It turns out that the skewness (or asymmetry) of the data plays a role:

- If the data are symmetric (but not normal), a smaller sample size may be enough.
- The more skewed (or asymmetric) the data, a larger sample size may be needed.

11.4.1 Example 1: Data are Uniform

In this example, we do the following:

- Simulate n observations from a standard uniform distribution.
- Calculate the sample mean.
- The previous two steps are repeated for 10,000 replicates.
- We set $n = 2, 5, 25$.
- We then plot the sampling distributions of the sample mean for each sample size. The plots are shown in Fig 11.3 below.

```
sampMean_Unif <- function(number){
  mean(runif(number))
}

Reps <- 10000

##find the sample means of n data points, with 10000 replicates
repSampMean_Unif <- function(number){
  return(replicate(Reps, sampMean_Unif(number)))
}

results_Unif <- sapply(c(2, 5, 25), repSampMean_Unif)

##set the max limit of the y axis
max_y1 <- max(density(results_Unif[,1])$y, density(results_Unif[,2])$y, density(results

##n = 2
```

```
plot(density(results_Unif[,1]), ylim=c(0, max_y1), main="Density Plot of Sample Mean with n Varie
##n = 5
lines(density(results_Unif[,2]), col="blue")
##n = 25
lines(density(results_Unif[,3]), col="red")
legend("topright", legend = c("n=2", "n=5", "n=25"), col = c("black","blue", "red"), lty = 1, cex
```

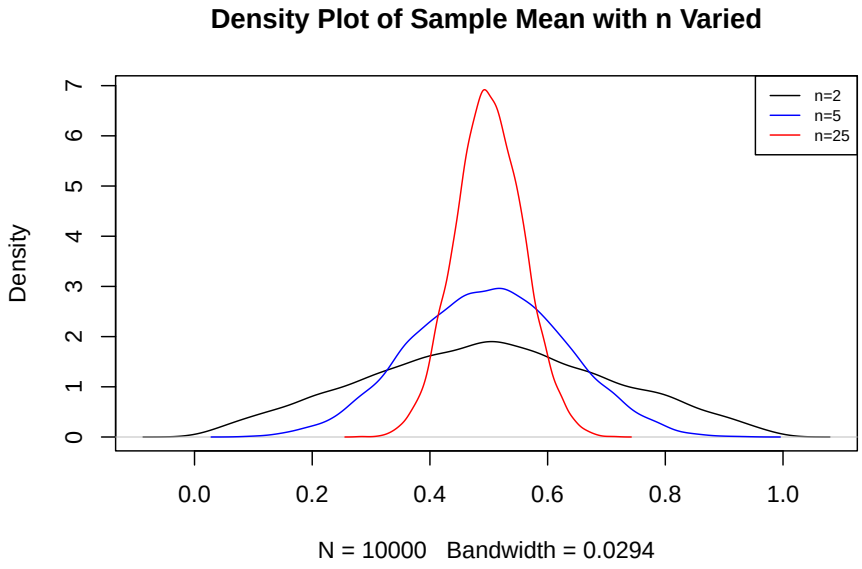


Figure 11.3: Sampling Distribution of Sample Mean when Data are Standard Uniform with n = 1, 5, 25

From Figure 11.3, we can see that when $n = 2$, the distribution of the sample means is not really normal. However, notice that when $n = 5$, the distribution already looks approximately normal. We did not need as large a sample size as 25 for the CLT to kick in.

11.4.2 Example 2: Data are Skewed

In this example, we do the following:

- Simulate n observations from an exponential distribution with rate parameter 1. This is known to be a skewed distribution.
- Calculate the sample mean.
- The previous two steps are repeated for 10,000 replicates.
- We set $n = 2, 5, 25$.

- We then plot the sampling distributions of the sample mean for each sample size. The plots are shown in Fig 11.4 below.

```
sampMean_E <- function(number){
  mean(rexp(number, rate = 1))
}

Reps <- 10000

repSampMean_E <- function(number){
  return(replicate(Reps, sampMean_E(number)))
}

results_E <- sapply(c(2, 5, 25), repSampMean_E)

max_y1 <- max(density(results_E[,1])$y, density(results_E[,2])$y, density(results_E[,3]

plot(density(results_E[,1]), ylim=c(0, max_y1), main="Density Plot of Sample Mean with
lines(density(results_E[,2]), col="blue")
lines(density(results_E[,3]), col="red")
legend("topright", legend = c("n=2", "n=5", "n=25"), col = c("black","blue", "red"), lt
```

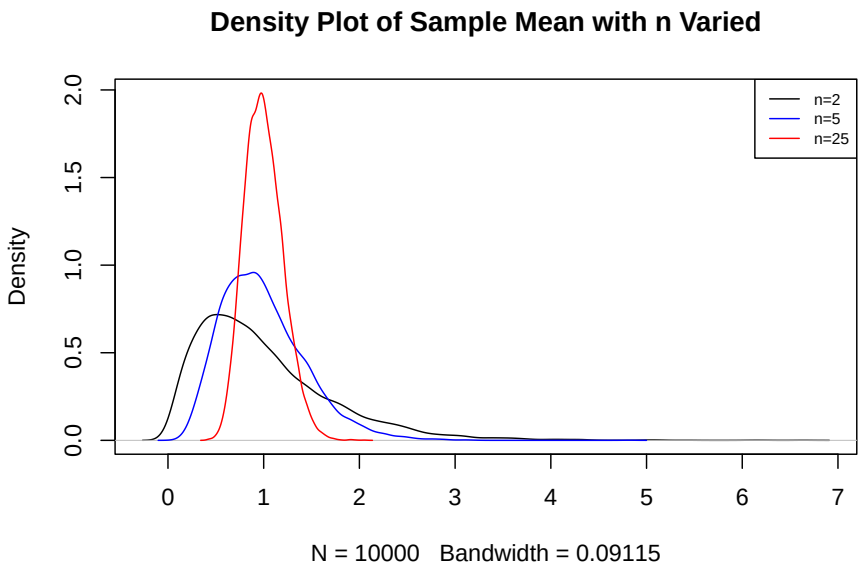


Figure 11.4: Sampling Distribution of Sample Mean when Data are Exp(1) with n = 1, 5, 25

From Figure 11.4, we can see that when $n = 5$, the distribution of the sample means when the data is exponential (skewed) is not normal. This was unlike the distribution of the sample means when the data were uniform. Although at $n = 25$, the distribution is closer to being approximately normal.

Thought question 1: How would you use this simulation to estimate the probability $P(\bar{X} > 1.2)$ when $n = 25$ and the data $X_1, \cdots, X_n$ are i.i.d. exponential with rate parameter 1? Compare the answer from the simulation with the answer based on the CLT.

Thought question 2: How would you use this simulation to estimate the probability $P(\bar{X} > 1.2)$ when $n = 5$ and the data $X_1, \cdots, X_n$ are i.i.d. exponential with rate parameter 1? Explain why we cannot calculate an answer based on the CLT.

11.5 Sampling Distribution of the Sample Proportion

Thus far, we've talked about the distribution of the sample mean (for example, the mean systolic blood pressure of American adults). The variable of interest, the blood pressure of American adults, is quantitative. What if the variable of interest is **binary** (categorical with two outcomes)? The parameter and statistic of interest with a binary variable is usually the population proportion and sample proportion respectively.

Suppose we know that the proportion of American adults who support a particular candidate is $p = 0.55$, which is our parameter of interest. If we were to take a poll of 1,000 American adults on this topic, the estimate would not be perfectly equal to 0.55, but how close to 0.55 might we expect the sample proportion to be? We want to understand how the sample proportion, $\widehat{p}$, behaves when the true population proportion is 0.55. We can simulate responses we would get from a simple random sample of 1,000 American adults, which is only possible because we know the actual support for the candidate is 0.55.

It turns out the CLT can also be applied to the sampling distribution of the sample proportions, since $\widehat{p}$ can be viewed as the average of the outcomes, if a supporter is encoded as 1 and a non supporter is encoded as 0.

When the data are independent, and if the sample size is large enough, the sampling distribution of the sample proportions is approximately $\widehat{p} \sim N\left(p, \frac{p(1-p)}{n}\right)$.

Most books will write that the sample size is large enough if $np \geq 10$ and $n(1-p) \geq 10$. Some books will say 5 instead of 10. The approximation to the normal distribution gets better as np and $n(1-p)$ get larger.

Thought question: Is there a contextual meaning behind the values of np and $n(1-p)$?

11.6 Other Common Sampling Distributions

Before listing a couple of other common sampling distributions, we state a couple of theoretical results involving expected values and variances.

Let X and Y denote random variables, and a, b, c denote some constants, then

$$E(aX + bY + c) = aE(X) + bE(Y) + c. \tag{11.1}$$

If X and Y are independent random variables, then

$$Var(aX + bY + c) = a^2Var(X) + b^2Var(Y). \tag{11.2}$$

We also state another theorem:

Theorem 11.1 (Linear Combination of Independent Normals)**.** *Let X and Y denote independent normal random variables. Any linear combination of X and Y is normal, i.e. $aX + bY$ is normal with a, b denoting constants.*

11.6.1 Sampling Distribution for Difference in Two Independent Sample Means

Suppose we want to investigate the difference in the average systolic blood pressure between female and male American adults. Suppose the systolic blood pressure for females has population mean μ_X and population variance σ_X^2 and the systolic blood pressure for males has population mean μ_Y and population variance σ_Y^2.

The population parameter is $\mu_X - \mu_Y$ and the statistic is $\bar{X} - \bar{Y}$, where X and Y denote the systolic blood pressures of female and male American adults respectively.

Under certain conditions, the sampling distribution of $\bar{X} - \bar{Y}$ is (approximately) $N(\mu_X - \mu_Y, \frac{\sigma_X^2}{n_X} + \frac{\sigma_Y^2}{n_Y})$, where n_X and n_Y denote the sample sizes of female and male American adults.

Thought Question 1: How would you use equations (11.1) and (11.2) and theorem 11.1 to explain this sampling distribution?

Thought Question 2 : What do you think are the conditions needed for this sampling distribution to be approximately normal?

11.6.2 Sampling Distirbution for Difference in Two Independent Sample Proportions

Suppose we want to investigate the difference in the proportion of female and male voters who support a candidate. Suppose the population proportion of

females who support is p_X and the population proportion of males who support is p_Y.

The population parameter is $p_X - p_Y$ and the statistic is $\hat{p}_X - \hat{p}_Y$.

Under certain conditions, the sampling distribution of $\hat{p}_X - \hat{p}_Y$ is (approximately) $N(p_X - p_Y, \frac{p_X(1-p_X)}{n_X} + \frac{p_Y(1-p_Y)}{n_Y})$, where n_X and n_Y denote the sample sizes of female and male voters.

Thought Question 1: How would you use equations (11.1) and (11.2) and theorem 11.1 to explain this sampling distribution?

Thought Question 2 : What do you think are the conditions needed for this sampling distribution to be approximately normal?

11.7 Sampling Distribution of Other Statistics

So far, we have talked about the sampling distributions of

- the sample mean (Sections 11.3 and 11.4)
- the sample proportion (Section 11.5)
- the difference in two independent sample means (Section 11.6.1)
- the difference in two independent sample proportions (Section 11.6.2)

For these statistics, we know their distributions, or approximate distributions, under certain conditions. We can perform probability calculations associated with these statistics without using Monte Carlo simulations.

However, you may encounter a situation where you need to estimate the sampling distribution of a statistic which has no known distribution. We saw an example in Section 11.4.2, where we needed the sampling distribution of the sample mean when the data are i.i.d. exponential with rate parameter 1, and when $n = 5$. The CLT does not apply in this situation so the sampling distribution of the sample mean should not be approximated by a normal distribution. However, we can still estimate probabilities using a Monte Carlo simulation as shown in Section 11.4.2.

Chapter 12

Confidence Intervals

12.1 Introduction

In Section 11, we learned about sampling distributions of statistics. We established that statistics such as the sample mean have randomness in them. If we were to obtain different random samples of 750 American adults, the sample mean blood pressure from the samples are likely to be different from each other. So is there variability in our sample mean due to random sampling.

You might notice in Section 11 that we know the sampling distribution of some common statistics (under certain conditions), or we can derive the sampling distribution using simulations. In these instances, the value of the corresponding parameters were known. This is unrealistic in real life; the purpose of what we do is to use data from our sample to infer the characteristics in the population!

This section will be more realistic: the values of the population parameters are unknown. In this section, we will introduce confidence intervals. Confidence intervals build on the ideas from Section 11: that statistics are random and we can quantify their uncertainty (variance). The purpose of a confidence interval is to provide a range of plausible values for an unknown population parameter, based on a sample. A confidence interval not only provides the estimated value of the parameter, but also a measure of uncertainty associated with the estimation.

We will first cover confidence intervals for the mean and confidence intervals for the proportion, two of the most basic confidence intervals. These intervals are based on the fact that the distribution of their corresponding statistics, the sample mean and sample proportion, are **known** as long as certain conditions are met. You will notice that the general ideas in finding confidence intervals are pretty similar; confidence intervals for other statistics that have known distributions will be constructed similarly.

We will learn about the bootstrap, which is used to construct confidence intervals

when the distribution of a statistic is **unknown**, in Section 13.

12.2 Confidence Interval for the Mean

12.2.1 Randomness of Estimators

Suppose we are trying to estimate the mean systolic blood pressure of all American adults, by using the sample mean of 750 randomly selected American adults. The sample mean is an estimator for the population mean. We want to be able to report the value of the estimator, as well as our uncertainty about the estimator. A way to measure the uncertainty of an estimator is through the **variance or standard error of the estimator**. Larger values indicate a higher degree of uncertainty, as an estimator with larger variance means that the value of the estimator is likely to be different among random samples.

We will start with the confidence interval for a mean, since its sampling distribution is known (see Section 11.3). We will use this setting to explain concepts associated with confidence intervals. We will then generalize these concepts to other estimators with known distributions.

12.2.2 Randomness of Confidence Intervals

The **confidence interval** is a probability applied to an estimator such as $\bar{X}_n$. Instead of focusing on the estimated value of the sample mean and its variance, we construct a confidence interval for the mean that takes the form:

$$I = \left(\bar{X}_n - \epsilon, \bar{X}_n + \epsilon\right). \tag{12.1}$$

Some terminology associated with intervals of the form in equation (12.1):

- ϵ is often called the **margin of error**. (Yes that margin of error that you often see reported in election polls). This value is a function of the standard error of the estimator, so it gives a measure of uncertainty of the estimated value. The margin of error can be interpreted as the maximum difference between a sample estimator and the true, unknown value in the population (the parameter) due to random sampling variability.
  - Remember that the uncertainty being measured is the uncertainty due to random sampling, not due to other sources of uncertainty such as not getting a representative sample, people lying, etc. As mentioned in earlier modules, other methods are used to handle such issues and belong to the field of survey sampling, which is very interesting and an active area of research. We will not get into these issues in this class.
- The value of $\bar{X}_n - \epsilon$ is often called the **lower bound** of the confidence interval.

- The value of $\bar{X}_n + \epsilon$ is often called the **upper bound** of the confidence interval.
- The value of $\bar{X}_n$ is often called the **point estimate** of the population mean.

Equation (12.1) is sometimes expressed as

$$\text{point estimate } \pm \text{ margin of error.} \tag{12.2}$$

Indeed, a lot of confidence intervals take on the form expressed in equation (12.2), as long as the sampling distribution of the estimator is symmetric. Given the interval for the mean expressed in equation (12.1), we ask what is the probability that the interval I includes the true value of the parameter μ, i.e. we want to evaluate

$$P(\mu \in I) = P(\bar{X}_n - \epsilon \leq \mu \leq \bar{X}_n + \epsilon). \tag{12.3}$$

It is important to bear in mind that since the estimator, the sample mean $\bar{X}_n$ is a random variable, there will also be randomness in the interval I. The numerical values of the lower and upper bounds will change with a different random sample, since the value of $\bar{X}_n$ will change.

The idea of the interval I being random is represented in Figure 12.1 below:

- The density curve in the top of Figure 12.1 represents the PDF of a random variable, that represents the distribution of some variable in the population that we wish to study. In this example, the distribution is normal.
- Each row of dots represents the values of 10 randomly sampled data points from the PDF.
- The colored lines in each row represent the lower and upper bounds of a 50% confidence interval calculated from the sampled data points in the row.
- The colored dot in the center of the confidence interval represents $\bar{x}$ for the sampled data points in the row.
- The intervals in blue represent confidence intervals that contain the value of μ, while the intervals in red represent confidence intervals that do not contain the value of μ.

In Figure 12.1, we note that 50% of the confidence intervals capture the value of μ, so the probability per equation (12.3) is 0.5. This matches with theory since each confidence interval in Figure 12.1 was computed at 50% confidence.

If we were to create 95% confidence intervals for each row in Figure 12.1, the upper and lower bounds of the intervals will be adjusted so that we will expect 19 of these 20 intervals to contain the value of μ.

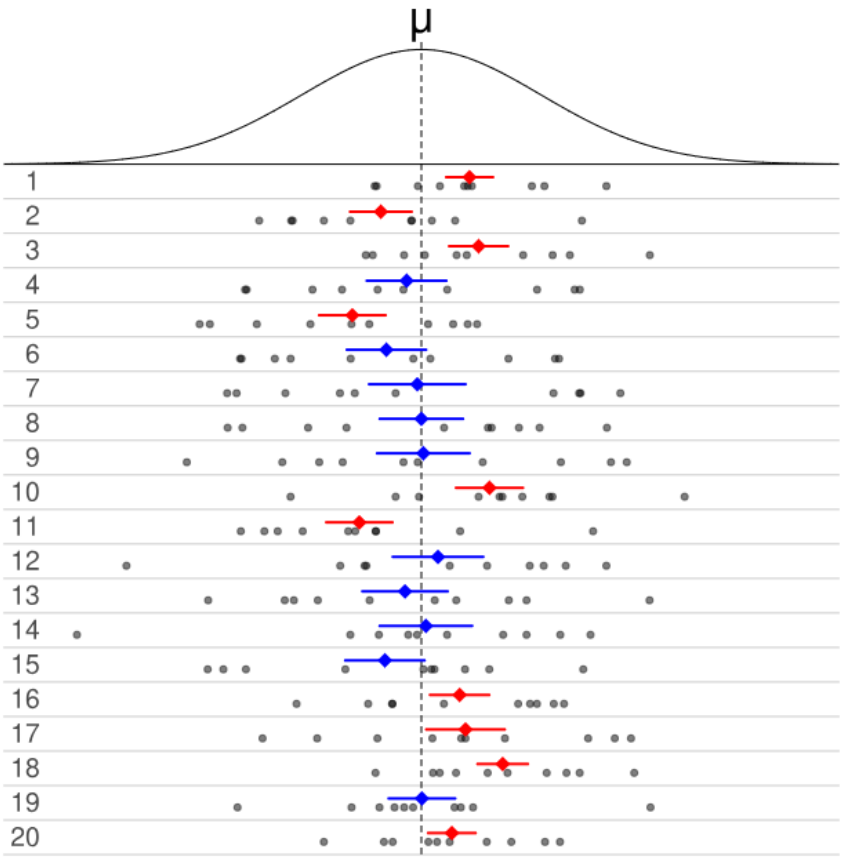


Figure 12.1: Randomness of Confidence Interval. Picture from wikipedia

This illustration gives us an interpretation of the probability associated with a confidence interval per equation (12.3): When we construct a 95% confidence interval, there is a 95% chance the random interval I will contain the true value of the parameter. In other words, if we have 100 random samples and we construct 95% confidence intervals based on each sample, we expect 95 of these intervals to contain the value of the parameter.

The idea of the probability that the random interval I captures the true parameter gives rise to the **confidence level**. The confidence level of a confidence interval is denoted by $1-\alpha$, i.e. if we construct an interval at 95% confidence, $\alpha = 0.05$. Equation (12.3) can be written as

$$P(\bar{X}_n - \epsilon \leq \mu \leq \bar{X}_n + \epsilon) = 1 - \alpha. \tag{12.4}$$

We will then say I is a $(1-\alpha)\times 100\%$ confidence interval, or I is a confidence interval with confidence level of $(1-\alpha)\times 100\%$.

Now that we have established that confidence intervals are random and the definition of the confidence level, we are ready to go into the details of constructing the confidence interval for the mean.

12.2.3 Constructing Confidence Interval for the Mean

We remind ourselves of the sampling distribution of the sample mean, $\bar{X}_n$, from Sections 11.3 and 11.4. There are a couple of conditions to consider:

1. $X_1, \cdots, X_n$ are i.i.d. from a normal distribution with finite mean μ and finite variance σ^2. Then $\bar{X}_n \sim N(\mu, \frac{\sigma^2}{n})$.
2. $X_1, \cdots, X_n$ are i.i.d. from any distribution with finite mean μ and finite variance σ^2, and if n is large enough, then $\bar{X}_n$ is approximately $N(\mu, \frac{\sigma^2}{n})$.

If either of these conditions are met, then the distribution of $\bar{X}_n$ after standardization is either a standard normal or approaches a standard normal distribution, so $\frac{\bar{X}_n - \mu}{\frac{\sigma}{\sqrt{n}}} = \frac{\bar{X}_n - \mu}{\sqrt{Var(\bar{X})}} = \frac{\bar{X}_n - \mu}{SE(X_n)}$ is either standard normal or approximately standard normal when n is large enough.

Note: $SE(\bar{X}_n)$ is the standard error of the sample mean. The standard error of any estimator is the standard deviation of the estimator, or the squareroot of the variance of the estimator.

To simplify the notation as it pertains to the confidence interval for the mean, we will let $\hat{Z} = \frac{\bar{X}_n - \mu}{SE(\bar{X})}$, so $\hat{Z}$ is standard normal or approximately standard normal. $\hat{Z}$ can be called the **standardized version of the sample mean**.

12.2.3.1 Critical Value

We perform some math operations on equation (12.4) to see how we can construct a confidence interval for the mean:

$$\begin{aligned} P(\bar{X}_n - \epsilon \leq \mu \leq \bar{X}_n + \epsilon) &= 1 - \alpha \\ \implies P(|\bar{X}_n - \mu| \leq \epsilon) &= 1 - \alpha \\ \implies P\left(|\hat{Z}| = |\frac{\bar{X}_n - \mu}{SE(\bar{X}_n)}| \leq \frac{\epsilon}{SE(\bar{X}_n)} = z^*\right) &= 1 - \alpha \\ \implies P(|\hat{Z}| \leq z^*) &= 1 - \alpha \\ \implies P(-z^* \leq \hat{Z} \leq z^*) &= 1 - \alpha. \end{aligned} \tag{12.5}$$

In equation (12.5), z^* is called the **critical value**. So we can see how it is related to the margin of error, ϵ: the margin of error is the critical value multiplied by the standard error of the estimator (which in this case is the standard error of the sample mean since we are constructing the confidence interval for the mean).

In words, equation (12.5) says that we want to find the critical value z^* so that the probability that a standardized score is between $-z^*$ and z^* is $1-\alpha$. Visually, this probability is displayed in Figure 12.2 below when $\alpha = 0.05$. We want to find the values on the horizontal axis so that the blue shaded area corresponds to a value of 0.95 (recall that area under a PDF represents probability).

We continue working with equation (12.5) to see how we obtain the value of z^*, as long as either of the two conditions for the sampling distribution of $\bar{X}_n$ to be known are met:

$$\begin{aligned} P(-z^* \leq \hat{Z} \leq z^*) &= P(\hat{Z} \leq z^*) - P(\hat{Z} \leq -z^*) \\ &= \Phi(z^*) - \Phi(-z^*) = 1 - \alpha. \end{aligned} \tag{12.6}$$

where $\Phi(z) = P(\hat{Z} \leq z)$ is the CDF of a standard normal. Due to the symmetry of the standard normal, $\Phi(-z^*) = 1 - \Phi(z^*)$, and we sub this into equation (12.6) and continue working with it to solve for z^*:

$$\begin{aligned} P(-z^* \leq \hat{Z} \leq z^*) &= 2\Phi(z^*) - 1 = 1 - \alpha \\ \implies \Phi(z^*) &= 1 - \frac{\alpha}{2} \\ \implies z^* &= \Phi^{-1}\left(1 - \frac{\alpha}{2}\right) \end{aligned} \tag{12.7}$$

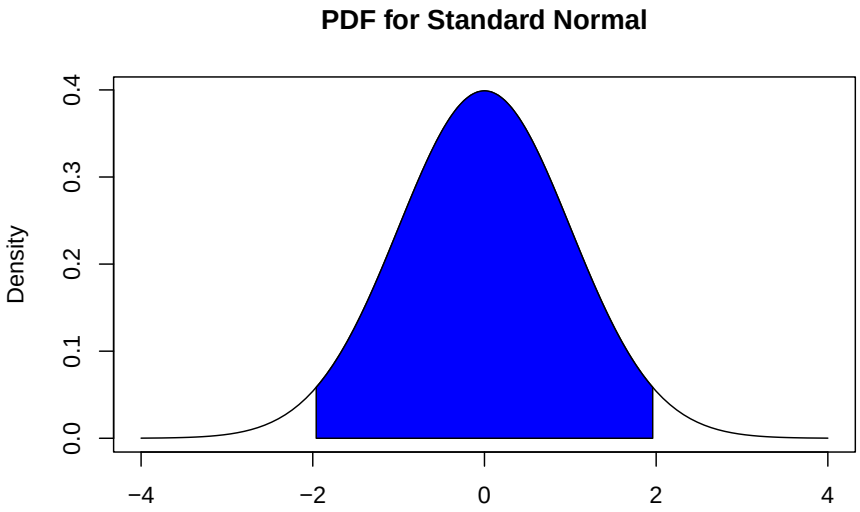


Figure 12.2: Finding Critical Value with 95% Confidence

So z^* is found by inverting the CDF of a standard normal evaluated at $1 - \frac{\alpha}{2}$. This quantity can easily be found using R. For example, for 95% confidence, $\alpha = 0.05$, we type:

```
alpha<-0.05
qnorm(1-alpha/2)
```

```
## [1] 1.959964
```

which tells us the critical value is 1.96 for 95% confidence.

Thought question: Can you show that the critical value for 96% confidence is about 2.054? Can you show that the critical value for 98% confidence is about 2.326?

12.2.3.2 Confidence Interval for the Mean

We are now ready to put the pieces together to work on the confidence interval for the mean:

$$\begin{aligned} P(-z^* \leq \hat{Z} \leq z^*) &= P(-z^* \leq \frac{\bar{X}_n - \mu}{SE(\bar{X}_n)} \leq z^*) \\ &= P\left(-z^* SE(\bar{X}_n) \leq \bar{X}_n - \mu \leq z^* SE(\bar{X}_n)\right) \\ &= P\left(\bar{X}_n - z^* SE(\bar{X}_n) \leq \mu \leq \bar{X}_n + z^* SE(\bar{X}_n)\right) \\ &= P\left(\bar{X}_n - z^* \frac{\sigma}{\sqrt{n}} \leq \mu \leq \bar{X}_n + z^* \frac{\sigma}{\sqrt{n}}\right). \end{aligned} \tag{12.8}$$

Therefore, the $(1-\alpha) \times 100\%$ confidence interval for the mean is

$$\left(\bar{x}_n - z^* \frac{\sigma}{\sqrt{n}}, \bar{x}_n + z^* \frac{\sigma}{\sqrt{n}}\right). \tag{12.9}$$

Again, the formula in equation (12.9) is only valid if either of the two conditions in Sections 11.3 or 11.4 is met, i.e. either the data are originally normal, or if the sample size is large enough. As noted in Section 11.4, a larger sample size will be required as the data is more asymmetric.

12.2.3.3 Coverage Probability

Figure 12.1 also illustrates the concept of **coverage probability**. 20 random samples were drawn, and corresponding 50% confidence intervals were constructed, and we find that 10 out of 20 of these intervals contained the true value of the parameter. The coverage probability is 50%, since 10 out of 20 intervals contained the true value value of the parameter. This matches the confidence level of 50%. The **coverage probability should match the confidence level**, if not, the distribution that we used for the sampling distribution of the estimator or statistic is probably incorrect. This is usually a by-product of not meeting conditions to construct the interval.

So in general, for confidence intervals constructed at $(1-\alpha) \times 100\%$ confidence, the coverage probability of the confidence intervals should be $(1-\alpha) \times 100\%$.

We run a Monte Carlo simulation to show that the coverage probability of confidence intervals for the mean using equation (12.9). The code below does the following:

- Simulate $X_1, \cdots, X_{10}$ i.i.d. from standard normal.
- Calculate 95% confidence interval for the mean using (12.9).
- Assess if the calculated confidence interval contains 0, since the true mean is 0 (we simulated data from standard normal)
- Repeat these steps for a total of 10 thousand replicates.
- Count the number of confidence intervals that contain 0, and divide by the number of replicates. This value estimates the coverage probability.

- If we did things correctly (correct use of formula, correct distribution for estimator), this estimated coverage probability should be close to the confidence level of 95%.

```
alpha <- 0.05

## function that simulates data from standard normal,
## then constructs CI for mean, and assesses if CI
## contains 0 or not
mean_Z_cover <- function(n, alpha){

  X <- rnorm(n) ##draw n data points from standard normal

  ##calculate elements needed for CI of mean
  xbar <- mean(X) ##sample mean
  SE <- 1/sqrt(n) ##SE of sample mean
  crit <- qnorm(1-alpha/2) ##critical value
  ME <- crit*SE ##margin of error
  lower <- xbar-ME ##lower bound of CI
  upper <- xbar+ME ##upper bound of CI

  ##does CI contain 0?
  result <- lower < 0 & upper > 0
  return(result)

}

## write another function to replicate this simulation, and
## find the proportion of samples with CIs that contain 0
reps <- 10000

rep_mean_Z_cover <- function(n, alpha){

  result <- replicate(reps, mean_Z_cover(n, alpha))
  return(mean(result))
}

## should be close to the value of 1-alpha (confidence level),
## which is 0.95 in this simulation

set.seed(31)
rep_mean_Z_cover(10, alpha)
```

```
## [1] 0.9473
```

The estimated coverage probability based on 10 thousand replicates is 94.73%, very close to the confidence level of 95%. This informs us that the distribution

assumed for the sample mean was correct.

12.2.4 Confidence Interval for the Mean Using t Distribution

You may have noticed that when we calculate a confidence interval for the mean using equation (12.9), it involves knowing the value of σ^2, the variance of the variable in the population, which is a parameter. However, we have mentioned that the whole purpose of estimation and confidence intervals is to estimate the value of unknown parameters and quantify the uncertainty associated with the estimate. The numerical value of parameters are very rarely known! So how could we actually use equation (12.9) in real life?

The solution to this is fairly intuitive, we use the sample variance $s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$ to estimate σ^2. What is less intuitive is that the distribution of the standardized version of the sample mean changes.

We had earlier mentioned that if certain conditions are met, then $\hat{Z} = \frac{\bar{X}_n - \mu}{\frac{\sigma}{\sqrt{n}}}$ is either standard normal or approximately standard normal. If we estimate σ^2 with s^2 and replace σ with s in $\hat{Z}$, we get a new random variable

$$T = \frac{\bar{X}_n - \mu}{\frac{s}{\sqrt{n}}} = \frac{\bar{X}_n - \mu}{SE(\bar{X}_n)}, \tag{12.10}$$

where $SE(\bar{X}_n)$ is now $\frac{s}{\sqrt{n}}$. It turns out that T follows another well-known distribution, called the t **distribution with** $n-1$ **degrees of freedom**. The PDF of a t distribution is pretty long and is not needed for this course (you can look it up on your own), but we take a look at the plot of its PDF and compare it with the plot of the PDF of a standard normal, in Figure 12.3 below.

```
##plot PDF from -5 to 5
x <- seq(-5, 5, length.out = 100)

##plot the standard normal
curve(dnorm(x), from = -5, to = 5, lwd = 2,
      ylab = "Density", xlab = "x",
      main = "Pdf of Standard Normal and t-Distribution")

##overlay the t-distribution with 1 and 10 degree of freedom
curve(dt(x, df = 1), from = -5, to = 5, col = "red", lwd = 2, add = TRUE)
curve(dt(x, df = 10), from = -5, to = 5, col = "blue", lwd = 2, add = TRUE)
legend("topright", legend = c("Standard Normal", "t-Distribution (df=1)", "t-Distributi
       col = c("black", "red", "blue"), lty = 1, lwd = 2)
```

From Figure 12.3, we note a few things about the t distribution:

- It is centered at 0, just like a standard normal.

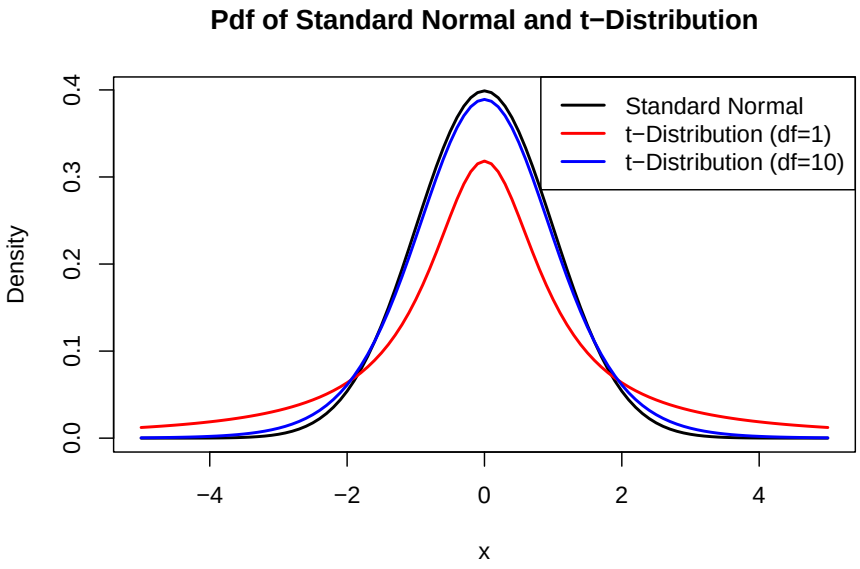


Figure 12.3: Plot of PDF for Z and t Distributions

- It is also symmetric and bell-shaped, just like a standard normal.
- It has heavier tails than the standard normal. In other words, extreme values (large or small) have slightly higher probabilities of occurring for a t distribution than for a standard normal.
- As the degree of freedom increases, the t distribution gets closer to a standard normal. Notice how the blue curve is closer to the standard normal curve in black, than the red curve is with the black curve. In fact, one can show mathematically that the PDF of a t distribution converges to the PDF of a standard normal as the degree of freedom increases to infinity.

How does the fact that we are working with a t distribution instead of a standard normal affect how we calculate a confidence interval for the mean? How does equation (12.9) change? We replace σ^2 with s^2 and z^* with t^*. The critical value is now denoted by t^* to emphasize that it is based on the t distribution.

As an example to find the critical value of a t distribution with 10 degrees of freedom and with 95% confidence:

```
alpha<-0.05
qt(1-alpha/2, 10) ##supply percentile first, then value of df
```

```
## [1] 2.228139
```

Therefore, the $(1-\alpha)\times 100\%$ confidence interval for the mean , when σ^2 is unknown, is

$$\left(\bar{x}_n - t^* \frac{s}{\sqrt{n}}, \bar{x}_n + t^* \frac{s}{\sqrt{n}}\right). \tag{12.11}$$

Again, the formula in equation (12.11) is only valid if either of the conditions in Sections 11.3 or 11.4 is met, i.e. either the data are originally normal, or if the sample size is large enough.

12.2.4.1 Worked Example

A sample of 66 obese adults was put on a low-carbohydrate diet for a year. The average weight loss was 11 lb and the standard deviation was 19 lb. Calculate a 99% confidence interval for the true average weight loss. Does the confidence interval provide support for the claim that the mean weight loss is positive?

From the question, we summarize the information as:

- $n = 66$,
- $df = n - 1 = 65$
- $\bar{x} = 11$,
- $s = 19$,
- $\alpha = 0.01$, so t^* is found using `qt(1-0.01/2, 65)` which is 2.653604.

We have no information about the distribution of the data. However, the sample size is 66, which is usually large enough to use the CLT, so we can use equation (12.11):

$$\left(11 - 2.653604\frac{19}{\sqrt{66}}, 11 + 2.653604\frac{19}{\sqrt{66}}\right)$$

which gives (4.793914, 17.20609). Since the entire confidence interval lies above 0, our data support the claim that the mean weight loss is positive.

Interpreting the CI: There is 99% probability that the random interval (4.793914, 17.20609) will include the true average weight loss of obese adults on a low-carbohydrate diet for a year.

What else can we say from the confidence interval?

- Values outside the confidence interval are considered to be "ruled out" as plausible values of the parameter. So if we wanted to assess if the average weight loss of obese adults on a low-carbohydrate diet for a year is 20 pounds, our interval does not support this claim, since the value of 20 lies outside the interval. We can say our data do not support the claim that the average weight loss of obese adults on a low-carbohydrate diet for a year is 20 pounds.

- Values inside the confidence interval are considered to be plausible values of the parameter. Any value inside the interval is considered plausible. A common mistake will be to specify a certain value in the interval, and conclude that the parameter is equal to that specific value. For example, it will be a mistake to say that since the value 10 lies inside the interval, the interval supports the claim that the average weight loss of obese adults on a low-carbohydrate diet for a year is 10 pounds. This is because other values in the interval are still considered plausible.
  - In such a situation, we will say that our data do not support the claim that the average weight loss of obese adults on a low-carbohydrate diet for a year is different from 10 pounds, since 10 lies inside the interval. We cannot rule out the value of 10.

12.2.4.2 Confidence Interval for Mean Using t VS Using z

A couple of interesting things to note with critical values associated with the t distribution:

- For the same level of confidence, t^* will never be smaller than z^*.

Thought question: Can you give an intuitive explanation as to why? Using Figure 12.3 may be helpful.

- The value of t^* gets closer to the value of z^* as the degree of freedom gets larger. This implies that t^* is approximately equal to z^* for large sample sizes.

Recall that the margin of error is the critical value multiplied by the standard error of the estimator. So margin of errors tend to be larger when σ^2 is unknown. It should make intuitive sense that we have a higher degree of uncertainty since we have an additional parameter to estimate.

Since the margin of error tends to be larger when σ^2 is unknown, it means the **width** of the confidence interval for the mean tends to be wider when σ^2 is unknown. The width of a confidence interval is the difference between its upper and lower bound, or twice the margin of error.

Thought question: Try running the Monte Carlo simulation in Section 12.2.3.3, but now use the sample variance instead of the population variance. Find the coverage probabilities of the confidence interval if we use the correct formula in equation (12.11), and if we use a z^* as the critical value with the sample variance.

12.2.4.3 Degrees of Freedom

An intuitive explanation of degrees of freedom is the number of independent pieces of information that can take on any numerical value, when estimating a parameter.

Generally speaking, we lose 1 degree of freedom for every equation that must be satisfied. In the context of estimating the population mean using the sample mean, we must always satisfy the equation $\bar{x} = \frac{\sum_{i=1}^{n} x_i}{n}$, so we lose 1 degree of freedom from an original set of n observations, so the degree of freedom is $n-1$ when calculating the CI for the mean.

12.2.5 Factors Affecting Precision of Confidence Intervals

The width of a confidence interval is used as a measure of **precision**. A wider width indicates less precision and a higher degree of uncertainty with our estimate. Narrower widths are preferred, as we are able to narrow the range of plausible values for the unknown parameter by ruling out more values. The factors (that are within the researcher's control) affecting the width of a confidence interval are the items used in calculating the margin of error:

- The level of confidence, $1-\alpha$: The width increases as the level of confidence increases.

Though question: Can you use Figure 12.2 to help you explain the width increases as level of confidence increases?

- The sample size, n: As n increases, the width decreases. This implies that we have more precision, and a lower degree of uncertainty, with larger sample sizes.

Other factors such as whether σ^2 is unknown or not, and the value of the variance, are usually not controllable by the researcher.

12.3 Confidence Interval for the Proportion

Next, we will go over the confidence interval for the proportion. This is another common confidence interval. The sample proportion is an estimator for the population proportion.

Proportions are used to summarize categorical variables, whereas means are used to summarize quantitative variables. Consider the following two research questions:

- We measure the systolic blood pressure of a sample of American adults. The variable is quantitative, since the answer is a numeric value where arithmetic operations can be applied. We then calculate the average systolic blood pressure of American adults. So we can work on a confidence interval for the mean, using equation (12.11).
- We ask a sample of voters whether they support a particular candidate. The variable is categorical, since the answer is yes or no, and we cannot apply arithmetic operations to the answer. We then calculate the proportion of voters who support the candidate. So we need a confidence interval

for the proportion.

12.3.1 Sampling Distribution of Sample Proportions

Restating the information from Section 11.5: When the data are independent, and if the sample size is large enough, the sampling distribution of the sample proportions is approximately $\hat{p} \sim N\left(p, \frac{p(1-p)}{n}\right)$.

12.3.2 Constructing Confidence Interval for the Proportion

A lot of the concepts and math from the confidence interval for the mean carry over to the confidence interval for the proportion. We will skip the math for the confidence interval for the proportion (although you should be able to derive this result by following the steps and adjusting).

The $(1-\alpha) \times 100\%$ confidence interval for the proportion is

$$\left(\hat{p} - z^* \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}, \hat{p} + z^* \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}\right). \tag{12.12}$$

From equation (12.12), we can see that

- The point estimate is $\hat{p}$.
- The standard error of $\hat{p}$ is $SE(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$.
- The margin of error is $z^* \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$.

The framework of confidence intervals usually takes on the form given in equation (12.2): we add and subtract the margin of error to and from the point estimate. The margin of error is the critical value multiplied by the standard error of the estimator. We see the confidence intervals for the mean and proportion take on this framework, and so do most confidence intervals of other estimators, as long as the sampling distribution of the estimator is symmetric.

Note: equation (12.12) is based on proportions, and not percentages. After calculating the confidence interval for the proportion, reporters often convert these values to percentages by multiplying them by a hundred.

From equation (12.12), we can also see that we have the factors that affect the width of the confidence interval for the proportion is the same as the confidence interval for the mean.

12.3.2.1 Worked Example

The Texas College Tobacco Project survey administered in 2016 found that in a sample of 5767 undergraduates ages 18–25, 525 said they had used electronic cigarettes at least once during the previous 30 days. Find a 95% confidence

interval for the proportion of all students in the population sampled who used e-cigarettes during the previous 30 days. Also report the margin of error. Do the data support the claim that more than 5% of students in the population used e-cigarettes during the previous 30 days?

From the question, we summarize the information as:

- $n = 5767$,
- $\hat{p} = \frac{525}{5767}$,
- $\alpha = 0.05$, so z^* is found using `qnorm(1-0.05/2)` which is 1.959964.

The number of "successes" is 525 and the number of "failures" is $5767 - 525 = 5242$. These are both a lot larger than 10, so we can work with sampling distribution of $\hat{p}$ and calculate a confidence interval using equation (12.12):

$$\left(\frac{525}{5767} - 1.959964\sqrt{\frac{\frac{525}{5767}(1-\frac{525}{5767})}{5767}}, \frac{525}{5767} + 1.959964\sqrt{\frac{\frac{525}{5767}(1-\frac{525}{5767})}{5767}} \right)$$

which gives (0.08361097, 0.09845943), with margin of error 0.007424228. The data support the claim that more than 5% of students in the population used e-cigarettes in the previous 30 days, since the entire interval lies above 0.05.

There is a 95% probability that the random interval (0.08361097, 0.09845943) contains the true proportion of students in the population used e-cigarettes in the previous 30 days.

12.4 Confidence Interval for Difference in Two Independent Means

If we want to compare the averages of a quantitative variable between two groups (based on a binary variable), we can construct a confidence interval for the difference in two independent means.

Given what we have established about confidence intervals and knowing the sampling distribution for the difference in two independent sample means (in Section 11.6.1), the $(1-\alpha) \times 100\%$ confidence interval for the difference in two independent means, is

$$\left((\bar{X} - \bar{Y}) - t^* \sqrt{\frac{s_X^2}{n_X} + \frac{s_Y^2}{n_Y}}), (\bar{X} - \bar{Y}) + t^* \sqrt{\frac{s_X^2}{n_X} + \frac{s_Y^2}{n_Y}}) \right). \tag{12.13}$$

The condition for this confidence interval is that both populations are normally distributed, or that the sample sizes are both large enough.

The only question is: what is the degrees of freedom associated with this test? It is actually quite complicated, and is approximated using the Welch–Satterthwaite equation. The `t.test()` function in R computes confidence intervals using this degrees of freedom.

In some older references, it is suggested to approximate the degrees of freedom using $min(n_X, n_Y) - 1$. The confidence interval using this approximation is considered to be more **conservative** than the approximation based on the Welch–Satterthwaite equation. A confidence interval that is more conservative means that it is wider than it needs to be.

Thought Question: How is the coverage probability of a conservative confidence interval related to its confidence level?

12.4.1 Pooled vs Unpooled

The confidence interval for the difference in two independent means given in equation (12.13) is the modern default. It is sometimes called a **Welch's** t confidence interval, or an **unpooled version** of the confidence interval.

You may have come across another version called the **pooled version** of the confidence interval. The pooled version has an additional assumption: that the variances of the quantitative variable is equal in the two populations (i.e. $\sigma_X^2 = \sigma_Y^2$). The pooled version has fallen out of favor especially when the variances are unequal and when the sample sizes are unequal, as the coverage probabilities do not match the confidence level.

12.5 Confidence Interval for Difference in Two Independent Proportions

If we want to compare the proportions (based on a binary variable) between two groups (based on a binary variable), we can construct a confidence interval for the difference in two independent proportions.

The $(1 - \alpha) \times 100\%$ confidence interval for the difference in two independent proportions, is

$$(\hat{p}_X - \hat{p}_Y) \pm z^* \sqrt{\frac{\hat{p}_X(1-\hat{p}_X)}{n_X} + \frac{\hat{p}_Y(1-\hat{p}_Y)}{n_Y}} \tag{12.14}$$

This is reliable if the number of successes and failures in both groups are large, i.e. $n_X\hat{P}_X, n_X(1-\hat{P}_X), n_Y\hat{P}_Y, n_Y(1-\hat{P}_Y)$ are all large.

Chapter 13

The Bootstrap

13.1 Introduction

In Section 12, we learned about confidence intervals for

- the mean,
- the proportion,
- the difference in two independent means, and
- the difference in two independent proportions.

These intervals are based on their known sampling distributions, which usually require some conditions to be met. Confidence intervals for any other parameter that are based on known sampling distributions work similarly. We work out the confidence interval of an estimator by using the framework in equation (12.2): we add and subtract the margin of error to and from the point estimate. The margin of error is the critical value multiplied by the standard error of the estimator. The critical value is based on the sampling distribution of the estimator. This framework requires us to know the sampling distribution and the standard error of the estimator.

This naturally leads to a question: what if the conditions for these sampling distributions are not met, or if the estimator we are working on has no known sampling distribution? In this section, we will learn about the bootstrap, which is used when the sampling distribution of an estimator is unknown.

13.2 The Bootstrap

There are estimators whose sampling distributions are unknown, for example, the sample median. How can we construct confidence intervals for the median?

One idea could be to collect many random samples, compute the sample me-

dian from each sample, and create a density plot like we did in Monte Carlo simulations in Figures 11.2, 11.3, and 11.4. However, this is not feasible in real life as we rarely have the time and money to go around obtaining many random samples. We are only able to work with the one random sample that we have.

A method to work around these issues is the **bootstrap**. The bootstrap will enable us to:

- Estimate the standard error and variance of estimators if there are no known formulas for them.
- Construct confidence intervals for estimators with unknown sampling distributions and unknown formulas for their standard errors.

We can do these without having to collect data from many random samples. Before going into how the bootstrap works, we remind ourselves of how Monte Carlo simulations work in terms of finding the sampling distribution and standard errors of estimators.

13.2.1 Estimating Sampling Distributions Using Many Random Samples

When using Monte Carlo simulations to estimate the sampling distribution of estimators (see the code that generated Figures 11.2, 11.3, and 11.4 as examples), we typically did the following:

1. Assume the data follow some known distribution with PDF f_X.
2. Simulate n data points from this assumed distribution K times, with K being large, to obtain K replicate datasets of size n denoted by $\mathcal{X}^1, \cdots, \mathcal{X}^K$.
3. For each dataset, calculate the value of the estimator $\hat{\Theta}$. Since we have K datasets, we will have K values for the estimators, denoted by $\hat{\Theta}^1, \cdots, \hat{\Theta}^K$.
4. We estimate the true mean of $\hat{\Theta}$ by taking the sample mean of $\hat{\Theta}^1, \cdots, \hat{\Theta}^K$.
5. We estimate the true variance of $\hat{\Theta}$ by taking the sample variance of $\hat{\Theta}^1, \cdots, \hat{\Theta}^K$. Square-rooting this value gives an estimate for the standard error of $\hat{\Theta}$.
6. We create a density plot of $\hat{\Theta}^1, \cdots, \hat{\Theta}^K$ to estimate the sampling distribution of $\hat{\Theta}$.

As written earlier, the problem with applying this idea to an estimator that has an unknown sampling distribution is that we do not know its associated PDF f_X. So we cannot carry out the steps listed. What the bootstrap does is to mimic these steps, when we only have access to data from one random sample. Next, we describe the steps that are carried out in the bootstrap. Pay attention to the similarities and differences in these steps with the steps above.

13.2.2 The Bootstrap Algorithm

Instead of simulating K replicate datasets from an assumed distribution, we simulate K replicate datasets by sampling, **with replacement**, n data points from the original dataset. The "with replacement" term is key: it means that when a data point is sampled, it is returned to the original dataset and it can be sampled again.

Let $\mathcal{X}$ denote the original dataset with n observations. Let $\hat{\theta}$ denote the value of the estimator from the original dataset.

1. Synthesize K bootstrapped datasets $\mathcal{Y}^{(1)}, \cdots, \mathcal{Y}^{(K)}$ where each bootstrapped dataset is derived by sampling n data points from the original dataset, with replacement.

2. For each dataset, calculate the value of the estimator $\hat{\Theta}$. Since we have K bootstrapped datasets, we will have K values for the estimators, denoted by $\hat{\Theta}^{(1)}, \cdots, \hat{\Theta}^{(K)}$.

3. We estimate the expected value of $\hat{\Theta}$ by taking the sample mean of $\hat{\Theta}^{(1)}, \cdots, \hat{\Theta}^{(K)}$. This value is denoted by $M_b(\hat{\Theta})$.

4. We estimate the true variance of $\hat{\Theta}$ by taking the sample variance of $\hat{\Theta}^{(1)}, \cdots, \hat{\Theta}^{(K)}$. This value is denoted by $V_b(\hat{\Theta})$, which we call the bootstrap variance of the estimator. Square-rooting $V_b(\hat{\Theta})$ gives an estimate for the standard error of $\hat{\Theta}$, denoted by $SE_b(\Theta)$. If K is large enough, $V_b(\hat{\Theta})$ will approximate the true variance of the estimator.

5. We can create a density plot of $\hat{\Theta}^{(1)}, \cdots, \hat{\Theta}^{(K)}$ to estimate the sampling distribution of $\hat{\Theta}$. If K is large enough, this density plot will approximate the true PDF f_X well.

Comparing this algorithm with Monte Carlo simulations, the main differences are:

- With the bootstrap, we do not assume any distribution for the original data. With Monte Carlo simulations, we do make an assumption on the distribution for the original data.

- The K datasets are produced slightly differently. For the bootstrap, they are synthesized by sampling with replacement n data points from the original dataset of size n. With Monte Carlo simulations, we draw n data points from its assumed distribution.

The math that proves the bootstrap works is beyond the level of this class, although it is freely available online.

The bootstrap tends to work better as the number of bootstrap datasets ,K, increases and as the sample size, n, increases.

13.2.3 Confidence Intervals using the Bootstrap

The **basic** (or reverse percentile) bootstrap $(1-\alpha)\times 100\%$ confidence interval for θ is

$$\left(\hat{\theta}-\hat{\delta}^*_{1-\alpha/2}, \hat{\theta}-\hat{\delta}^*_{\alpha/2}\right), \tag{13.1}$$

where $\delta^* = (\hat{\Theta}^{(1)}, \cdots, \hat{\Theta}^{(K)}) - \hat{\theta}$, which is the difference between the estimator in the K bootstrap samples and the value of the estimator in the original dataset, so $\hat{\delta}^*_\beta$ is the $\beta \times 100$th percentile of δ^*.

You may notice that the confidence interval in equation (13.1) does not fit the general framework given in equation (12.2). This is because the sampling distribution of the estimator may not be symmetric.

The math that derives this confidence interval is beyond the level of this class, although it is freely available online.

The bootstrap confidence interval tends to work better as K increases and as n increases.

Note: there are variations of the confidence interval based on the bootstrap. We will use the basic bootstrap confidence interval as specified in equation (13.1).

13.2.4 Worked Example

We have data on 400 credit card customers. A density plot of their credit limits is shown below:

```
library(ISLR2)
Data<-ISLR2::Credit$Limit
plot(density(Data), xlab="Credit Limit", main="Density Plot of Credit Limits")
```

The density plot in Figure 13.1 informs us that the values of credit limits are skewed, so the median is a better measure of centrality for credit limits. We wish to estimate the population median credit limit, give a measure of uncertainty, as well as report a 95% confidence interval for the median credit limit. Since the sampling distribution of the sample median is unknown, we use the bootstrap:

```
##actual sample median
samp_median <- median(Data)
samp_median
```

```
## [1] 4622.5
```

```
K<-1000 ##Number of bootstrap samples
set.seed(77)

##the sample function, with replace=TRUE, samples with replacement.
```

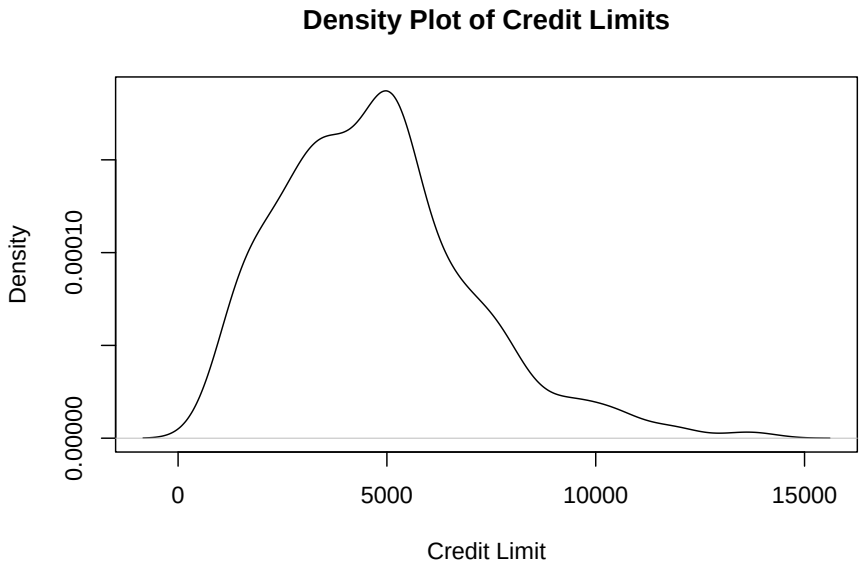


Figure 13.1: Density Plot of Credit Limits

```
##The replicate function repeats the sample function K times
##if no size is specified in sample(), default is the length of the vector called Data
boot.samples<-replicate(K,sample(Data, replace=TRUE))
##Each column in boot.samples represents each bootstrap sample

##find the sample median of each column, since each column represents a bootstrap sample
boot.medians<-apply(boot.samples,2,median)


##bootstrap SE of the median is just the SD of the sample medians from each bootstrap sample
sd(boot.medians)
```

```
## [1] 152.0525
```

```
##deltas
boot_median_error <- boot.medians- samp_median

##sort deltas
boot_median_error_sort <- sort(boot_median_error)

##find CI based on equation
alpha<-0.05
```

```
p1 <- K*(alpha/2)
p2 <- K*(1 - alpha/2)

samp_median - boot_median_error_sort[c(p2,p1)]
```

```
## [1] 4380.0 4949.5
```

We report the following:

- The sample median is $4622.5.
- The standard error of the sample median is $152.0525.
- The 95% confidence interval for the median credit limit is (4380.000, 4949.5). There is 95% probability that the true population median credit limit lies in the random interval between $4380 and $4949.50.

13.3 Parametric Vs Non-Parametric Methods

The confidence intervals from Section 12 are based on the known sampling distributions of estimators, and have a closed form formula to calculate them. These confidence intervals are typically based on some conditions in order to be valid.

Such methods are sometimes called **parametric methods**, as the distributions they are based on fits a known probability distribution which can be specified by an equation via its PDF, f_X.

The bootstrap, on the other hand, does not assume a known probability distribution. Therefore, the bootstrap is considered a **non-parametric method**, as the associated probability distribution is unknown and not associated with an equation like a PDF. Non-parametric methods tend to be more versatile, in the sense that they are based on less assumptions or conditions, and so can be applied in a wider variety of settings.

The only assumptions / conditions associated with the bootstrap are:

- The sample has to be representative of the population (similar to parametric methods).
- The underlying population does not have very heavy tails.

Chapter 14

Hypothesis Testing for Means

14.1 Introduction

Consider this scenario: you are conducting clinical trials to assess whether a new vaccine is more effective than existing vaccines. Suppose the efficacy of existing vaccines on a certain disease is 80%, and you conduct clinical trials on 30 patients, and the new vaccine is effective on 25 patients. How strong is this result in trying to prove that the new vaccine is more effective than existing vaccines? Questions like these lead to hypothesis testing.

In previous modules, we have established the fact that estimators are likely to deviate from the value from their corresponding parameter, due to random sampling. Confidence intervals allow us to provide a measure of uncertainty over our estimator as well as a range of plausible values for the parameter, using concepts regarding the sampling distribution of estimators. These concepts can also be used to assess whether our observed value for the estimator is different enough from a potential value of the parameter that random sampling alone is unlikely to be the only reason for the difference. Once this assessment is done, we then want to make a conclusion and decision about the unknown parameter. This is the big picture idea behind hypothesis testing. **Hypothesis testing** is a method for making a systematic decision with statistical guarantees.

Hypothesis testing typically has the following steps:

1. State a hypothesis, based on the research question.
2. Assume the hypothesis is true, and then compute a metric that measures how much our observed data deviates from the assumed hypothesis.
3. Compare the metric with some sort of threshold to assess if our observed data deviates "far enough" from the assumed hypothesis, and then make

a decision.

For ease of exposition, we will introduce these ideas in the framework of the hypothesis test for the mean. We then show how the framework applies in two other situations: hypothesis test for paired means and hypothesis test for two independent means.

A hypothesis is a statement which we assess whether is true or not using our observed data. In the framework of hypothesis testing, we state two competing hypotheses:

- The first hypothesis is the **null hypothesis**. This is normally the "status quo".
- The other hypothesis is the **alternative hypothesis**. This can be viewed as a statement that is "opposite" of the null hypothesis.

Many authors present these hypotheses and hypothesis testing using a court trial as an analogy. Defendants are assumed to be "innocent until proven guilty". The "null hypothesis" or status quo in this setting is that the defendant is innocent, and the "alternative hypothesis" is that the defendant is guilty. The prosecutor needs to then present evidence that contradicts the null hypothesis in order to prove guilt. The jurors, who come into the trial assuming the defendant is innocent, then assess how strong is the evidence presented: if its strength is beyond a certain threshold (i.e. beyond "reasonable doubt") then there is evidence to reject the null hypothesis and decide the defendant is guilty. However, if the evidence is not strong enough, there is not enough evidence to reject the null hypothesis and the jury renders a not guilty verdict.

14.2 Hypothesis Test for the Mean

When testing for the mean, the parameter is the population mean. This happens when the variable we are measuring is quantitative. Consider this example:

The term "Freshman 15" is an expression which says that college students gain 15 pounds (on average), in their first year in college. Researchers claim that with better education regarding healthier lifestyle habits, this gain is less than 15 pounds, on average.

We will conduct a hypothesis test for the mean. Some authors call this a 1-sample t test for the mean, since it involves only 1 sample (college students) and is based on the t distribution.

Next, we cover the first step in hypothesis testing: how to write the null and alternative hypothesis statements.

14.2.1 Null and Alternative Hypotheses

Null and alternative hypotheses are statements regarding the value of a parameter.

For the Freshman 15 example, the null hypothesis is that the average weight gain for first year college students is 15 pounds. This is denoted as $H_0 : \mu = 15$, where H_0 denotes the null hypothesis. The alternative hypothesis is that the average weight gain for first year college students is less than 15 pounds. This is denoted as $H_a : \mu < 15$, where H_a denotes the alternative hypothesis.

14.2.1.1 Features of Null and Alternative Hypotheses

- Null and alternative hypotheses are always about a population parameter, not a sample estimator. The reason is that we want to make a claim about the population, based on data in our sample. Notice in the Freshman 15 example, that our null and alternative hypotheses are about the population mean μ and not the sample mean $\bar{x}$.
- The **null hypothesis** is a statement that says the parameter is equal to some specific value. Let μ_0 denote this specific value. So we write $H_0 : \mu = \mu_0$. In the Freshman 15 example, $\mu_0 = 15$.
- The **alternative hypothesis** is a statement against the null hypothesis. For the hypothesis test for the mean, there are a few possible alternative hypotheses. Which one we use is **driven by the research question**. The alternative hypothesis could say the parameter is:
  - Different from some specific value, i.e. $H_a : \mu \neq \mu_0$. This is called a **two-sided alternative** since the parameter could be greater or less than some specific value.
  - Greater than some specific value, i.e. $H_a : \mu > \mu_0$. This is called a **one-sided alternative** since the parameter is greater than some specific value.
  - Less than some specific value, i.e. $H_a : \mu < \mu_0$. This is also called a one-sided alternative since the parameter is less than some specific value.

So for the hypothesis test for the mean, there are 3 options for the alternative hypothesis.

In the Freshman 15 example, researchers claim that with better education, the average weight gain is less than 15 pounds, so that is why we write $H_a : \mu < 15$.

14.2.1.2 Some Comments about the Null Hypothesis with a One-sided Alternative

A number of textbooks take the view that the null and alternative hypotheses are complements, so if we write $H_a : \mu < \mu_0$, then we must write $H_0 : \mu \geq \mu_0$.

This way of writing a null hypothesis is different from that I wrote earlier: that the null hypothesis says the parameter is equal to μ_0. There are a couple of reasons why I say the null hypothesis is a statement involving an equality, rather than an inequality:

1. The calculations performed to assess the evidence our data provides are done assuming the null hypothesis is true. You will realize in the later subsections that one can only perform these calculations if the null hypothesis says the parameter is equal to some specific value.

2. I have seen many people get confused with the null and alternative hypotheses if they insist the null and alternative must be opposite with a one-sided alternative. Indeed, if you look at Practice Exercise 9.3 and 9.4 in our book, the author has gotten these confused in the proposed solutions.

14.2.2 Test Statistic

After writing the null and alternative hypotheses, the next step is to compute a value that measures how far our sample data are from their expected value if the null hypothesis is true. This value is called the **test statistic**. How a test statistic is calculated is based on the sampling distribution of the estimator being used.

We remind ourselves of the sampling distribution of the sample mean, $\bar{X}_n$, from Section 11.3. There are a couple of conditions to consider:

1. $X_1, \cdots, X_n$ are i.i.d. from a normal distribution with finite mean μ and finite variance σ^2. Then $\bar{X}_n \sim N(\mu, \frac{\sigma^2}{n})$.

2. $X_1, \cdots, X_n$ are i.i.d. from any distribution with finite mean μ and finite variance σ^2, and if n is large enough, then $\bar{X}_n$ is approximately $N(\mu, \frac{\sigma^2}{n})$.

If either of these conditions are met, then the distribution of $\bar{X}_n$ after standardization is either a standard normal or approaches a standard normal distribution, so $\frac{\bar{X}_n - \mu}{\frac{\sigma}{\sqrt{n}}} = \frac{\bar{X}_n - \mu}{\sqrt{Var(\bar{X})}} = \frac{\bar{X}_n - \mu}{SE(\bar{X}_n)}$ is either standard normal or approximately standard normal when n is large enough.

We then learned in Section 12.2.4 that since the value of the population variance σ^2 is almost always unknown in real life, we use the sample variance s^2 instead. So we work with $\frac{\bar{X}_n - \mu}{\frac{s}{\sqrt{n}}}$ instead, which follows a t distribution with $n-1$ degrees of freedom.

Based on this distribution, the test statistic for a hypothesis test for the mean is

$$\hat{t} = \frac{\bar{x}_n - \mu_0}{\frac{s}{\sqrt{n}}} = \frac{\bar{x}_n - \mu_0}{SE(\bar{x}_n)}, \tag{14.1}$$

where $SE(\bar{x}_n)$ is $\frac{s}{\sqrt{n}}$. This can be called a t **statistic** to reflect that our test statistic is based on the t distribution. The t statistic follows a t_{n-1} distribution (the subscript refers to the degrees of freedom) if either of the 2 conditions are met for the sampling distribution of $\bar{X}_n$ to be valid, and if we assume the null hypothesis is true, that the true mean of the data is μ_0.

Note: A number of hypothesis tests involving other estimators and parameters are also based on the t_{df} distribution, and have their own t statistics. However, t statistics will take on the general form

$$\hat{t} = \frac{\text{estimated value } - \text{ value in } H_0}{\text{standard error of estimator}}. \tag{14.2}$$

Notice how equation (14.2) is a generalization of equation (14.1).

Note: A variation of this test is the 1-sample z-test for the mean. This assumes the population standard deviation is known, but that is unlikely in reality. The sample standard deviation in equation (14.1) is replaced by the population standard deviation and the test statistic becomes a Z statistic as it is compared with a standard normal distribution.

We go back to the Freshman 15 example. Suppose researchers collect a random sample of 50 first-year college students. Their average weight gain is 14 pounds, with standard deviation of 3 pounds. Derive the value of the t statistic.

Using equation (14.1), the t statistic is:

$$\begin{aligned}\hat{t} &= \frac{14-15}{\frac{3}{\sqrt{50}}} \\ &= -2.357023.\end{aligned}$$

This t statistic is valid since our sample size is 50, which is usually large enough for the approximation to the t distribution to be valid.

Remember that the test statistic is a measure of how far our sample data are from their expected value if the null hypothesis is true. Larger values of the test statistic (in magnitude) imply that our data deviate further from the null hypothesis, i.e. our data provide more evidence against the null hypothesis.

The t statistic also has this nice definition: it is the distance between our sample estimator and the value of the parameter if the null hypothesis is true, in terms of number of standard errors. So for our Freshman 15 example, the sample mean is 2.357 standard errors smaller than the hypothesized value of 15.

14.2.2.1 Factors Affecting t Statistic for Mean

We have established that larger values of the test statistic (in magnitude) imply more evidence against the null hypothesis. For the t statistic for the mean, the

factors that affect its magnitude are:

- The difference between the sample mean $\bar{x}$ of our data and the value of the population mean if the null is true μ_0. This difference is called the **effect size**. Larger effect sizes lead to larger test statistics, in magnitude. This should make sense since a larger effect size implies the sample mean deviates more from the expected mean if the null hypothesis is true.
- The sample size n. Larger sample sizes lead to larger magnitudes for the t statistic, since $\frac{\bar{x}_n - \mu_0}{\frac{s}{\sqrt{n}}}$ can be written as $\sqrt{n}\frac{\bar{x}_n - \mu_0}{s}$. This implies that even if the effect size stays the same, larger sample sizes provide more evidence against the null.

14.2.3 Making a Decision

After calculating the value of the test statistic, we need to assess if our data provide enough evidence against the null hypothesis or not. There are two approaches to making this assessment: one involves the p-value, and the other involves the critical value.

- Using the p-value, we **reject the null hypothesis if our p-value is less than the significance level** (i.e. our data have enough evidence against the null hypothesis). Otherwise, we fail to reject the null hypothesis (i.e. our data do not have enough evidence against the null hypothesis)

The **significance level** of the test, denoted by α, is the probability of wrongly rejecting the null hypothesis when the null hypothesis is actually true. This error is called a type I error, which we will define more formally in a later subsection. So if we conduct a hypothesis test with significance level $\alpha = 0.05$, we are saying that we are willing to have a 5% probability of wrongly concluding that we have enough evidence against the null hypothesis, when it is actually true.

- Using the critical value, we **reject the null hypothesis if our test statistic is more extreme than the critical value in the direction of the alternative hypothesis** (i.e. our data have enough evidence against the null hypothesis). Otherwise, we fail to reject the null hypothesis (i.e. our data do not have enough evidence against the null hypothesis).

Both of these approaches are based on the sampling distribution of the estimator, and on the value of the significance level. Both of these approaches will always lead to the same conclusion, so you only need to use one of them.

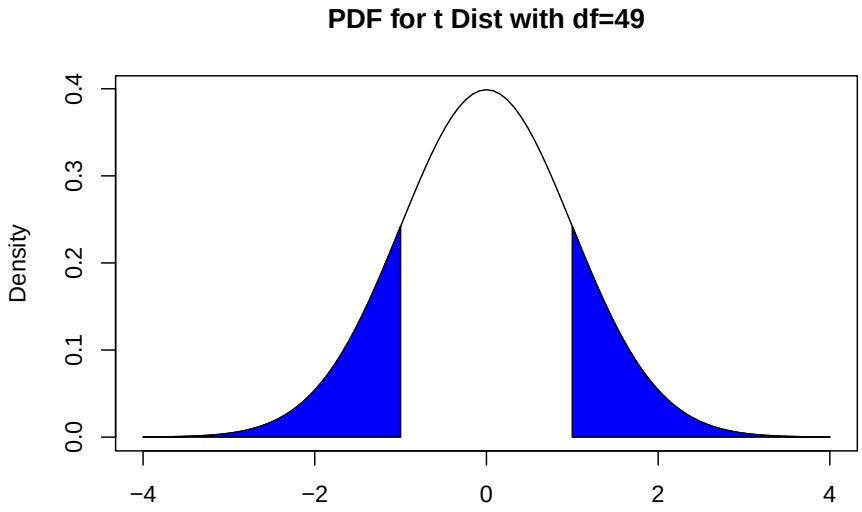
14.2.3.1 P-Value

The **p-value** is the probability of observing the value of our test statistic, or a value more extreme in the direction of the alternative hypothesis, if the null hypothesis is true.

Informally, this is also the probability of observing the value of our sample mean, or a value more extreme in the direction of the alternative hypothesis, if the null hypothesis is true.

Recall that for a hypothesis test of the mean, the t statistic follows a t_{n-1} distribution, as our calculations assume the null hypothesis is true. Visually, we can use areas under the PDF of a t_{n-1} to illustrate how the p-value is found. The specific area under the PDF depends on which of the three alternative hypotheses we are using. In Figures **??**, 14.1, and 14.2 below, we have examples on the relevant areas under the PDF if the value of the t statistic is $\hat{t} = 1$.

1. If $H_a : \mu \neq \mu_0$, i.e. we have a two-sided alternative, the corresponding areas under the PDF of the t_{n-1} distribution that is the p-value is shown below in Figure **??**:



It is the area to the right of $|\hat{t}|$ and the area to the left of -$|\hat{t}|$. Using probability notation, this is $P(|t_{n-1}| \geq 1)$.

2. If $H_a : \mu > \mu_0$, i.e. the population mean is greater than a specified value, the corresponding area under the PDF of the t_{n-1} distribution that is the p-value is shown below in Figure 14.1:

It is the area to the right of $\hat{t}$. Using probability notation, this is $P(t_{n-1} \geq 1)$.

3. If $H_a : \mu < \mu_0$, i.e. the population mean is less than a specified value, the corresponding area under the PDF of the t_{n-1} distribution that is the p-value is shown below in Figure 14.2:

It is the area to the left of $\hat{t}$. Using probability notation, this is $P(t_{n-1} \leq 1)$.

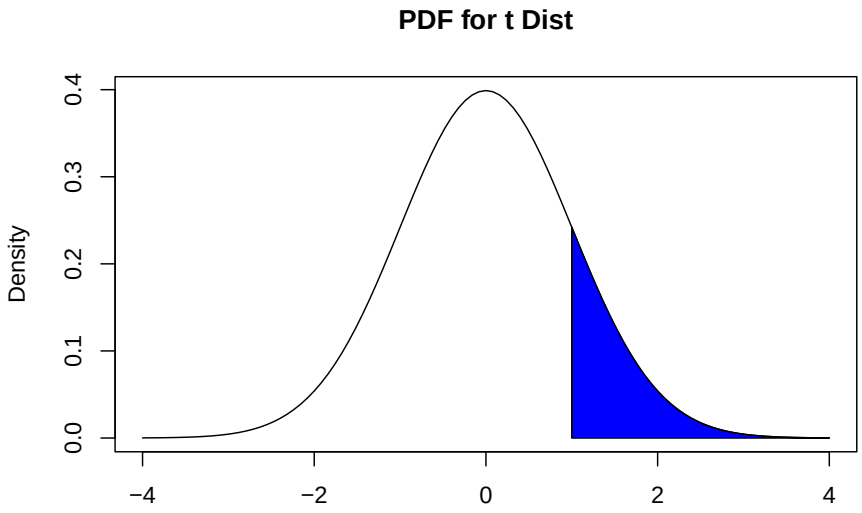


Figure 14.1: Finding P-value, with Greater Than Alternative

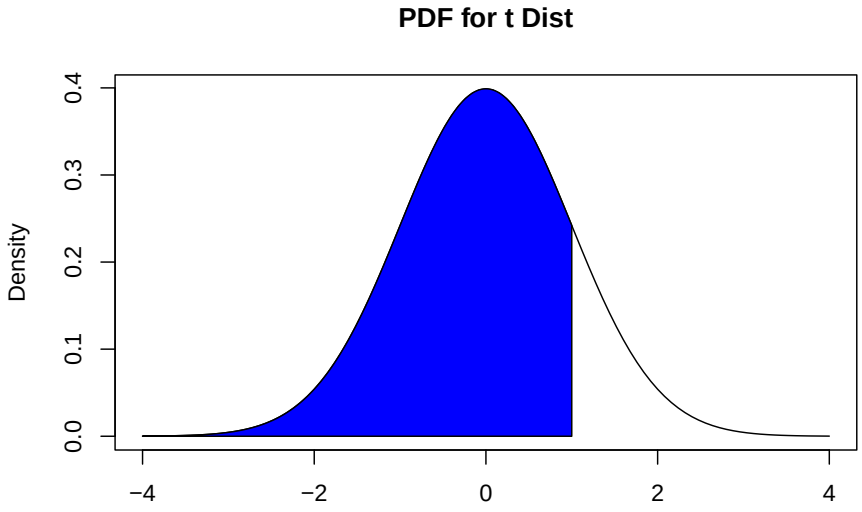


Figure 14.2: Finding P-value, with Less Than Alternative

Going back to the Freshman 15 example, we found the t statistic to be -2.357023. We had earlier written the alternative hypothesis as $H_a : \mu < 15$, so we find the PDF under a t_{49} distribution using the area to the left of -2.357023, in a manner similar to Figure 14.2.

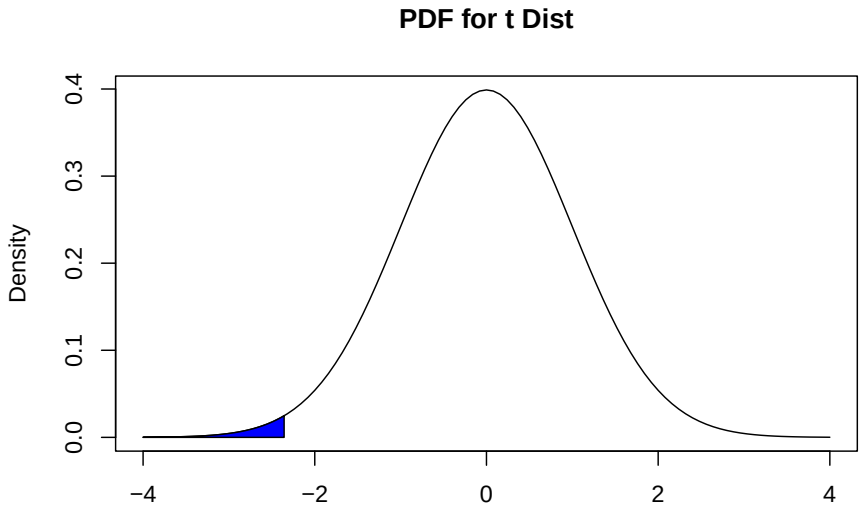


Figure 14.3: Finding P-value, For Freshman 15 Example

Using R, this p-value is 0.01123. If our test is conducted at 5% significance level, we reject the null hypothesis, since this p-value is less than 0.05.

```
pt(-2.357023, 49) ##enter t stat, then df
```

```
## [1] 0.01123111
```

Thought question: Suppose the research question for the Freshman 15 example was tweaked, so the alternative hypothesis is $H_a : \mu > 15$. Assuming everything else stays the same, show that the p-value is now 0.98877. If the alternative hypothesis is $H_a : \mu \neq 15$, show that the p-value is 0.02246.

14.2.3.2 Critical Value

Informally, the critical value is the value of the test statistic that is considered "enough" to say that we have enough evidence against the null hypothesis to reject it, based on the value of the significance level.

We use the PDF of a t distribution to visualize what a critical value is. We want to find the "cut off" value, denoted by t^*_{n-1}, of the distribution where the

area under its PDF in the direction of the alternative hypothesis is equal to the significance level α. Since we have three options for the alternative hypothesis, we have 3 different areas under the PDF to consider. In Figures 14.4, 14.5, and 14.6 below, we have examples on the relevant areas under the PDF if $\alpha = 0.05$ and $n = 50$:

1. If $H_a : \mu \neq \mu_0$, i.e. we have a two-sided alternative, we want the areas under the PDF of the t_{49} distribution to the right of t^*_{49} and to the left of $-t^*_{49}$ to be equal to 0.05. This is shown in Figure 14.4:

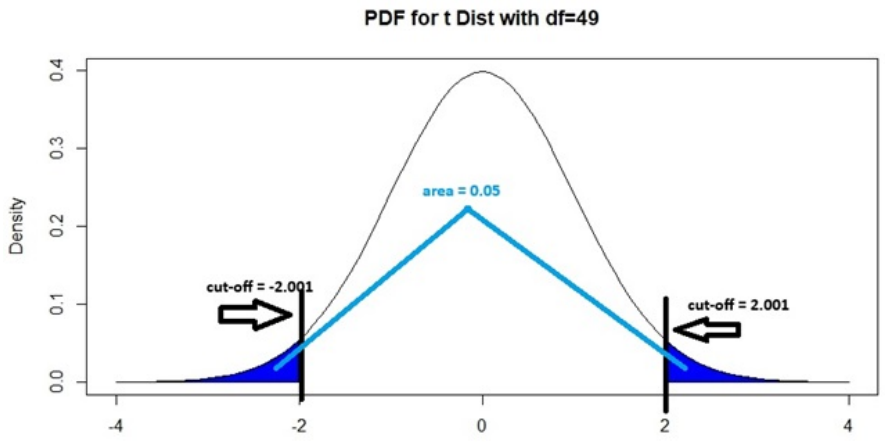


Figure 14.4: Critical Value for 2-Sided Alternative, with 0.05 Sig Level

Note that since the t distribution is symmetric and both areas in blue add up to 0.05, each shaded area must be 0.025. So $-t^*_{49}$ and t^*_{49} correspond to the 2.5th and 97.5th percentiles of a t_{49} distribution respectively. We can show that $-t^*_{49}$ is -2.009575 by typing

```
alpha<-0.05
n<-50
qt(alpha/2, n-1)
```

```
## [1] -2.009575
```

and by symmetry of the t distribution, t^*_{49} is 2.009575. The critical value is therefore $t^*_{49} = 2.009575$ for a 2-sided test. We will reject the null hypothesis if our t statistic is to the right of 2.009575 or if it is to the left of -2.009575.

In general, for any significance level α, the negative critical value $-t^*_{n-1}$ corresponds to the $(\alpha/2) \times 100$th percentile, and the positive critical value t^*_{n-1} corresponds to the $(1 - \alpha/2) \times 100$th percentile of the appropriate t_{n-1} distribution, if $H_a : \mu \neq \mu_0$.

2. If $H_a : \mu > \mu_0$, i.e. the population mean is greater than some specified value, we want the area under the PDF of the t_{49} distribution to the right of t^*_{49} to be equal to 0.05. This is shown in Figure 14.5:

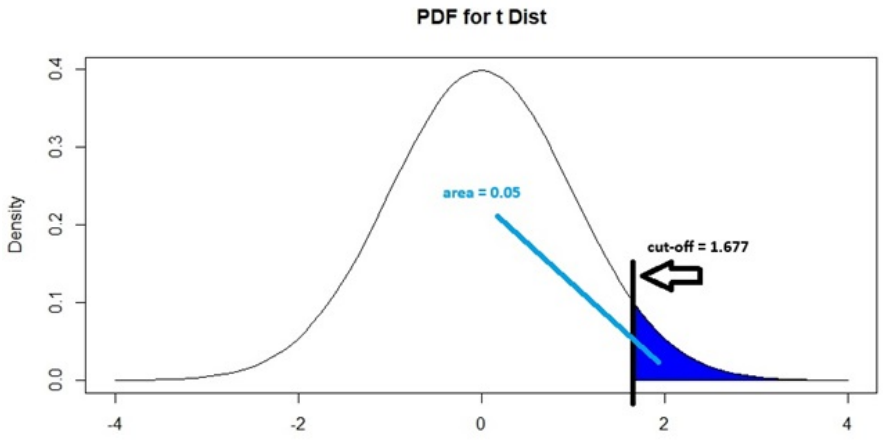


Figure 14.5: Critical Value for Greater Than Alternative, with 0.05 Sig Level

Note that t^*_{49} corresponds to the 95th percentile of a t_{49} distribution. We can show that t^*_{49} is 1.677 by typing

```
alpha<-0.05
n<-50
qt(1-alpha, n-1)
```

```
## [1] 1.676551
```

The critical value is therefore $t^*_{49} = 1.676551$ when $H_a : \mu > \mu_0$. We will reject the null hypothesis if our t statistic is to the right of 1.676551.

In general, for any significance level α, the critical value t^*_{n-1} corresponds to the $(1-\alpha) \times 100$th percentile of the appropriate t_{n-1} distribution, if $H_a : \mu > \mu_0$.

3. If $H_a : \mu < \mu_0$, i.e. the population mean is less than some specified value, we want the area under the PDF of the t_{49} distribution to the left of t^*_{49} to be equal to 0.05. This is shown in Figure 14.6:

Note that t^*_{49} corresponds to the 5th percentile of a t_{49} distribution. We can show that t^*_{49} is -1.677 by typing

```
alpha<-0.05
n<-50
qt(alpha, n-1)
```

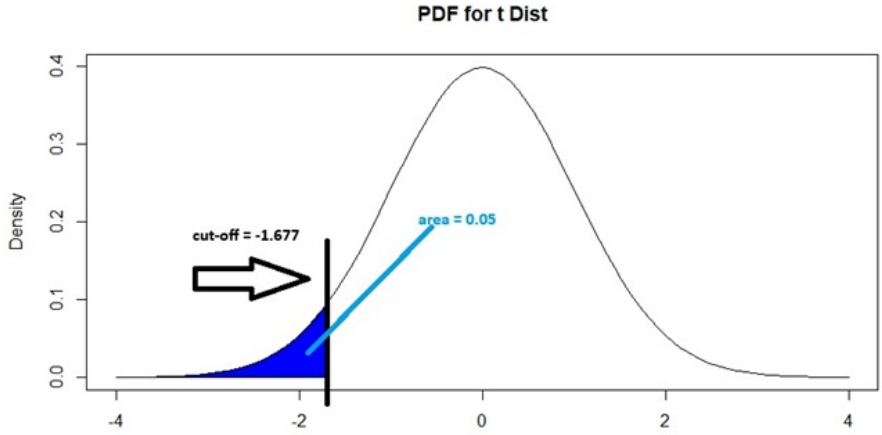


Figure 14.6: Critical Value for Less Than Alternative, with 0.05 Sig Level

```
## [1] -1.676551
```

The critical value is therefore $t^*_{49} = -1.676551$ when $H_a : \mu < \mu_0$. We will reject the null hypothesis if our t statistic is to the left of -1.676551.

In general, for any significance level α, the critical value t^*_{n-1} corresponds to the $(\alpha) \times 100$th percentile of the appropriate t_{n-1} distribution, if $H_a : \mu < \mu_0$.

Going back to the Freshman 15 example, we found the t statistic to be -2.357023. We had earlier written the alternative hypothesis as $H_a : \mu < 15$, so the critical value is the 5th percentile of a t_{49} distribution, which we found to be -1.676551. Since our t statistic is to the left of this critical value, we reject the null hypothesis.

Thought question: Consider a hypothesis test for the mean with $n = 10$. Assume the conditions for the sampling distribution of the sample mean to be met. Suppose $\alpha = 0.10$. If the alternative hypothesis is 2-sided, show that the critical value is 1.833113. If the alternative hypothesis is 1-sided, show that the critical value is 1.383029.

14.2.4 Writing Conclusions

In the previous subsection, we decide whether to reject the null hypothesis based on two approaches:

- If the p-value of our test is less than the significance level.
- If the test statistic of our test is more extreme than the critical value, in the direction of the alternative hypothesis.

The decision based on both approaches should always be the same and never contradict each other. All you need is to use one of these approaches, the other is redundant. It is up to you to choose which one you want to use, but you must be clear which one you are using.

We decide between:

1. Rejecting the null hypothesis. This implies that we have enough evidence against the null hypothesis to reject it and support the alternative hypothesis. Our data support the claim in the alternative hypothesis. Using the court trial analogy, this is equivalent to saying we have enough evidence to prove a defendants guilt as we reject the assumption that the defendant is innocent, and give a guilty verdict.

2. Failing to reject the null hypothesis. This implies that we do not have enough evidence against the null hypothesis and so we do not reject it. Our data do not support the claim in the alternative hypothesis. Using the court trial analogy, this is equivalent to saying we do not have enough evidence to prove a defendants and so give a not guilty verdict.

A common mistake with failing to reject the null hypothesis is to say that our data support and proves the null hypothesis is true. Using the court trial analogy, a not guilty verdict is not the same as saying the defendant is innocent.

After making a decision, we write a conclusion in the context of our research question. Using the Freshman 15 example, we rejected the null hypothesis. So our conclusion is our data support the claim that the average weight gain among first-year college students is less than 15 pounds.

We look at one more example to show how a hypothesis test is done from start to finish.

14.2.5 Worked Example

A university bookstore claims that, on average, a student will pay $101.75 per class for textbooks. The student newspaper suspects that the claimed average is too low and investigates by randomly selecting 9 courses from the current semester offerings and finding the textbook costs for each. The students conducting the investigation reason that with the collection of courses being offered courses, the textbook prices are normally distributed.

We have one quantitative variable, the amount of money students will pay per class for textbooks. A t-test for a mean is appropriate since textbook prices are normally distributed.

From the question, we have $H_0 : \mu = 101.75, H_a : \mu > 101.75$.

```
## Record sample data collected
news.samp <- c(140, 125, 150, 87, 170, 195, 94, 127, 53)
```

```
## Conduct hypothesis test
t.test(news.samp, mu=101.75, alternative="greater")
```

```
##
##  One Sample t-test
##
## data:  news.samp
## t = 1.7147, df = 8, p-value = 0.06237
## alternative hypothesis: true mean is greater than 101.75
## 95 percent confidence interval:
##  99.63542      Inf
## sample estimates:
## mean of x
##  126.7778
```

```
text.test <- t.test(news.samp, mu=101.75, alternative="greater")
text.test$statistic ##t stat
```

```
##        t
## 1.714676
```

```
text.test$parameter ##degrees of freedom
```

```
## df
##  8
```

```
text.test$p.value ##p-value
```

```
## [1] 0.06237429
```

```
text.test$estimate ##value of estimator (sample mean here)
```

```
## mean of x
##  126.7778
```

- The test statistic is $t = $ 1.7146763.
- The p-value is 0.0623743.

So we reject the null hypothesis. Our data support the student newspaper' claim that the claimed average by the bookstore is too low.

14.3 Hypothesis Test for Paired Means

The hypothesis test for paired means (also called the paired t-test or the hypothesis test for two dependent means) is a special case of the hypothesis test for the mean where instead of testing the mean of one population, the difference of two means is tested by using the differences in pairs of data points that are dependent. As this is a special case of the one-sample t-test, the same assumptions are required: the differences of the paired values are normally distributed,

or if the number of paired values is large enough.

The test statistic is

$$\hat{t} = \frac{\bar{d}_n - \mu_0}{\frac{s_d}{\sqrt{n_d}}} = \frac{\bar{d}_n - \mu_0}{SE(\bar{d}_n)}, \tag{14.3}$$

where $\bar{d}_n$ denotes the sample average difference between the pairs, s_d denotes the standard deviation of the differences between the pairs, n_d denotes the number of pairs, and μ_0 denotes the mean difference under the null hypothesis. The test statistic follows a t_{n_d-1} distribution.

Thought question: compare equation (14.3) with equations (14.1) and (14.2). Do you see their similarities?

In order to conduct a paired t-test, the optional `t.test()` argument `paired` should be designated as `TRUE`. When conducting a paired t-test, the data are given in two vectors (the second will be subtracted from the first position-by-position).

14.3.1 Worked Example

The `anorexia` dataset in the `MASS` package provides data on patient weights before and after receiving different forms of therapy for anorexia. We will look at the control group for this example, and we want to see if there is a change in weights before and after the study period.

From the question, we have $H_0 : \mu_d = 0, H_a : \mu_d \neq 0$.

```
library(MASS)

paired.Data <- MASS::anorexia

##only control group
control <- paired.Data[paired.Data$Treat == "Cont",]

pair_result <- t.test(control$Prewt, control$Postwt, mu=0, alternative="two.sided", paired=TRUE)

pair_result
```

```
##
##  Paired t-test
##
## data:  control$Prewt and control$Postwt
## t = 0.28723, df = 25, p-value = 0.7763
## alternative hypothesis: true mean difference is not equal to 0
## 95 percent confidence interval:
##  -2.776708  3.676708
```

```
## sample estimates:
## mean difference
##            0.45
```

- The test statistic is $t = $ 0.2872254.
- The p-value is 0.7763071. So we fail to reject the null hypothesis. The data do not provide evidence that the weights of the patients in the control group change before and after the study period.
- The 95% confidence interval is -2.7767076, 3.6767076. Notice how this interval contains the value under the null hypothesis, 0. The conclusion is consistent with the hypothesis test.

We could have also used the `t.test()` function a little differently. We could have found the differences in weights between each pair of readings for each patient, and then performed a 1-sample t-test on these differences.

```
## do as a 1 sample t test on pairs of differences
diffs <- control$Prewt - control$Postwt

other_result <- t.test(diffs, mu= 0, alternative = "two.sided", paired = FALSE)

other_result
```

```
##
##  One Sample t-test
##
## data:  diffs
## t = 0.28723, df = 25, p-value = 0.7763
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  -2.776708  3.676708
## sample estimates:
## mean of x
##      0.45
```

Notice the results are identical in both approaches.

14.4 Hypothesis Test for Difference in Two Independent Means

The hypothesis test for difference in two independent means (also called a 2-sample t-test) compares the average of a quantitative variable across two independent populations (created by a binary variable). The condition for this test is that both populations are normally distributed, or that the sample sizes are both large enough.

The test statistic is

$$\hat{t} = \frac{(\bar{x} - \bar{y}) - (\mu_X - \mu_Y)}{\sqrt{\frac{s_X^2}{n_X} + \frac{s_Y^2}{n_Y}}} = \frac{(\bar{x} - \bar{y}) - (\mu_X - \mu_Y)}{SE(\bar{X} - \bar{Y})}, \tag{14.4}$$

where $\bar{x}, \bar{y}$ denote the sample averages in each sample, s_X, s_Y denote the standard deviation in each sample, n_X, n_Y denote the sample sizes in in each sample, and $\mu_X - \mu_Y$ denotes the mean difference under the null hypothesis. The test statistic follows a t distribution. The degrees of freedom is approximated using the Welch–Satterthwaite equation.

In some older references, it is suggested to approximate the degrees of freedom using $min(n_X, n_Y) - 1$. The hypothesis test using this approximation is considered to be more **conservative** than the approximation based on the Welch–Satterthwaite equation. A hypothesis test that is conservative is a test when the Type I error rate (probability of wrongly rejecting the null) is below the significance level, and is less powerful. We will cover these concepts in the next section, Section 15.

Thought question: what are the factors that lead to stronger evidence against the null hypothesis for this test?

Similar to what we wrote in Section 12.4.1, the test statistic for the difference in two independent means given in equation (14.4) is the modern default. It is sometimes called a **Welch's** t test, or an **unpooled version** of test.

A **pooled version** of this test exists. It has an additional assumption: that the variances of the quantitative variable is equal in the two populations (i.e. $\sigma_X^2 = \sigma_Y^2$). The pooled version has fallen out of favor especially when the variances are unequal and when the sample sizes are unequal, as the type I error rates do not match the significance level.

14.4.1 Worked Example

We want to assess if there is a significant difference in the average body masses of male and female Gentoo penguins.

From the question, we have $H_0 : \mu_f - \mu_m = 0, H_a : \mu_f - \mu_m \neq 0$.

```
library(palmerpenguins)

Data <- palmerpenguins::penguins

result_2samp <- t.test(body_mass_g ~ sex, data = Data,
       subset = species %in% c("Gentoo"))

result_2samp
```

```
##
##  Welch Two Sample t-test
```

```
##
## data:  body_mass_g by sex
## t = -14.761, df = 116.64, p-value < 2.2e-16
## alternative hypothesis: true difference in means between group female and group male
## 95 percent confidence interval:
##  -913.1130 -697.0763
## sample estimates:
## mean in group female   mean in group male
##             4679.741             5484.836
```

- The test statistic is $t = $ -14.7613684.
- The p-value is virtually 0. So we reject the null hypothesis. The data provide evidence that the average weights of female and male Gentoo penguins are significantly different
- The 95% confidence interval is -913.1130428, -697.0763297. Notice how this interval does not contain the value under the null hypothesis, 0. In fact, since the entire interval is below 0, we can also say that the male Gentoo penguins are heavier, on average, than their female counterparts. The conclusion is consistent with the hypothesis test.

14.5 Additional Comments on Hypothesis Test of Parameters

14.5.1 Two-Sided Tests and Confidence Intervals

Notice how hypothesis tests and confidence intervals are based on the sampling distribution of the relevant estimator? If you look at how the critical value of a two-sided test and the critical value of a confidence interval are derived, you will notice that they are found in the exact same manner. Refer to Figure 14.4 and Figure 12.2, and notice how these pictures look exactly the same, with the middle portion of the PDF having an area of $1-\alpha$ and the two tail ends having an area of α.

What this implies is that conclusions from a $(1-\alpha)\times 100\%$ confidence interval will be consistent with a 2-sided hypothesis test conducted at significance α. Also notice how we use the same notation α for the confidence level and significance level?

- If the null hypothesis is **rejected** for a 2-sided test, than the value of the parameter under the null hypothesis will lie **outside** the corresponding confidence interval. This should make sense since the confidence interval provides a range of plausible values for the parameter and values outside the interval are considered to be "ruled out".
- If the null hypothesis is **not rejected** for a 2-sided test, than the value of the parameter under the null hypothesis will lie **inside** the corresponding confidence interval. This should make sense since the confidence interval

provides a range of plausible values for the parameter and values outside the interval are considered to be "ruled out".

We noted this in the worked examples in Sections 14.3.1 and 14.4.1.

14.5.2 Reporting Conclusions

You may have noticed that we make a binary decision at the end of a hypothesis test: we either reject or fail to reject a null hypothesis, by comparing our test statistic with a critical value or by comparing a p-value (which depends on the value of the test statistic) with a significance level.

While the decision is binary, note that the numerical value of a test statistic is not binary. We have mentioned that larger values (in magnitude) imply more evidence against the null hypothesis. Are p-values of 0.049 and 0.051 really that much different? So the p-value also can be interpreted in the same manner: smaller values imply more evidence against the null hypothesis.

Therefore, when writing conclusions, it is recommended to also provide the p-value or the value of the test statistic, instead of just reporting that the "null hypothesis is rejected" or "the null hypothesis is not rejected". Readers than then make an assessment of how strong the evidence is against the null hypothesis, based on their judgement of what the value of the significance level should be.

Chapter 15

Type I and II Errors, Significance Level, Power

15.1 Introduction

Section 14 introduced hypothesis testing for quantitative data (mean, paired means, two independent means). In this section, we will cover some additional concepts associated with hypothesis testing: type I and II errors, significance level, and power.

15.2 Type I and Type II Errors

The decision we make from hypothesis testing is not going to be perfect, as we have inherent uncertainty in estimators due to random sampling. So we may sometimes reject a null hypothesis when it is true, or we may fail to reject a null hypothesis when the alternative hypothesis is true. These errors are called **type I** and **type II** errors respectively.

We go back to the university bookstore worked example in Section 14.2.5. The null and alternative hypotheses were $H_0 : \mu = 101.75, H_a : \mu > 101.75$.

- The type I error in this study is concluding that students pay more than \$101.75, on average, when the average amount paid is \$101.75.
- The type II error in this study is concluding that there is not enough evidence that students pay more than \$101.75, on average, when the average amount paid greater \$101.75.

Next, we discuss how we can measure or control the probability of making these errors.

15.3 Significance Level and Type I Error

The probability of making a type I error is controlled by the **significance level**, α, of the hypothesis test. The significance level is the probability of wrongly rejecting the null hypothesis when the null hypothesis is actually true. So if we conduct a hypothesis test with significance level $\alpha = 0.05$, we are saying that we are willing to have a 5% probability of making a type I error if the null hypothesis is true.

The value of the significance level for a hypothesis should be made in consultation with a subject matter expert and understanding the practical cost of making a type I error. From an ethical standpoint, this value should be decided upon before collecting and evaluating any data, as the significance level should not be adjusted in order to be able to conclude that the null hypothesis is rejected.

We will run a Monte Carlo simulation to demonstrate the significance level determines the probability of making a type I error. This simulation is very similar to the one that was done in Section 12.2.3.3 to explain coverage probabilities. You should realize that the probability of making a type I error is directly related to coverage probability which continues with the idea written in Section 14.5.1 about how confidence intervals and 2-sided hypothesis tests are consistent. The code below does the following:

- Simulate $X_1, \cdots, X_{10}$ i.i.d. from standard normal.
- Calculate a t statistic for the hypothesis test using equation (14.1).
  - The hypotheses will be $H_0 : \mu = 0, H_a : \mu \neq 0$.
  - Note that our data are simulated from a standard normal, so the null hypothesis is true in this simulation.
- Assess if the calculated t statistic is more extreme than the critical value.
  - We set the significance level to be $\alpha = 0.05$.
  - The critical value is $t^*_9 = 2.262157$ since we are conducting a 2-sided test with $n = 10$.
- Repeat these steps for a total of 10 thousand replicates.
- Count the number of times we reject the null hypothesis, and divide by the number of replicates. This value estimates the probability of making a type I error.
  - If we did things correctly the estimated probability of making a type I error should be close to the value of the significance level of 0.05.

```
## sig level
alpha <- 0.05

## function that simulates data from standard normal,
## then constructs test for mean, and assesses if test
## wrongly rejects H0
mean_type1 <- function(n, alpha){
```

```
  X <- rnorm(n) ##draw n data points from standard normal

  ##calculate elements needed for CI of mean
  xbar <- mean(X) ##sample mean
  SE <- sd(X)/sqrt(n) ##SE of sample mean
  t.stat <- xbar/SE
  crit <- qt(1-alpha/2, n-1) ##critical value

  ## check if reject H0
  reject <- abs(t.stat) > crit

  return(reject)

}

## write another function to replicate this simulation, and
## find the proportion of samples that wrongly reject H0
reps <- 10000

rep_mean_type1 <- function(n, alpha){

  result <- replicate(reps, mean_type1(n, alpha))
  return(mean(result))
}

## should be close to the value of the sig level, alpha,
## which is 0.05 in this simulation

set.seed(31)
rep_mean_type1(10, alpha)
```

```
## [1] 0.0531
```

The estimated probability of making a type I error is 0.0531, which matches the significance level which was set to be 0.05.

Thought question: Feel free to play around with the code and adjust the value of the significance level, change the alternative hypothesis to a 1-sided test, or any other factor. The estimated probability of making a type I error should always match the significance level.

If the type I error of a hypothesis test does not match its significance level, the test is considered unreliable.

15.4 Power and Type II Error

The **power** of a test is inversely related to the probability of making a type II error. The power of a test is its probability of being able to correctly reject the null alternative hypothesis, when the alternative hypothesis is true. Based on this definition, the power is the complement of the probability of making a type II error, i.e. power of a test is 1 minus the probability of making a type II error.

Calculating the power of a hypothesis test can be done by hand but is not usually a simple exercise. We will instead use Monte Carlo simulations to estimate the power of a hypothesis test.

Suppose we want to estimate the power of the hypothesis test for the mean if the data are i.i.d from $N(1,1)$ (and so has a population mean of 1), with sample size of 10, and significance level set to 0.05, and the hypotheses are $H_0 : \mu = 0, H_a : \mu \neq 0$. The code below does the following:

- Simulate $X_1, \cdots, X_{10}$ i.i.d. from $N(1,1)$.
- Calculate a t statistic for the hypothesis test using equation (14.1).
  - The hypotheses are $H_0 : \mu = 0, H_a : \mu \neq 0$.
  - Note that our data are simulated from $N(1,1)$, so the alternative hypothesis is true in this simulation.
- Assess if the calculated t statistic is more extreme than the critical value.
  - We set the significance level to be $\alpha = 0.05$.
  - The critical value is $t^*_9 = 2.262157$ since we are conducting a 2-sided test with $n = 10$.
- Repeat these steps for a total of 10 thousand replicates.
- Count the number of times we reject the null hypothesis, and divide by the number of replicates. This value estimates the probability of correctly rejecting the null hypothesis, i.e. the power of this test.

```
## sig level
alpha <- 0.05

## function that simulates data from N(1,1),
## then constructs test for mean, and assesses if test
## correctly rejects H0
mean_power <- function(n, alpha){

  X <- rnorm(n, mean = 1, sd = 1) ##draw n data points N(1,1)
  ##calculate elements needed for CI of mean
  xbar <- mean(X) ##sample mean
  SE <- sd(X)/sqrt(n) ##SE of sample mean
  t.stat <- xbar/SE
  crit <- qt(1-alpha/2, n-1) ##critical value

  ## check if reject H0
  reject <- abs(t.stat) > crit
```

```
  return(reject)

}

## write another function to replicate this simulation, and
## find the proportion of samples that wrongly reject H0
reps <- 10000

rep_mean_power <- function(n, alpha){

  result <- replicate(reps, mean_power(n, alpha))
  return(mean(result))
}

## should be close to the value of the sig level, alpha,
## which is 0.05 in this simulation

set.seed(31)
rep_mean_power(10, alpha)
```

```
## [1] 0.8003
```

The estimated power of this test is 0.8003. So if the data come from a $N(1, 1)$, the power of testing $H_0 : \mu = 0, H_a : \mu \neq 0$ using a sample of size 10 and significance level of 0.05 is 0.8003.

Note: we must specify a specific value for the parameter that makes the alternative hypothesis true. In this simulation, we simulated data from a distribution with population mean 1. The value of the power will change if we used another value for the mean of the distribution. This value directly affects the effect size. The power is specific to this value for the alternative hypothesis, as well as to the specific values of n and α.

Power analysis or power calculations are often done in well-designed experiments to determine what minimum sample size is needed for a hypothesis test to guarantee the hypothesis test achieves a certain power with a pre-determined significance level and specific alternative hypothesis (which impacts the effect size).

Thought question: How would you tweak the code above to do a power analysis to find the minimum sample size needed to have an estimated power of at least 0.90 (assuming every other factor remains the same)?

15.4.1 Factors Affecting Power

Ideally, we would like the power of our hypothesis test to be large. So what are the factors of the power of a hypothesis test? Recall that power measures the

probability of being able to correctly reject a null hypothesis, and that larger values (in magnitude) of a test statistic provide more evidence against the null hypothesis. So the factors that make a test statistic larger (in magnitude) also lead to an increase in the power of a hypothesis test. As mentioned in Section 14.2.2.1, these are:

- The effect size. There are a couple of definitions of effect size:
  1. The difference between the true value of the parameter and the value of the parameter under the null hypothesis.
  2. A "standardized" difference between the true value of the parameter and the value of the parameter under the null hypothesis. For a 1-sample t-test, this is $\frac{\mu_x - \mu_0}{s}$. For a 2-sample t-test, this is $\frac{(\mu_x - \mu_y) - (\text{diff in null})}{\sqrt{s_x^2 + s_y^2}}$. General guidelines for interpretating the effect size when defined this way:
     - 0.2: small effect.
     - 0.5: moderate effect.
     - 0.8: large effect.

In either definition, larger effect sizes lead to larger test statistics, in magnitude. This should make sense since a larger effect size implies the sample mean is likely to deviate more from the null value if the null hypothesis is true.

- The sample size n. Larger sample sizes lead to larger magnitudes for the test statistic.
- The standard error of the estimator. Smaller standard errors of the estimator lead to larger magnitudes for the test statistic.
- Another factor that increases the power of a hypothesis test is to raise the significance level α. Think of the following extreme example: Suppose we set the significance level to be as low as possible, $\alpha = 0$. Then the hypothesis test will always not reject the null hypothesis, and we never make a type I error. But always not rejecting the null hypothesis means that if the alternative hypothesis is true, we will always make a type II error, so the power will be 0. This illustrates the probabilities of making a type I and type II error have an inverse relationship. Reducing one is done at the expense of the other.

15.4.2 Worked Examples

We will use a couple of functions from the `pwr` package to compute power. These functions are useful if you are working in terms of standardized effect size.

1. We go back to the example in Section 14.2.5. A university bookstore claims that, on average, a student will pay $101.75 per class for textbooks. The student newspaper suspects that the claimed average is too low and investigates by randomly selecting 9 courses from the current semester

offerings and finding the textbook costs for each. The students conducting the investigation reason that with the collection of courses being offered courses, the textbook prices are normally distributed.

- What is the effect size based on the sample?
- What is the power of their planned test to identify a significant difference when the true cost per textbook is moderately higher?
- What is the minimum sample size to attain a power of at least 0.80 with moderate effect size?

From the question, we have $H_0 : \mu = 101.75, H_a : \mu > 101.75$.

```
## data
news.samp <- c(140, 125, 150, 87, 170, 195, 94, 127, 53)

## effect size
efs <- (mean(news.samp) - 101.75)/sd(news.samp)
efs
```

```
## [1] 0.5715588
```

```
library(pwr)

## power from pwr package
pow <- pwr.t.test(d = 0.5, n = 9, sig.level = 0.05,
                  type = "one.sample",
                  alternative = "greater")$power
pow
```

```
## [1] 0.3927739
```

```
## Determine the sample size required for power of 0.8
## need to round up to whole number
min.n <- ceiling(pwr.t.test(d = 0.5, sig.level = 0.05,
                            power = 0.8, type = "one.sample",
                            alternative = "greater")$n)
min.n
```

```
## [1] 27
```

The effect size is 0.5715588.

The power with moderate effect size is 0.3927739.

The minimum sample size for moderate effect size with at least 0.80 power is 27.

2. A weight study was conducted on 307 sets of twins in Australia over the age of 50 to determine if there is evidence that identical twins have significantly different weights. What is the power if the effect size is small?

```
## Calculate the power
pow2 <- pwr.t.test(d = 0.2, n = 307, sig.level = 0.05,
                   type = "paired", alternative = "two.sided")$power
pow2
```

```
## [1] 0.937399
```

```
## Determine the sample size required for power of 0.8
min.n2 <- ceiling(pwr.t.test(d = 0.2, sig.level = 0.05,
                     power = 0.8, type = "paired",
                     alternative = "two.sided")$n)
min.n2
```

```
## [1] 199
```

The power with small effect size is 0.937399.

The minimum sample size for small effect size with at least 0.80 power is 199.

3. A biologist is interested in whether different species of iris have different sepal widths. He is particularly interested in whether the Versicolor and Virginica species are significantly different. With random samples of 150 flowers from each species, what is the power of the biologist's test when the effect size is small?

```
## Calculate the power
pow3 <- pwr.t.test(d = 0.2, n = 150, sig.level = 0.05,
                   type = "two.sample",
                   alternative = "two.sided")$power
pow3
```

```
## [1] 0.4078021
```

```
## Determine the sample size required for power of 0.8
min.n3 <- ceiling(pwr.t.test(d = 0.2, sig.level = 0.05,
                             power = 0.8, type = "two.sample",
                             alternative = "two.sided")$n)
min.n3
```

```
## [1] 394
```

The power with small effect size is 0.4078021.

The minimum sample size for small effect size with at least 0.80 power is 394.

Note that the `pwr.t.test()` function assumes equal sample sizes when applied to a 2-sample t-test. The `pwr.2t.test()` function should be used when sample sizes are not the same.

4. The biologist conducting the experiment to identify differences in sepal widths between iris species has planted the irises, but 5 of the Versicolor

plants have died as have 9 of the virginica plants. What is the power of the biologist's updated test when the effect size is small?

```
## Calculate the power
pow4 <- pwr.t2n.test(d = 0.2, n1 = 145, n2 = 141,
                     sig.level = 0.05,
                     alternative = "two.sided")$power
pow4
```

```
## [1] 0.3919092
```

The power with small effect size is 0.3919092.

5. The biologist decides that it is possible to plant additional virginica irises for the study. How many more should the biologist plant for 50% power?

```
## Determine the sample size required for power of 0.5
min.n5 <- ceiling(pwr.t2n.test(d = 0.2, n1 = 145, sig.level = 0.05, power = 0.5, alternative = "t
min.n5 - 141
```

```
## [1] 148
```

The biologist needs to plant 148 more virginica irises for the study to achieve 0.50 power.

15.5 Statistical Vs Practical Significance

However, even with a p-value being reported, we need to be careful with conflating a "statistically significant" result (i.e. null hypothesis is rejected) with a "practically significant" result.

We had earlier mentioned in Section 15.4 that the power of a hypothesis test can be increased when the sample size gets larger. This implies that we may be able to reject the null hypothesis when we have a small effect size but we have a huge sample size.

Consider this hypothetical example. We have $H_0 : \mu = 100, H_a : \mu \neq 100$, with $\bar{x} = 101, \sigma = 10$, and various values for the sample size n. Suppose these values represent IQ scores. We work out the p-values as well as the corresponding 95% confidence intervals for the mean, and present the results in Table 15.1 below.

Table 15.1: Sample Size on P-Values and CIs

n	p-value	95% CI
25	0.617	(97.080, 104.920)
350	0.061	(99.952, 102.408)
400	0.455	(100.020, 101.980)
2,500	5.7×10^{-7}	(100.608, 101.392)

Most would probably agree that the effect size in this example is small, yet for a sample size of 400 or more, we will reject the null hypothesis. When the sample size is huge, the p-value is extremely small, which we may interpret as having strong evidence against the null hypothesis, so we have "statistical significance". On the other hand, the small effect size tells us that the practical significance may be of little consequence.

A couple of recommendations when reporting results from a hypothesis test:

- If possible, report the corresponding confidence interval, so the reader can see what is the range of plausible values for the parameter and make their own decision. However, not every hypothesis test has an associated confidence interval; we will see one such class of hypothesis tests in the next subsection.
- Researchers need to establish the minimum effect size for a result to be considered practically significant. From an ethical standpoint, this value should be decided upon before collecting and evaluating any data, as the this value should not be adjusted in order to be able to conclude that the null hypothesis is rejected.

To remind ourselves, we use hypothesis tests to test for "statistical significance". What we are testing is whether the deviation of our sample estimate from its value if the null hypothesis is true can be attributed to random sampling alone or not. If we reject the null hypothesis, the deviation is unlikely to be attributed to random sampling alone.

To summarize, hypothesis testing is used to assess "statistical significance", but we use subject matter knowledge to assess "practical significance".

Bibliography

Bonnell, J. and Ogihara, M. (2024). *Exploring Data Science with R and the Tidyverse*. Chapman and Hall/CRC, 1st edition. ISBN 978-1032341705.

Claerbout, F. and Karrenbach, M. (1992). Electronic documents give reproducible research a new meaning. In *SEG Technical Program Expanded Abstracts 1992*.

Horst, A., Hill, A., and Gorman, K. (2020). *palmerpenguins: Palmer Archipelago (Antarctica) penguin data*. R package version 0.1.0.

Peng, R. (2015). *R Programming for Data Science*. Lulu.com, 5th edition. ISBN 978-1365056826.

Wickham, H., Cetinkaya-Rundel, M., and Grolemun, G. (2023). *R for Data Science*. O'Reilly, 2nd edition. ISBN 978-1492097402.

Xie, Y., Dervieux, C., and Riederer, E. (2020). *R Markdown Cookbook*. Chapman and Hall/CRC, Boca Raton, Florida. ISBN 978-0367563837.